

# UNIDAD III: Gestión de la seguridad en la Arquitectura TI

---

## Índice de Contenido

- [Introducción](#)
- [Desarrollo de Contenidos](#)
  - [Arquitectura TI](#)
  - [Sistemas de Información](#)
  - [Bases de Datos](#)
  - [Sistemas Operativos](#)
- [Autoevaluación](#)
- [Bibliografía y Webgrafía](#)
- [Glosario](#)

## Introducción

## Desarrollo de Contenidos

### Arquitectura TI

La **Arquitectura TI** es la estructura conceptual que define la organización, el comportamiento y la evolución de los sistemas de información de una empresa. En el contexto de la seguridad, una arquitectura bien diseñada no es un lujo, sino una necesidad: permite implementar controles de manera consistente, escalable y alineada con los objetivos del negocio. Así como un edificio mal diseñado se derrumba con el primer temblor, un sistema mal arquitecturado colapsa ante el primer ataque.

Esta sección se conecta directamente con las unidades anteriores. En la **Unidad I** aprendimos a evaluar la viabilidad técnica de un proyecto; la arquitectura es el principal determinante de esa viabilidad. En la **Unidad II** planificamos e ingeniamos el proceso; la arquitectura es el "plano" que guía la construcción. Ahora, en la Unidad III, la abordamos desde la óptica de la seguridad, porque una arquitectura insegura es una sentencia de fracaso a largo plazo.

Como dijo el experto en seguridad Andrew S. Townley: *"The value proposition of security architecture is simple. If you have a security architecture and you're able to understand how that architecture enables and supports achieving the objectives that you want, it gives you, as the owner of those objectives, confidence that you're really going to get them"*. La arquitectura de seguridad no es un fin en sí mismo, sino un medio para alcanzar los objetivos del negocio con confianza.

#### Recuadro de definiciones clave:

| Término                | Definición simple                                                                                                                        |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Arquitectura TI</b> | Estructura que define cómo se organizan, comunican y evolucionan los sistemas de información de una organización.                        |
| <b>DMZ</b>             | Zona de red de amortiguamiento entre Internet y la red interna, donde se colocan servidores públicos.                                    |
| <b>STRIDE</b>          | Marco de Microsoft para clasificar amenazas en seis categorías: Suplantación, Manipulación, Repudio, Revelación, Denegación y Elevación. |
| <b>Zero Trust</b>      | Modelo de seguridad que no confía en nada por defecto y verifica continuamente cada acceso.                                              |
| <b>Microservicio</b>   | Arquitectura donde una aplicación se divide en servicios pequeños, independientes y desplegables por separado.                           |

#### Mapa conceptual de la sección

Para ayudarte a visualizar el panorama completo, aquí tienes un mapa conceptual de lo que abordaremos:

???text

Arquitectura TI

|  |                                                         |
|--|---------------------------------------------------------|
|  |                                                         |
|  | — Definición y tipos                                    |
|  | — Arquitectura Empresarial (EA)                         |
|  | — Arquitectura de Soluciones                            |
|  | — Arquitectura de Infraestructura                       |
|  |                                                         |
|  | — Frameworks de arquitectura                            |
|  | — TOGAF 10                                              |
|  | — Zachman                                               |
|  | — SABSA                                                 |
|  |                                                         |
|  | — Principios de diseño seguro                           |
|  | — Defensa en profundidad                                |
|  | — Privilegio mínimo                                     |
|  | — Seguridad por defecto                                 |
|  | — Separación de funciones                               |
|  | — Registro y auditoría                                  |
|  | — Cifrado generalizado                                  |
|  | — Zero Trust                                            |
|  |                                                         |
|  | — Modelado de amenazas: STRIDE                          |
|  |                                                         |
|  | — Arquitectura de red segura                            |
|  | — DMZ                                                   |
|  | — VLANs                                                 |
|  | — Zonas de confianza                                    |
|  |                                                         |
|  | — Arquitectura en la nube                               |
|  | — Modelo de responsabilidad compartida                  |
|  |                                                         |
|  | — Arquitectura de datos y seguridad en APIs             |
|  |                                                         |
|  | — Aplicación práctica: Microservicios                   |
|  |                                                         |
|  | — Trade-offs arquitectónicos                            |
|  |                                                         |
|  | — Ciclo de vida de la arquitectura y gestión de cambios |
|  |                                                         |
|  | — Métricas de calidad                                   |
|  |                                                         |
|  | — Cronograma y presupuesto                              |
|  |                                                         |
|  | — Herramientas para el diseño                           |
|  |                                                         |
|  | — El rol del ingeniero                                  |
|  |                                                         |
|  | — Caso integrador                                       |
|  |                                                         |
|  | — Comprobación de aprendizaje                           |
|  | ???                                                     |

**Pregunta guía antes de comenzar:** Antes de leer, piensa: ¿qué crees que significa "arquitectura de seguridad"? ¿Crees que es lo mismo que tener un firewall o un antivirus? ¿O es algo más amplio?

---

## Definición y tipos de arquitectura TI

Existen tres niveles principales de arquitectura TI que todo ingeniero en sistemas debe conocer. Cada uno responde a una pregunta clave que guía la toma de decisiones.

| Tipo de arquitectura                   | Enfoque                                                                                                                                     | Pregunta clave que responde                                                | Ejemplo                                                           | Implicaciones de seguridad                                                                                        |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Arquitectura Empresarial (EA)</b>   | Alinea la TI con los objetivos estratégicos del negocio. Define los procesos, datos, aplicaciones e infraestructura a nivel organizacional. | <i>¿Qué tecnología necesitamos para cumplir la estrategia del negocio?</i> | Marco TOGAF aplicado a una empresa de comercio electrónico        | Define políticas de seguridad a nivel organizacional; ej. "todos los datos de clientes deben cifrarse en reposo". |
| <b>Arquitectura de Soluciones</b>      | Define la estructura de un sistema o solución específica, detallando componentes, interacciones y tecnologías.                              | <i>¿Cómo construimos este sistema en particular de forma segura?</i>       | Arquitectura de microservicios para una plataforma de reservas    | Define controles a nivel de cada componente: autenticación entre servicios, cifrado de comunicaciones, etc.       |
| <b>Arquitectura de Infraestructura</b> | Describe los componentes físicos y virtuales (servidores, redes, almacenamiento) y cómo se interconectan.                                   | <i>¿Dónde y cómo se alojan los sistemas?</i>                               | Topología de red con segmentación VLAN, firewalls y balanceadores | Define perímetros de seguridad (DMZ), reglas de firewall, segmentación de red.                                    |

### Frameworks de arquitectura empresarial

Los frameworks proporcionan metodologías estandarizadas para diseñar y gobernar la arquitectura TI. Los más relevantes para la seguridad son:

| Framework                                                      | Organización    | Enfoque en seguridad                                                                                                                                                                                                                                                  | Aplicación práctica                                                                                                                                                             |
|----------------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TOGAF</b> (The Open Group Architecture Framework)           | The Open Group  | Incluye el dominio de "Seguridad" dentro de la arquitectura; su método ADM (Architecture Development Method) integra la seguridad en cada fase. La versión <b>TOGAF 10</b> (2022) refuerza el enfoque en agilidad y arquitectura nativa de nube.                      | Una empresa que implementa TOGAF debe definir principios de seguridad en la fase Preliminar y arquitectura de seguridad en la fase C (Arquitectura de Sistemas de Información). |
| <b>Zachman Framework</b>                                       | John Zachman    | No prescribe cómo hacer la arquitectura, pero proporciona una taxonomía que obliga a considerar la seguridad desde múltiples perspectivas (datos, función, red, personas, tiempo, motivación).                                                                        | Útil para inventariar y relacionar todos los activos de seguridad de la organización.                                                                                           |
| <b>SABSA</b> (Sherwood Applied Business Security Architecture) | SABSA Institute | Framework específico de arquitectura de seguridad. Extiende Zachman con un enfoque en gestión de riesgos y seguridad de negocio. Su enfoque en capas (contextual, conceptual, lógica, física, etc.) lo hace ideal para diseñar arquitecturas de seguridad desde cero. | Ideal para diseñar la arquitectura de seguridad desde cero, integrando controles técnicos, organizativos y legales.                                                             |

### Profundizando en TOGAF 10:

La versión 10 de TOGAF representa un cambio significativo en cómo se aborda la seguridad. Como señala la documentación oficial: *"Security Architecture is a critical component of enterprise architecture that ensures organizational assets, information, and systems are protected against threats and vulnerabilities. In the context of the TOGAF Architecture Development Method (ADM), security must be integrated throughout all phases of architectural development, not treated as an afterthought"*.

En la práctica, esto significa que la seguridad no es una fase separada, sino una consideración continua a lo largo de todo el ciclo de desarrollo arquitectónico. Durante la Fase Preliminar se establecen los marcos de gobierno de seguridad; en la Fase A (Visión

de Arquitectura) se definen los requisitos y objetivos de seguridad basados en los impulsores del negocio; y en las Fases B, C y D (Arquitectura de Negocio, Sistemas de Información y Tecnología) la seguridad se incrusta en cada dominio arquitectónico.

### Profundizando en SABSA:

El framework SABSA, desarrollado por John Sherwood, se basa en el principio de que la seguridad no existe como un fin en sí mismo, sino al servicio del negocio. Como explica la documentación de SABSA: *"It's founded on the recognition that security in an enterprise – and thereby the security architectures that support the security function of that enterprise – do not exist as an end goal in and of themselves. Instead, they exist solely in service of – and to enable – the business"*.

SABSA utiliza una matriz con seis capas de abstracción (Contextual, Conceptual, Lógica, Física, Componente y Gestión de Servicios) y seis interrogantes (Qué, Por qué, Cómo, Quién, Dónde, Cuándo) para garantizar que todos los aspectos de la seguridad sean considerados. Este enfoque integral asegura que la arquitectura de seguridad sea trazable desde los objetivos de negocio hasta los controles técnicos concretos.

---

## Principios de diseño seguro en la arquitectura TI

Independientemente del framework utilizado, existen principios universales de diseño seguro que deben aplicarse en toda arquitectura TI:

**1. Defensa en profundidad (Defense in Depth):** No confiar en un solo control de seguridad. Superponer múltiples capas de defensa (firewall perimetral, firewall interno, IDS/IPS, autenticación multifactor, cifrado, etc.) para que si una capa falla, la siguiente la contenga.

Como señaló Durga Prasad Dube, CISO de Reliance Industries: *"The true art of defense is maintaining resilience... Victory is achieved when your defenses are so strong that the attacker chooses not to attack"*. La defensa en profundidad no se trata de comprar más herramientas, sino de diseñar sistemas resilientes que disuadan al atacante.

**2. Privilegio mínimo (Least Privilege):** Cada usuario, proceso o sistema debe tener solo los permisos mínimos necesarios para realizar su función. A nivel de arquitectura, esto implica segmentar redes, restringir comunicaciones entre servicios y aplicar RBAC.

**3. Seguridad por defecto (Secure by Default):** Las configuraciones predeterminadas deben ser las más seguras posibles.

**Ejemplo de mala práctica:** Un servidor web que expone su versión y lista de directorios. **Ejemplo de buena práctica:** Un servidor Linux que viene con firewall activo (ufw) y puertos cerrados por defecto. Otro ejemplo común: cambiar las contraseñas por defecto de routers y equipos de red antes de ponerlos en producción.

**4. Separación de funciones (Segregation of Duties):** Las funciones críticas deben dividirse entre diferentes personas o sistemas para evitar conflictos de interés y fraudes. Ejemplo: quien autoriza un cambio en la base de datos no debe ser quien lo ejecuta.

**5. Registro y auditoría (Logging and Auditing):** Toda acción relevante en el sistema debe quedar registrada con suficiente detalle (quién, qué, cuándo, desde dónde) para permitir auditorías forenses.

**6. Cifrado generalizado (Encrypt Everything):** Los datos deben cifrarse en reposo (en bases de datos, discos) y en tránsito (TLS/HTTPS entre servicios).

**7. Zero Trust Architecture:** Este principio moderno asume que ningún usuario, dispositivo o red es confiable por defecto, incluso si están dentro del perímetro de la organización. Requiere verificación continua de identidad y autorización para cada acceso.

---

## Zero Trust Architecture: fundamentos y pilares

El término **Zero Trust** fue acuñado por John Kindervag, entonces analista de Forrester, en 2009. Kindervag postuló que la confianza era una vulnerabilidad y que la seguridad debería seguir una estrategia de *"Never trust, always verify"*. Como explica el documento de NIST SP 800-207: *"Zero Trust is the term for an evolving set of network security paradigms that move network defenses from static, network-based perimeters to focus on users, assets, and resources"*.

En esencia, una arquitectura Zero Trust implica que **no hay confianza implícita** otorgada a los sistemas basada en su ubicación física o de red (red local vs. Internet). La organización debe asumir que no hay confianza implícita y analizar y evaluar continuamente los riesgos para sus activos y funciones comerciales.

## Los tres principios fundamentales de Zero Trust según el SEI-CMU:

1. **Assume breach (Asumir la brecha):** Las organizaciones deben asumir en todo momento que hay una presencia maliciosa dentro de su entorno e implementar controles de seguridad para minimizar el impacto.
2. **Verify, don't trust (Verificar, no confiar):** En lugar de asumir la legitimidad, las organizaciones deben verificar continuamente todos los componentes de su infraestructura de TI para asegurarse de que no han sido comprometidos.
3. **Least privilege (Mínimo privilegio):** Una vez verificados, los usuarios, dispositivos y servicios deben recibir el mínimo acceso posible necesario para completar su función, y durante el período más corto posible.

## Los siete pilares de Zero Trust según NIST SP 800-207:

NIST SP 800-207 proporciona siete principios rectores para implementar Zero Trust:

| Principio                                                                          | Descripción                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Todos los recursos son considerados recursos                                    | Todas las fuentes de datos y servicios de computación son recursos que deben ser protegidos.                                                                                                                |
| 2. Toda la comunicación es segura                                                  | Toda la comunicación debe ser segura, independientemente de la ubicación de la red.                                                                                                                         |
| 3. El acceso a los recursos se otorga por sesión                                   | El acceso a los recursos se otorga por sesión individual y se evalúa dinámicamente.                                                                                                                         |
| 4. El acceso a los recursos se determina por política                              | El acceso a los recursos se determina mediante políticas dinámicas que consideran el estado del dispositivo, la identidad del usuario y otros atributos.                                                    |
| 5. La organización monitorea y mide la integridad y seguridad de todos los activos | La organización monitorea y mide continuamente la integridad y la postura de seguridad de todos los activos.                                                                                                |
| 6. Toda la autenticación y autorización es dinámica y estrictamente aplicada       | Toda la autenticación y autorización es dinámica y se aplica de manera estricta antes de conceder el acceso.                                                                                                |
| 7. La organización recopila la mayor cantidad de información posible               | La organización recopila la mayor cantidad de información posible sobre el estado actual de los activos, la infraestructura de red y las comunicaciones, y la utiliza para mejorar la postura de seguridad. |

## Modelado de amenazas en la arquitectura: STRIDE

Una arquitectura segura no se construye solo con buenas intenciones; debe ser **diseñada para resistir ataques específicos**. El modelado de amenazas es la técnica que permite identificar proactivamente las amenazas potenciales en el diseño arquitectónico. El marco **STRIDE**, creado por Microsoft, es el más utilizado para clasificar amenazas en seis categorías.

Como explica la documentación de Microsoft: *"One way to ensure your applications have these properties is to employ threat modeling using STRIDE, an acronym for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege"*. El modelado de amenazas es un paso clave en el ciclo de vida de cualquier software.

Aplicar STRIDE en la fase de diseño arquitectónico permite anticipar ataques y definir controles antes de escribir una sola línea de código.

| Categoría STRIDE         | ¿Qué busca?                         | Ejemplo concreto en la arquitectura                                                 | Control arquitectónico recomendado                                                                                      |
|--------------------------|-------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Spoofing (suplantación)  | Alguien se hace pasar por otro      | Un atacante usa credenciales robadas de un microservicio para comunicarse con otro. | TLS mutuo (mTLS) entre servicios; autenticación basada en tokens JWT con corta expiración.                              |
| Tampering (manipulación) | Modificación no autorizada de datos | Un empleado altera el monto de una factura antes de enviarla a la base de datos.    | Cifrado de extremo a extremo; firmas digitales en transacciones críticas; integridad de datos a nivel de base de datos. |

| Categoría STRIDE                    | ¿Qué busca?                               | Ejemplo concreto en la arquitectura                                                           | Control arquitectónico recomendado                                                                                                          |
|-------------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Repudiation (repudio)               | Negar haber realizado una acción          | Un usuario afirma no haber enviado un correo que sí envió.                                    | Logs de auditoría centralizados (SIEM) con sellado de tiempo y firmas; registros inmutables en blockchain o bases de datos append-only.     |
| Information Disclosure (revelación) | Fuga de información confidencial          | Un bucket S3 mal configurado expone datos de clientes.                                        | Cifrado en reposo y en tránsito; controles de acceso granulares (IAM); políticas de bucket privadas por defecto.                            |
| Denial of Service (denegación)      | Indisponibilidad del sistema              | Un ataque DDoS satura la pasarela de pagos impidiendo ventas.                                 | Rate limiting; Web Application Firewall (WAF); arquitectura escalable (auto-scaling); CDN para absorber tráfico.                            |
| Elevation of Privilege (elevación)  | Obtener permisos superiores a los debidos | Un atacante pasa de usuario normal a administrador del sistema explotando una vulnerabilidad. | Principio de mínimo privilegio en contenedores y bases de datos; separación de cuentas de servicio; actualizaciones de seguridad oportunas. |

**Ejercicio de modelado:** Al diseñar el módulo de autenticación de un sistema, pregúntate: ¿Cómo evitamos que alguien suplante a otro usuario (S)? ¿Cómo aseguramos que la sesión no sea manipulada (T)? ¿Cómo registramos quién inició sesión para que no pueda repudiarlo (R)? ¿Qué pasa si un atacante intenta 10.000 inicios de sesión por minuto (D)? ¿Cómo evitamos que un usuario estándar se convierta en administrador (E)?

## Arquitectura de red segura

La red es la columna vertebral de la infraestructura. Una arquitectura de red mal diseñada permite a un atacante moverse lateralmente después de comprometer un solo punto. La segmentación es la clave.

### Conceptos fundamentales:

- **DMZ (Zona Desmilitarizada):** Subred que actúa como zona de amortiguamiento entre la red interna (confiable) y la red externa (no confiable, como Internet). Alberga servidores que deben ser accesibles desde Internet (web, correo, DNS).
- **VLANs (Redes Locales Virtuales):** Permiten dividir lógicamente la red en segmentos aislados, incluso si comparten el mismo cableado físico.
- **Zonas de confianza:** Clasificar los activos por nivel de criticidad y aislarlos en zonas (ej. Zona Roja = datos financieros, Zona Amarilla = aplicaciones, Zona Verde = acceso público).

### Diagrama de arquitectura de red segura (defensa en profundidad):

```

???mermaid
graph TB
  subgraph "Internet"
    A[Usuarios / Atacantes]
  end
  subgraph "Perímetro"
    B[Firewall Perimetral / WAF]
  end
  subgraph "DMZ"
    C[Servidores Web Apache/Nginx]
    D[Servidor de Correo]
  end
  subgraph "Red Interna"
    E[Firewall Interno]
  end
  subgraph "VLAN Aplicaciones"
    F[Servidor de Aplicaciones]
  end

```

```

Node.js / PHP]
end
subgraph "VLAN Datos"
G[Servidor Base de Datos
Cifrada]
end
subgraph "VLAN Administración"
H[Servidor de Monitoreo
SIEM / Wazuh]
end
end
subgraph "Capa de Respaldo"
I[Sistema de Backups
Offsite]
end

```

```

A -->|HTTPS| B
B -->|Puertos 80/443| C
C -->|Proxy reverso / API| E
E -->|Solo puerto 3000| F
F -->|Puerto 3306<br/>solo desde IP de app| G
F -.->|Logs| H
G -.->|Logs| H
G -->|Respaldo cifrado| I

```

???

*Legenda:* Las líneas sólidas representan flujo de datos autorizado y controlado por firewalls. Las líneas punteadas representan flujo de logs. La segmentación por VLANs impide que un atacante que comprometa el servidor web (DMZ) acceda directamente a la base de datos.

## Arquitectura en la nube y responsabilidad compartida

Con la migración masiva a la nube, es crucial entender el **modelo de responsabilidad compartida**. El proveedor de nube (AWS, Azure, GCP) es responsable de la *seguridad de la nube* (hardware, red física, hipervisor), mientras que el cliente es responsable de la *seguridad en la nube* (configuración, datos, acceso, parches de SO en IaaS).

| Nivel de servicio                           | Responsabilidad del proveedor                     | Responsabilidad del cliente                                               |
|---------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| <b>IaaS</b> (Infraestructura como Servicio) | Virtualización, almacenamiento físico, red        | SO, aplicaciones, datos, firewalls internos, parches del SO               |
| <b>PaaS</b> (Plataforma como Servicio)      | SO, middleware, runtime                           | Aplicaciones, datos, control de acceso a la plataforma                    |
| <b>SaaS</b> (Software como Servicio)        | Toda la pila (aplicación, datos, infraestructura) | Configuración de usuarios, políticas de acceso, integración con identidad |

**Implicación arquitectónica:** Al diseñar una solución en la nube, el ingeniero debe definir claramente qué controles de seguridad implementa el proveedor y cuáles debe implementar él mismo. Por ejemplo, si se usa un RDS (Base de Datos como Servicio), el proveedor se encarga del parcheo del motor de BD, pero el cliente debe configurar grupos de seguridad (firewall), habilitar cifrado en reposo y gestionar las credenciales de acceso.

## Arquitectura de datos y seguridad en APIs

En las arquitecturas modernas, los datos fluyen entre múltiples servicios a través de APIs. La seguridad de las APIs es la seguridad de los datos.

- **Arquitectura de datos:** Define cómo se almacenan, procesan e integran los datos. Implica clasificación de datos (público, interno, confidencial, restringido), flujos de datos (origen, transformación, destino) y políticas de retención.
- **Seguridad en APIs:**
  - **Autenticación y Autorización:** Uso de OAuth2.0 / OpenID Connect para delegar autenticación; JWT (JSON Web Tokens) con firmas robustas (RS256) y corta expiración.
  - **Rate Limiting:** Limitar el número de peticiones por IP o usuario para prevenir ataques de fuerza bruta y DoS.
  - **Validación de entrada:** Usar esquemas estrictos (JSON Schema) para validar el formato y tipo de datos; sanitizar entradas para evitar inyecciones (SQL, NoSQL, Command).
  - **Cifrado en tránsito:** Forzar TLS 1.3 para todas las comunicaciones API.

**Ejemplo de mal diseño:** Una API que expone un endpoint `/users/{id}` sin verificar que el usuario autenticado tenga permiso para ver ese `id`. Esto es un Broken Access Control (A01 de OWASP), evitable con una arquitectura que centralice la autorización.

---

### Trade-offs arquitectónicos: decisiones difíciles en la vida real

En la práctica, el arquitecto de seguridad debe tomar decisiones difíciles que implican compensaciones. Estas son algunas de las más comunes:

| Decisión                         | Opción A                                                                                          | Opción B                                                                       | Trade-off                                                                                                                |
|----------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Seguridad vs. Rendimiento</b> | Cifrar todos los datos, incluyendo logs y métricas.                                               | Cifrar solo datos sensibles; logs en texto plano para facilitar debugging.     | Cifrar todo aumenta la seguridad pero puede degradar el rendimiento y complicar el análisis de logs.                     |
| <b>Costo vs. Protección</b>      | Implementar una arquitectura de microservicios con mTLS, WAF y SIEM.                              | Usar una arquitectura monolítica simple con firewall básico.                   | La primera es más segura pero mucho más costosa de implementar y mantener.                                               |
| <b>Usabilidad vs. Control</b>    | Autenticación con SSO (Single Sign-On) y MFA solo para accesos administrativos.                   | MFA obligatoria para todos los usuarios, incluyendo clientes.                  | La MFA universal es más segura pero puede frustrar a los usuarios y reducir la conversión.                               |
| <b>Agilidad vs. Estabilidad</b>  | Adoptar la última versión de cada componente para tener las últimas características de seguridad. | Usar versiones LTS (Long Term Support) con parches de seguridad backporteados. | La primera es más ágil pero puede introducir inestabilidad; la segunda es más estable pero puede perder funcionalidades. |

### Ejemplo concreto de trade-off en una PYME nicaragüense:

Una distribuidora de alimentos en Managua debe decidir entre:

- **Opción A:** Contratar un servicio de seguridad gestionado (MSSP) que cuesta C\$ 25,000 mensuales pero incluye monitoreo 24/7 y respuesta a incidentes.
- **Opción B:** Implementar herramientas open source (Wazuh, OSSEC, Fail2ban) con un costo de implementación de C\$ 50,000 y un costo operativo de C\$ 5,000 mensuales (personal de TI dedicado).

El trade-off es claro: la Opción A es más cara pero requiere menos esfuerzo interno; la Opción B es más barata a largo plazo pero requiere personal capacitado y dedicado. La decisión dependerá de la disponibilidad de talento y del presupuesto de la empresa.

---

### Ciclo de vida de la arquitectura y gestión de cambios

La arquitectura no es estática; evoluciona a lo largo del ciclo de vida del proyecto. Un arquitecto debe gestionar esta evolución de manera controlada.

### Fases del ciclo de vida de la arquitectura:

| Fase del proyecto          | Actividad de arquitectura                                                                              | Entregable                                           |
|----------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <b>Inicio</b>              | Definir principios arquitectónicos de alto nivel y requisitos de seguridad.                            | Documento de visión arquitectónica.                  |
| <b>Planificación</b>       | Diseñar la arquitectura detallada, incluyendo componentes, interfaces y tecnologías.                   | Especificación de arquitectura.                      |
| <b>Ejecución</b>           | Implementar la arquitectura según el diseño; resolver problemas de integración.                        | Arquitectura implementada (código, infraestructura). |
| <b>Monitoreo y control</b> | Verificar que la arquitectura cumple con los requisitos de seguridad y rendimiento; gestionar cambios. | Reportes de cumplimiento; solicitudes de cambio.     |
| <b>Cierre</b>              | Documentar la arquitectura final y las lecciones aprendidas.                                           | Documentación de arquitectura final.                 |

### Gestión de cambios en arquitectura:

Cuando un requisito nuevo obliga a rediseñar la arquitectura, se debe seguir un proceso controlado:

1. **Solicitud de cambio:** Documentar el cambio propuesto, su justificación y el impacto estimado.
2. **Evaluación:** Analizar el impacto en costo, tiempo, seguridad y rendimiento.
3. **Aprobación:** Obtener la aprobación del comité de arquitectura o del patrocinador.
4. **Implementación:** Ejecutar el cambio siguiendo un plan de migración.
5. **Verificación:** Validar que el cambio funcionó como se esperaba y no introdujo nuevas vulnerabilidades.

**Ejemplo:** Una empresa decide añadir un nuevo servicio de pagos a su arquitectura de microservicios. El cambio requiere:

- Nuevos endpoints en la API Gateway.
- Nueva base de datos para el servicio de pagos.
- Actualización de las políticas de red para permitir la comunicación entre servicios.
- Nuevos certificados TLS para el nuevo servicio.

El arquitecto debe evaluar el impacto, planificar la migración (posiblemente en fases) y verificar que la seguridad no se vea comprometida.

### Métricas de calidad de la arquitectura

Para evaluar la calidad de una arquitectura, se utilizan métricas específicas que permiten medir su efectividad y detectar problemas tempranamente.

| Métrica                                  | Definición                                                        | Fórmula / Medición                                              | Valor objetivo                     |
|------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------|
| <b>MTTR</b> (Mean Time To Recover)       | Tiempo promedio para recuperar un servicio después de un fallo.   | Suma de tiempos de recuperación / Número de incidentes.         | < 1 hora para servicios críticos.  |
| <b>MTBF</b> (Mean Time Between Failures) | Tiempo promedio entre fallos de un sistema.                       | Tiempo total de operación / Número de fallos.                   | > 30 días para servicios críticos. |
| <b>Cobertura de segmentación</b>         | Porcentaje de la red que está segmentada con controles de acceso. | (Número de segmentos con controles / Total de segmentos) × 100. | > 90%.                             |
| <b>Cobertura de cifrado</b>              | Porcentaje de datos que están cifrados en reposo y en tránsito.   | (Datos cifrados / Datos totales) × 100.                         | 100% para datos sensibles.         |
| <b>Tiempo de detección de amenazas</b>   | Tiempo promedio para detectar una amenaza.                        | Medición con simulaciones de ataque (red team).                 | < 10 minutos.                      |

| Métrica                                 | Definición                                     | Fórmula / Medición                       | Valor objetivo |
|-----------------------------------------|------------------------------------------------|------------------------------------------|----------------|
| <b>Tiempo de respuesta a incidentes</b> | Tiempo promedio para responder a un incidente. | Medición con simulaciones de incidentes. | < 30 minutos.  |

### Cronograma y presupuesto para implementar una arquitectura de seguridad

Un Project Manager debe planificar la implementación de la arquitectura de seguridad. Aquí hay un ejemplo para una PYME que migra de una arquitectura plana a una segmentada:

#### Cronograma (12 semanas):

| Semana | Actividad                                                           | Hito                                                 |
|--------|---------------------------------------------------------------------|------------------------------------------------------|
| 1-2    | Evaluación de la arquitectura actual y diseño de la nueva.          | Documento de diseño aprobado.                        |
| 3-4    | Adquisición de hardware/software (firewalls, switches, servidores). | Equipos adquiridos y configurados.                   |
| 5-6    | Implementación de la DMZ y los firewalls.                           | DMZ operativa.                                       |
| 7-8    | Configuración de VLANs y políticas de firewall interno.             | Segmentación de red implementada.                    |
| 9-10   | Migración de aplicaciones y datos a la nueva arquitectura.          | Aplicaciones migradas y funcionando.                 |
| 11     | Pruebas de seguridad (penetration testing, vulnerabilidades).       | Informe de pruebas con hallazgos críticos resueltos. |
| 12     | Capacitación del personal y puesta en producción.                   | Arquitectura en producción.                          |

#### Presupuesto estimado (en C\$):

| Concepto                                                        | Costo (C\$)    |
|-----------------------------------------------------------------|----------------|
| Firewall empresarial (2 unidades para alta disponibilidad)      | 80,000         |
| Switches gestionables con VLAN                                  | 25,000         |
| Servidores adicionales (para DMZ y aplicación)                  | 60,000         |
| Licencias de software de seguridad (SIEM, WAF)                  | 40,000         |
| Consultoría para diseño e implementación (80 horas × C\$ 500/h) | 40,000         |
| Capacitación del personal (3 personas × 20 horas × C\$ 300/h)   | 18,000         |
| <b>Subtotal</b>                                                 | <b>263,000</b> |
| Reserva para imprevistos (15%)                                  | 39,450         |
| <b>Total</b>                                                    | <b>302,450</b> |

### Aplicación práctica: Principios de seguridad en una arquitectura de microservicios

Consideremos una aplicación de comercio electrónico con arquitectura de microservicios. A continuación se muestra cómo aplicar los principios anteriores de manera concreta:

| Principio              | Implementación en la arquitectura                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Defensa en profundidad | WAF (Web Application Firewall) → API Gateway → Autenticación JWT → Rate Limiting → Validación de entrada en cada servicio → Base de datos con cifrado |

| Principio               | Implementación en la arquitectura                                                                                                                                                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Privilegio mínimo       | Cada microservicio tiene su propia base de datos y solo puede acceder a ella; los servicios se comunican mediante tokens cortos (Service Mesh) con permisos específicos (ej. el servicio de pedidos solo puede leer la tabla de clientes, no modificarla)           |
| Seguridad por defecto   | Los contenedores se ejecutan sin privilegios de root; las imágenes se escanean en busca de vulnerabilidades antes del despliegue (Trivy, Snyk); el orquestador (Kubernetes) tiene políticas de red que deniegan todo el tráfico excepto el explícitamente permitido |
| Separación de funciones | El servicio de pedidos no puede acceder directamente a la tabla de pagos; solo a través del servicio de pagos que aplica sus propias reglas de validación y auditoría                                                                                               |
| Registro y auditoría    | Todos los microservicios envían logs estructurados a un sistema centralizado (ELK Stack) con correlación de trazabilidad (OpenTelemetry); se auditan todos los accesos a datos sensibles (ej. número de tarjeta)                                                    |
| Cifrado generalizado    | TLS mutuo (mTLS) entre servicios (Service Mesh como Istio); cifrado AES-256 en bases de datos (TDE); tokens JWT firmados con RS256; secretos de API gestionados con HashiCorp Vault                                                                                 |
| Zero Trust              | Cada solicitud se autentica y autoriza en cada salto (no se confía en la red interna); se implementa microsegmentación con políticas de red (NetworkPolicy en Kubernetes) que aíslan cada servicio                                                                  |

## Herramientas para el modelado y diseño de arquitectura segura

Los siguientes recursos permiten llevar la teoría a la práctica:

- **Microsoft Threat Modeling Tool:** Herramienta gratuita para crear diagramas de flujo de datos (DFD) y generar automáticamente amenazas STRIDE.
- **OWASP Threat Dragon:** Herramienta open source para modelado de amenazas, integrable con repositorios de código.
- **draw.io / diagrams.net:** Plataforma gratuita para dibujar diagramas de arquitectura, con plantillas de nube (AWS, Azure, GCP) y redes.
- **CIS Benchmarks:** Guías de configuración segura para sistemas operativos, aplicaciones y dispositivos de red.

## El rol del ingeniero en la arquitectura de seguridad

El Ingeniero en Sistemas de Información debe ser capaz de:

- **Evaluar la arquitectura existente** e identificar vulnerabilidades estructurales (ej. falta de segmentación, inexistencia de DMZ, comunicaciones sin cifrar, APIs sin autenticación).
- **Diseñar arquitecturas seguras** aplicando los principios de defensa en profundidad, privilegio mínimo, Zero Trust y modelado de amenazas.
- **Seleccionar frameworks** de arquitectura (TOGAF, SABSA) según el contexto de la organización y los requisitos normativos.
- **Documentar la arquitectura** incluyendo decisiones de seguridad, justificación de tecnologías, flujos de datos y modelo de amenazas.
- **Comunicar los riesgos** de la arquitectura actual a los tomadores de decisiones en términos de negocio (ej. "una brecha en esta API costaría aproximadamente C\$ 2 millones por multas y pérdida de clientes").
- **Realizar modelado de amenazas** en la fase de diseño, antes de escribir código, utilizando marcos como STRIDE.
- **Gestionar los trade-offs** entre seguridad, rendimiento, costo y usabilidad, documentando las decisiones y sus justificaciones.
- **Planificar la evolución de la arquitectura** a lo largo del ciclo de vida del proyecto, gestionando los cambios de manera controlada.

### Preguntas reflexivas para el ingeniero:

- ¿Cómo cambiaría el diseño si supiéramos que el 50% del tráfico proviene de redes móviles (poca confiabilidad)?
- ¿Qué sucede si el proveedor de nube sufre una caída regional? ¿Cómo afecta a la arquitectura?
- ¿Estamos preparados para un ataque de ransomware que cifre los backups? ¿Tenemos copias inmutables?
- ¿Cómo justificarías ante la dirección la inversión en una arquitectura más segura pero más costosa?

---

## Caso integrador: Diseño de arquitectura de seguridad para una PYME nicaragüense

### Escenario:

"Distribuidora del Norte, S.A." es una PYME en Estelí, Nicaragua, que distribuye productos agrícolas a 200 clientes en la región. Actualmente tiene:

- Un solo servidor Windows Server 2019 que funciona como web, aplicación y base de datos.
- Sin firewall; el servidor está conectado directamente a Internet.
- Todos los empleados (5) usan el mismo usuario y contraseña (admin/1234) para acceder al sistema.
- No hay backups; los datos se almacenan en el mismo disco del servidor.
- La empresa maneja datos de clientes (nombres, direcciones, teléfonos, historial de compras).

### Requisitos de la nueva arquitectura:

1. Separar las capas web, aplicación y datos.
2. Implementar controles de acceso basados en roles (RBAC).
3. Cifrar los datos sensibles de los clientes.
4. Implementar backups automáticos con RPO de 24 horas y RTO de 4 horas.
5. Cumplir con la Ley 787 de Protección de Datos Personales de Nicaragua.
6. Presupuesto máximo: C\$ 250,000.

### Diseño de la arquitectura propuesta:

```
graph TB
    subgraph "Internet"
        A[Clientes y Empleados]
    end
    subgraph "Firewall Perimetral"
        B[Router/Firewall Mikrotik con reglas]
    end
    subgraph "DMZ"
        C[Servidor Web Apache/Nginx Ubuntu 22.04 LTS]
    end
    subgraph "Red Interna"
        D[Firewall Interno (por software)]
        subgraph "VLAN Aplicaciones"
            E[Servidor de Aplicaciones Node.js / PHP 8.2]
        end
        subgraph "VLAN Datos"
            F[Servidor MySQL 8.0 TDE habilitado]
        end
        subgraph "VLAN Administración"
            G[SIEM Wazuh Monitoreo centralizado]
        end
    end
    subgraph "Capa de Respaldo"
        H[NAS Local]
```

+ Backups Offsite en nube]  
end

```
A -->|HTTPS| B
B -->|Puertos 80/443| C
C -->|Proxy reverso| D
D -->|Solo puerto 3000| E
E -->|Puerto 3306<br/>solo desde IP de app| F
F -->|Backup cifrado| H
C -. ->|Logs| G
E -. ->|Logs| G
F -. ->|Logs| G
```

???

#### Aplicación de principios y criterios:

| Principio / Criterio          | Implementación                                                                                                             |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Defensa en profundidad</b> | Firewall perimetral + firewall interno + segmentación por VLANs.                                                           |
| <b>Privilegio mínimo</b>      | Cada servidor solo puede comunicarse con el siguiente en la cadena; el servidor web no puede acceder directamente a la BD. |
| <b>Cifrado generalizado</b>   | HTTPS (TLS 1.3) para comunicaciones externas; TDE en MySQL para datos en reposo.                                           |
| <b>Registro y auditoría</b>   | SIEM Wazuh centraliza logs de todos los servidores; alertas en tiempo real.                                                |
| <b>Viabilidad legal</b>       | Cumple con Ley 787: datos cifrados, controles de acceso, políticas de retención.                                           |
| <b>Viabilidad económica</b>   | Costo estimado: C\$ 220,000 (dentro del presupuesto).                                                                      |
| <b>Viabilidad operativa</b>   | Capacitación de 2 días al personal; interfaz similar a la actual para facilitar la transición.                             |

#### Presupuesto detallado:

| Concepto                                                     | Costo (C\$)    |
|--------------------------------------------------------------|----------------|
| Router/Firewall MikroTik                                     | 15,000         |
| Servidor Web (Ubuntu) - equipo usado pero renovado           | 25,000         |
| Servidor Aplicaciones (Ubuntu) - equipo usado pero renovado  | 30,000         |
| Servidor Base de Datos (Ubuntu) - equipo nuevo con RAID      | 50,000         |
| NAS para backups locales                                     | 20,000         |
| Licencias y servicios en la nube para backups offsite        | 10,000/año     |
| Horas de consultoría (60 horas × C\$ 500/h)                  | 30,000         |
| Capacitación al personal (3 personas × 16 horas × C\$ 300/h) | 14,400         |
| <b>Subtotal</b>                                              | <b>194,400</b> |
| Reserva para imprevistos (10%)                               | 19,440         |
| <b>Total</b>                                                 | <b>213,840</b> |

## Lecciones aprendidas del caso:

1. La segmentación por VLANs es esencial para contener un ataque. Si un atacante compromete el servidor web, no podrá acceder a la base de datos.
2. El cifrado de datos en reposo (TDE) protege la información incluso si el disco es robado.
3. Los backups offsite son críticos para la recuperación ante desastres (ransomware, incendios, etc.).
4. La inversión en seguridad (C\$ 213,840) es significativamente menor que el costo potencial de una brecha (multas, pérdida de clientes, recuperación).

## Caso real adaptado (Nicaragua, contexto de PYME):

Una distribuidora de alimentos en Managua sufrió un ataque de ransomware que paralizó sus operaciones por 5 días. La investigación mostró que su servidor (Windows Server 2012) tenía el puerto RDP (3389) abierto a Internet, sin MFA, y sin segmentación de red. El atacante ingresó por fuerza bruta, instaló el ransomware y cifró tanto los archivos del sistema como los backups que estaban en el mismo servidor. La empresa perdió más de C\$ 300,000 en ventas no realizadas y costos de recuperación. La lección: una arquitectura plana y sin controles de acceso es una invitación al desastre.

## Comprobación de aprendizaje

**Ejercicio 3.1.1:** Identifica qué principio de diseño seguro se viola en cada situación:

- a) Un servidor de base de datos está directamente accesible desde Internet para que los desarrolladores puedan consultarlo desde casa.
- b) Todos los empleados tienen acceso de administrador local en sus computadoras.
- c) Una aplicación web no utiliza HTTPS porque "es más rápido".
- d) No se registran los intentos fallidos de inicio de sesión.
- e) Un microservicio puede acceder a las tablas de cualquier otro microservicio en la misma base de datos.

*Respuestas esperadas:* a) Defensa en profundidad (falta DMZ) y privilegio mínimo; b) Privilegio mínimo; c) Cifrado generalizado; d) Registro y auditoría; e) Privilegio mínimo y separación de funciones.

**Ejercicio 3.1.2 (Aplicación de STRIDE):** Para un sistema de banca en línea, identifica una amenaza para cada categoría STRIDE y propone un control arquitectónico.

| Categoría        | Amenaza                                                                                | Control arquitectónico                                                             |
|------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Suplantación (S) | Un atacante roba las credenciales de un usuario.                                       | MFA (autenticación multifactor) obligatoria; detección de comportamiento anómalo.  |
| Manipulación (T) | Un atacante modifica el monto de una transferencia en tránsito.                        | Firmas digitales en todas las transacciones; TLS 1.3.                              |
| Repudio (R)      | Un usuario niega haber realizado una transferencia.                                    | Logs de auditoría con firma digital y sello de tiempo; registro inmutable.         |
| Revelación (I)   | Un atacante accede a los números de tarjeta de los clientes.                           | Cifrado de datos sensibles en la BD; tokenización de tarjetas.                     |
| Denegación (D)   | Un ataque DDoS satura la plataforma de pagos.                                          | WAF, rate limiting, CDN, auto-scaling.                                             |
| Elevación (E)    | Un usuario estándar explota una vulnerabilidad para obtener permisos de administrador. | Principio de mínimo privilegio; separación de roles; actualizaciones de seguridad. |

**Ejercicio 3.1.3 (Trade-offs):** Una empresa de comercio electrónico debe decidir entre implementar MFA para todos los usuarios (incluyendo clientes) o solo para administradores. La MFA para todos los usuarios costaría C\$ 50,000 adicionales en desarrollo y soporte, y podría reducir la tasa de conversión en un 2%. Los administradores ya tienen MFA. ¿Qué recomendarías y por qué?

*Respuesta esperada:* Depende del perfil de riesgo de la empresa. Si maneja datos financieros sensibles y está sujeta a regulaciones (ej. PCI-DSS), la MFA para todos los usuarios puede ser obligatoria. Si el riesgo es bajo y la pérdida de conversión es significativa,

se podría priorizar la MFA solo para administradores y monitorear intentos de acceso anómalos. El trade-off es entre seguridad y experiencia de usuario/ingresos.

### Ejercicio 3.1.4 (Caso práctico contextualizado):

**Caso:** Una PYME nicaragüense de logística tiene su sistema de gestión de inventarios en un solo servidor Windows que funciona como web, aplicación y base de datos. No hay firewall, todos los empleados usan el mismo usuario (admin/1234) y el servidor está conectado directamente a Internet. Recientemente, un atacante explotó una vulnerabilidad en el servicio de impresión (Spooler) y cifró todos los datos con ransomware.

- Identifica al menos 5 fallos arquitectónicos que permitieron el ataque.
- Propón una nueva arquitectura de seguridad de tres capas (web, aplicación, datos) con segmentación, especificando qué cambios se necesitan.
- ¿Cómo aplicarías el principio de Zero Trust en esta nueva arquitectura?

*Respuesta esperada (abierta, pero debe incluir):*

- Fallos: falta de DMZ, servidor monolítico, sin segmentación, usuario compartido, puertos innecesarios abiertos (RDP, Spooler).
- Nueva arquitectura: Firewall perimetral → DMZ con servidor web (Linux) → Firewall interno → VLAN App (Node.js) → VLAN Datos (MySQL) con backups offsite.
- Zero Trust: MFA para todos los accesos, microsegmentación, políticas de red que deniegan por defecto, auditoría continua, verificación de integridad de dispositivos.

---

## Sistemas de Información

Los **sistemas de información** son conjuntos de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones y el control en una organización. Podemos pensar en ellos como el **sistema nervioso de una organización**: reciben estímulos (datos), los procesan, y generan respuestas (información) que permiten al cuerpo (la organización) funcionar, adaptarse y sobrevivir.

Desde la perspectiva de seguridad, cada sistema de información debe protegerse de acuerdo con el valor de los datos que maneja y el impacto que tendría una brecha en su confidencialidad, integridad o disponibilidad. Un sistema de información inseguro no solo pone en riesgo los datos, sino la continuidad misma del negocio.

### Conexión con unidades anteriores:

- En la **Unidad I** aprendimos a evaluar la **viabilidad operativa y técnica** de un proyecto. La seguridad de los sistemas de información es un pilar fundamental de esa viabilidad: un sistema que no puede asegurarse es inviable.
- En la **Unidad II** planificamos e ingeniamos el proceso. La definición de controles de acceso, la elección de estándares (OWASP) y la planificación de auditorías son actividades que deben realizarse durante la ingeniería del proceso.
- En la **Unidad III**, la seguridad de los sistemas de información es el corazón de la protección de los activos digitales de la organización. Aquí aprendemos a asegurar la arquitectura desde el nivel más operativo.

**Pregunta guía antes de comenzar:** Piensa en un sistema de información que uses a diario (ej. la plataforma de tu universidad, una red social, un sistema de banca en línea). ¿Qué controles de seguridad crees que tiene? ¿Qué pasaría si esos controles fallaran? ¿Cómo afectaría eso a la viabilidad del proyecto que lo sustenta?

---

### Recuadro de definiciones clave

| Término                       | Definición simple                                                                                                           |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Sistema de Información</b> | Conjunto de componentes (hardware, software, datos, procesos, personas) que recolectan, procesan y distribuyen información. |
| <b>RBAC</b>                   | Control de acceso basado en roles: los permisos se asignan a roles, y los usuarios a esos roles.                            |
| <b>ABAC</b>                   | Control de acceso basado en atributos: las decisiones dependen de atributos del usuario, recurso y contexto.                |

| Término          | Definición simple                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------|
| <b>DAC</b>       | Control de acceso discrecional: el propietario del recurso decide quién accede.                           |
| <b>OWASP</b>     | Open Web Application Security Project: organización que publica el Top 10 de riesgos en aplicaciones web. |
| <b>DevSecOps</b> | Integración de la seguridad en el ciclo de vida del desarrollo (Development, Security, Operations).       |

### Componentes de un sistema de información y su seguridad

Un sistema de información no es solo software. Para asegurarlo correctamente, debemos entender todos sus componentes y los riesgos asociados a cada uno.

| Componente      | Descripción                                                                   | Aspectos de seguridad                                                  | Pregunta clave de seguridad                                            | Ejemplo de control                                      |
|-----------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Hardware</b> | Dispositivos físicos (servidores, estaciones de trabajo, dispositivos de red) | Seguridad física, control de acceso a instalaciones, cifrado de discos | ¿Quién tiene acceso físico a este equipo?                              | Cercos perimetrales, tarjetas de acceso, BitLocker/LUKS |
| <b>Software</b> | Programas, aplicaciones, sistemas operativos                                  | Gestión de parches, hardening, análisis de vulnerabilidades            | ¿El software está actualizado y configurado de forma segura?           | WSUS, Nessus, OWASP Dependency Check                    |
| <b>Datos</b>    | Información almacenada y procesada                                            | Cifrado, clasificación, políticas de retención                         | ¿Los datos sensibles están protegidos incluso si el disco es robado?   | DLP, cifrado AES-256, políticas de backup               |
| <b>Procesos</b> | Procedimientos de negocio y TI                                                | Separación de funciones, autorización formal, auditoría de procesos    | ¿Hay controles para evitar que una sola persona tenga demasiado poder? | Flujo de autorización de cambios, seguridades duales    |
| <b>Personas</b> | Usuarios, administradores, desarrolladores                                    | Capacitación, concienciación, controles de acceso basados en roles     | ¿Los usuarios están capacitados para reconocer amenazas?               | Programa de phishing simulado, cursos de seguridad      |

#### Profundizando en el componente "Personas":

El factor humano es, con frecuencia, el eslabón más débil de la seguridad. Según el **Informe de Investigación de Brechas de Datos de Verizon (2023)**, el **74% de las brechas** involucran el elemento humano, incluyendo errores, uso de credenciales robadas, phishing y abuso de privilegios. Por ello, la capacitación y concienciación no son un lujo, sino una necesidad.

#### Ejemplo de programa de concienciación efectivo:

Un banco centroamericano implementó un programa de simulación de phishing mensual. Los empleados que hacían clic en enlaces sospechosos recibían capacitación inmediata de 5 minutos. En 6 meses, la tasa de clics en correos simulados bajó del 32% al 6%. Además, se estableció un canal de reporte rápido donde los empleados podían reportar correos sospechosos con un solo clic, y el equipo de seguridad respondía en menos de 5 minutos.

### Seguridad por diseño (Security by Design)

Este principio establece que la seguridad debe considerarse en cada etapa del ciclo de vida de un sistema o proceso, no como un añadido posterior. Implica:

- **Definir requisitos de seguridad** al mismo tiempo que los requisitos funcionales.

- **Realizar modelado de amenazas** (*threat modeling*) para identificar riesgos potenciales antes de escribir código. Un marco práctico es **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege), que ya fue abordado en la sección de Arquitectura TI.
- **Aplicar el principio de mínimo privilegio** en cada interacción.
- **Documentar decisiones de seguridad** para auditorías futuras.

**Ejemplo de modelado de amenazas para un sistema de transferencias bancarias:**

| Componente                     | Amenaza STRIDE                                                            | Control de seguridad                                 |
|--------------------------------|---------------------------------------------------------------------------|------------------------------------------------------|
| API de transferencias          | S (Spoofing) - Un atacante usa un token robado para hacer transferencias. | TLS mutuo y JWT con corta expiración.                |
| Base de datos de transacciones | T (Tampering) - Un empleado modifica el monto de una transferencia.       | Firmas digitales y auditoría de cambios en la BD.    |
| Logs de transacciones          | R (Repudiation) - Un usuario niega haber hecho una transferencia.         | Logs inmutables y firma digital de cada transacción. |

### Controles de acceso en sistemas de información

El control de acceso es el mecanismo fundamental para proteger los sistemas de información. Existen tres modelos principales:

| Modelo                                       | Siglas | Descripción                                                                                                                                                             | Ventajas                                                                    | Desventajas                                                                         | Ejemplo de aplicación                                                                           |
|----------------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Control de Acceso Basado en Roles</b>     | RBAC   | Los permisos se asignan a roles (ej. "Administrador", "Analista", "Consultor") y los usuarios se asignan a roles.                                                       | Fácil de administrar; escalable; alineado con la estructura organizacional. | Puede ser rígido si los roles no reflejan necesidades específicas.                  | Active Directory con grupos de seguridad; cada grupo tiene permisos sobre carpetas compartidas. |
| <b>Control de Acceso Basado en Atributos</b> | ABAC   | Las decisiones de acceso se basan en atributos del usuario (departamento, cargo, ubicación), del recurso (clasificación, propietario) y del contexto (hora, ubicación). | Muy flexible; soporta políticas complejas.                                  | Más complejo de implementar; requiere un motor de políticas (PDP).                  | AWS IAM con políticas basadas en etiquetas; Google Cloud IAM con condiciones.                   |
| <b>Control de Acceso Discrecional</b>        | DAC    | El propietario del recurso decide quién puede acceder. Común en sistemas de archivos (permisos UNIX).                                                                   | Simple y descentralizado.                                                   | No escala bien; difícil de auditar; los usuarios pueden otorgar permisos inseguros. | Permisos de archivos en Linux (rwx).                                                            |

### Profundizando en ABAC con un ejemplo detallado:

Una política ABAC típica podría expresarse en lenguaje natural como:

"Un médico puede acceder a la historia clínica de un paciente solo si:

1. Es el médico tratante del paciente (relación médico-paciente),
2. Está en horario laboral (contexto temporal),
3. El paciente no ha solicitado restricción de acceso (atributo del recurso), y
4. La solicitud proviene de un dispositivo autorizado (atributo del dispositivo)."

En una implementación técnica con XACML (eXtensible Access Control Markup Language), esta política podría verse así:

???xml

Médico  
Historia Clínica  
Leer

Médico.esTratante(Paciente) AND  
Médico.horarioLaboral() AND  
Paciente.noHaSolicitadoRestriccion() AND  
Dispositivo.esAutorizado()

Permitir

???

#### Flujo de decisión ABAC:

???mermaid  
graph TD  
A[Usuario solicita acceso a un recurso] --> B{¿El usuario tiene atributos requeridos?}  
B -->|No| C[Denegar acceso]  
B -->|Sí| D{¿El recurso tiene atributos compatibles?}  
D -->|No| C  
D -->|Sí| E{¿El contexto cumple las condiciones?}  
E -->|No| C  
E -->|Sí| F[Permitir acceso]  
???

**Recomendación:** Para sistemas de información corporativos, se recomienda RBAC como base, complementado con ABAC para casos de uso específicos que requieran políticas contextuales. DAC debe evitarse en sistemas críticos.

#### Auditoría de sistemas de información

La auditoría de sistemas de información evalúa si los controles de seguridad son adecuados y si se cumplen las políticas y regulaciones. Existen dos tipos principales:

| Tipo de auditoría        | Enfoque                                                                                                              | Periodicidad recomendada           | Entregable típico                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------|----------------------------------------------------------|
| <b>Auditoría interna</b> | Realizada por el equipo de auditoría de la propia organización. Evalúa el cumplimiento de políticas internas.        | Continua (automática) o trimestral | Informe de hallazgos y recomendaciones                   |
| <b>Auditoría externa</b> | Realizada por una firma independiente. Certifica el cumplimiento de estándares (ISO 27001) o regulaciones (Ley 787). | Anual                              | Certificado de conformidad o informe de no conformidades |

#### Flujo del proceso de auditoría:

???mermaid  
graph LR  
A[Planificación] --> B[Recopilación de evidencias]  
B --> C[Análisis y evaluación]  
C --> D[Elaboración de informe]  
D --> E[Presentación de hallazgos]

E --> F[Seguimiento de acciones correctivas]

F --> A

???

### Métricas y KPIs para sistemas de información:

La seguridad no se puede mejorar si no se mide. Aquí hay algunos indicadores clave de rendimiento (KPIs) para sistemas de información:

| KPI                                       | Definición                                                                     | Fórmula                                                      | Valor objetivo |
|-------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------|----------------|
| <b>Tiempo medio de detección (MTTD)</b>   | Tiempo promedio para detectar un incidente de seguridad.                       | Suma de tiempos de detección / Número de incidentes.         | < 30 minutos   |
| <b>Tiempo medio de respuesta (MTTR)</b>   | Tiempo promedio para contener y resolver un incidente.                         | Suma de tiempos de resolución / Número de incidentes.        | < 4 horas      |
| <b>Cobertura de parches</b>               | Porcentaje de sistemas con parches críticos aplicados en las últimas 48 horas. | (Sistemas parcheados / Total de sistemas) × 100.             | > 90%          |
| <b>Tasa de éxito de phishing simulado</b> | Porcentaje de empleados que hacen clic en enlaces de phishing simulados.       | (Clics / Correos enviados) × 100.                            | < 10%          |
| <b>Disponibilidad del sistema</b>         | Porcentaje de tiempo que el sistema está operativo y accesible.                | (Tiempo total - Tiempo de inactividad) / Tiempo total × 100. | > 99.5%        |

### Ejemplo de informe de auditoría interna:

| ID   | Hallazgo                                                                                        | Nivel de Riesgo | Recomendación                                                                            | Plazo   | Responsable                |
|------|-------------------------------------------------------------------------------------------------|-----------------|------------------------------------------------------------------------------------------|---------|----------------------------|
| A-01 | No se ha realizado un inventario actualizado de activos de información en los últimos 12 meses. | Alto            | Realizar inventario completo en el próximo trimestre, incluyendo clasificación de datos. | 3 meses | CISO                       |
| A-02 | El sistema de gestión de backups no incluye backups offsite.                                    | Crítico         | Implementar backups offsite con pruebas de restauración mensuales.                       | 1 mes   | Administrador de sistemas  |
| A-03 | Las cuentas de usuarios inactivos (más de 90 días) no se deshabilitan automáticamente.          | Medio           | Implementar política de desactivación automática de cuentas inactivas.                   | 2 meses | Administrador de identidad |
| A-04 | No se registran los accesos a datos sensibles de los clientes (ej. historiales médicos).        | Alto            | Implementar auditoría de acceso a datos sensibles con logging centralizado.              | 2 meses | DBA                        |
| A-05 | El sistema de facturación no tiene MFA para accesos administrativos.                            | Crítico         | Implementar MFA para todos los accesos administrativos en un plazo de 15 días.           | 15 días | Administrador de sistemas  |

### Seguridad en aplicaciones web (OWASP Top 10)

Las aplicaciones web son uno de los sistemas de información más expuestos. El **OWASP Top 10** es la referencia más utilizada para identificar los riesgos de seguridad más críticos en aplicaciones web. A continuación se presentan los principales riesgos con ejemplos de código vulnerable y seguro.

**Nota sobre versiones:** El OWASP Top 10 se actualiza periódicamente. La versión actual es la de **2021**, que introdujo cambios significativos como la inclusión de *Insecure Design* como una categoría separada y el movimiento de *Cryptographic Failures* a una posición más alta. Se recomienda consultar la versión más reciente en el sitio oficial de OWASP.

| Riesgo OWASP                          | Descripción                                                                                                       | Ejemplo de código vulnerable                                                                                                                  | Ejemplo de código seguro                                                                                                                                                                                                                 | Mitigación                                                                                                                         |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>A01: Broken Access Control</b>     | Fallos en la autorización que permiten a un usuario acceder a funcionalidades o datos que no le corresponden.     | <pre>javascript // Vulnerable app.get('/perfil/:id', (req, res) =&gt; { const perfil = db.getUser(req.params.id); res.send(perfil); });</pre> | <pre>javascript // Seguro app.get('/perfil/:id', (req, res) =&gt; { if (req.user.id !== parseInt(req.params.id)) { return res.status(403).send('No autorizado'); } const perfil = db.getUser(req.params.id); res.send(perfil); });</pre> | Validar autorización en cada petición; usar RBAC/ABAC del lado del servidor; nunca confiar en parámetros de URL para autorización. |
| <b>A02: Cryptographic Failures</b>    | Uso incorrecto de criptografía: cifrado débil, transmisión sin TLS, almacenamiento de contraseñas en texto plano. | <pre>javascript // Vulnerable const hash = crypto.createHash('md5'); hash.update(password); const hashed = hash.digest('hex');</pre>          | <pre>javascript // Seguro const bcrypt = require('bcrypt'); const saltRounds = 12; const hashed = await bcrypt.hash(password, saltRounds);</pre>                                                                                         | Usar bcrypt/argon2 para contraseñas; TLS 1.3 para comunicaciones; AES-256 para datos en reposo.                                    |
| <b>A03: Injection</b>                 | Envío de datos no confiables a un intérprete (SQL, NoSQL, OS, LDAP).                                              | <pre>javascript // Vulnerable const query = `SELECT * FROM usuarios WHERE username = '\${username}' AND password = '\${password}'`;</pre>     | <pre>javascript // Seguro const query = 'SELECT * FROM usuarios WHERE username = ? AND password = ?'; connection.query(query, [username, password]);</pre>                                                                               | Usar consultas parametrizadas (prepared statements); validar y sanitizar entradas; ORM con escape automático.                      |
| <b>A04: Insecure Design</b>           | Vulnerabilidades por diseño deficiente, no por errores de implementación.                                         | Un sistema de votación donde un mismo usuario puede votar mil veces porque no hay límite por IP o cuenta.                                     | Diseñar con límites de votación por usuario y por IP; auditoría de todos los votos.                                                                                                                                                      | Modelado de amenazas (STRIDE) en la fase de diseño; realizar análisis de riesgos antes de codificar.                               |
| <b>A05: Security Misconfiguration</b> | Configuraciones inseguras por defecto, directorios listables, cabeceras HTTP incorrectas.                         | Servidor muestra listado de directorios; se usa <code>admin/admin</code> como credenciales por defecto.                                       | Hardening basado en CIS Benchmarks; escaneo automático con herramientas como OpenSCAP.                                                                                                                                                   | Hardening basado en CIS Benchmarks; escaneo automático con OpenSCAP; revisar configuraciones antes de producción.                  |

| Riesgo OWASP                                   | Descripción                                                                | Ejemplo de código vulnerable                                     | Ejemplo de código seguro                                                                                                   | Mitigación                                                                                                                          |
|------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>A06: Vulnerable and Outdated Components</b> | Uso de librerías, frameworks o componentes con vulnerabilidades conocidas. | Aplicación usa Log4j versión 2.14 (vulnerable a CVE-2021-44228). | <code>npm audit</code> o <code>snyk test</code> para identificar dependencias vulnerables; actualizar a versiones seguras. | Inventario de dependencias; análisis automático con OWASP Dependency Check o Snyk; parchear en $\leq 48$ horas para CVSS $\geq 9$ . |

## DevSecOps: integrando la seguridad en el ciclo de vida del desarrollo

El enfoque tradicional de agregar seguridad al final del ciclo de desarrollo ha demostrado ser ineficaz. **DevSecOps** integra la seguridad en cada fase del pipeline de desarrollo. Esto significa que la seguridad no es responsabilidad exclusiva de un equipo, sino de todos los involucrados en el desarrollo, operaciones y aseguramiento de la calidad.

### Principios clave de DevSecOps:

1. **Seguridad como código:** Las políticas de seguridad y los controles se definen mediante código (Infrastructure as Code, Policy as Code).
2. **Automatización:** Las pruebas de seguridad (SAST, DAST, SCA) se ejecutan automáticamente en cada commit o pull request.
3. **Colaboración:** Los equipos de desarrollo, operaciones y seguridad trabajan juntos desde el inicio del proyecto.
4. **Cambio cultural:** La seguridad es responsabilidad de todos, no solo de un equipo especializado.

### Ejemplo de pipeline DevSecOps:

```

???mermaid
graph LR
  A[Commit de código] --> B[SAST  
SonarQube / Checkmarx]
  B --> C[SCA  
Snyk / OWASP DC]
  C --> D[Pruebas unitarias]
  D --> E[Construcción]
  E --> F[DAST  
OWASP ZAP]
  F --> G[Despliegue]
  G --> H[Monitoreo y respuesta  
Wazuh / SIEM]
  ???

```

### Herramientas clave en DevSecOps:

| Fase                                               | Herramienta                   | Función                                                                                    |
|----------------------------------------------------|-------------------------------|--------------------------------------------------------------------------------------------|
| <b>SAST</b> (Static Application Security Testing)  | SonarQube, Checkmarx, Fortify | Analiza el código fuente en busca de vulnerabilidades (ej. inyección SQL, XSS).            |
| <b>SCA</b> (Software Composition Analysis)         | Snyk, OWASP Dependency Check  | Analiza dependencias (librerías, frameworks) en busca de vulnerabilidades conocidas (CVE). |
| <b>DAST</b> (Dynamic Application Security Testing) | OWASP ZAP, Burp Suite         | Prueba la aplicación en ejecución simulando ataques.                                       |

| Fase                                                   | Herramienta              | Función                                                                         |
|--------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------|
| <b>IAST</b> (Interactive Application Security Testing) | Contrast Security        | Combina SAST y DAST, analizando la aplicación desde dentro mientras se ejecuta. |
| <b>Monitoreo y respuesta</b>                           | Wazuh, ELK Stack, Splunk | Detecta y responde a incidentes en tiempo real.                                 |

### Caso de éxito: DevSecOps en Amazon

Amazon integró la seguridad directamente en su pipeline de desarrollo (DevSecOps). Cada pieza de código, antes de llegar a producción, pasa por análisis estático automatizado (SAST), escaneo de dependencias vulnerables y pruebas de seguridad dinámicas. Si algo falla, el despliegue se bloquea automáticamente. Esto no ralentiza el desarrollo; lo acelera porque los problemas se detectan en minutos, no en semanas. El resultado: miles de despliegues diarios con un nivel de seguridad que antes requería auditorías manuales.

**Lección clave:** La seguridad no es una fase separada del desarrollo; es parte integral del ciclo de vida del software.

### Caso de fracaso: atención al cliente y PCI DSS

Una empresa de comercio electrónico permitía a sus agentes de atención al cliente ver el número completo de tarjetas de crédito en pantalla para "agilizar la resolución de quejas". Esto violaba PCI DSS (que exige enmascarar los dígitos, mostrando solo los últimos 4). Un agente descontento copió números de tarjetas y los vendió en foros clandestinos. La empresa fue multada y perdió la capacidad de procesar pagos durante tres meses. La lección: el proceso de atención al cliente no había sido diseñado con controles de confidencialidad; primó la "agilidad" sobre la seguridad.

**Lección clave:** La seguridad debe integrarse en los procesos de negocio desde el diseño, no ser un añadido posterior.

### Caso real adaptado (Nicaragua): Sistema de gestión académica comprometido

En 2022, una universidad nicaragüense (cuyo nombre se omite por razones de confidencialidad) sufrió una brecha en su sistema de gestión académica. Un estudiante de ingeniería descubrió que modificando el parámetro `id_estudiante` en la URL de consulta de calificaciones podía ver las notas de cualquier compañero. La vulnerabilidad (Broken Access Control, A01 de OWASP) había estado presente durante más de dos años, pero nunca fue detectada porque la universidad no realizaba auditorías de seguridad ni pruebas de penetración.

#### Consecuencias:

- Exposición de datos de 12,000 estudiantes (nombres, cédulas, calificaciones).
- Multa de la Dirección de Protección de Datos Personales por incumplimiento de la Ley 787 (C\$ 150,000).
- Daño reputacional significativo: los padres de familia exigieron respuestas.
- Costo de remediación: C\$ 200,000 en consultoría, parches y capacitación.

#### Lecciones aprendidas:

1. La autorización nunca debe confiar en parámetros de URL; debe basarse en la sesión del usuario.
2. Las auditorías de seguridad regulares son esenciales, incluso para sistemas aparentemente "pequeños".
3. La Ley 787 no es solo una formalidad; tiene consecuencias reales.
4. La capacitación en seguridad para desarrolladores es fundamental para evitar este tipo de vulnerabilidades.

### Gobernanza de sistemas de información

Cada sistema de información debe tener un **dueño** asignado, responsable de que los controles de seguridad se apliquen y se actualicen. El dueño reporta al comité de seguridad las desviaciones, los incidentes relacionados con el sistema y las métricas de cumplimiento.

### Responsabilidades del dueño del sistema:

1. Asegurar que el sistema cumpla con las políticas de seguridad de la organización.
2. Autorizar los accesos al sistema (o delegar la autorización).
3. Revisar periódicamente los logs de acceso y actividad.
4. Aprobar los cambios significativos en el sistema.
5. Participar en la evaluación de riesgos del sistema.
6. Asegurar que el sistema tenga un plan de contingencia y recuperación.

### Ejemplo de reporte trimestral del dueño del sistema:

"Sistema: Plataforma de Ventas en Línea  
Dueño: Gerente de Ventas Digitales  
Incidentes de seguridad en el trimestre: 0  
Parches aplicados: 100% de los críticos en 48 horas  
Accesos concedidos: 3 nuevos usuarios, 2 revocados  
Próxima auditoría programada: 15 de diciembre  
Riesgo residual actual: Bajo (dentro del apetito de riesgo aprobado)"

### Seguridad en sistemas de información para PYMES

Las pequeñas y medianas empresas a menudo tienen recursos limitados, pero eso no significa que deban descuidar la seguridad. Aquí hay un conjunto de controles básicos pero efectivos:

| Control                   | Herramientas open source / bajo costo                  | Impacto                                           |
|---------------------------|--------------------------------------------------------|---------------------------------------------------|
| Firewall de red           | pfSense, OPNsense                                      | Bloquea accesos no autorizados a la red.          |
| Antivirus / EDR           | ClamAV, Wazuh (con módulo EDR)                         | Detecta y bloquea malware.                        |
| Gestión de parches        | WSUS (para Windows), unattended-upgrades (para Linux)  | Mantiene los sistemas actualizados.               |
| Backups                   | Bacula, Duplicati, rsync                               | Recupera datos en caso de incidente.              |
| Autenticación multifactor | Google Authenticator, FreeOTP                          | Protege contra robo de credenciales.              |
| Concienciación            | Programas de phishing simulado gratuitos (ej. Gophish) | Reduce el riesgo de ataques de ingeniería social. |
| Monitoreo                 | Wazuh, ELK Stack                                       | Detecta actividad anómala en tiempo real.         |

### Ejemplo de implementación en una PYME nicaragüense:

Una cooperativa de café en Jinotega implementó el siguiente conjunto de controles con un presupuesto de C\$ 50,000:

- Firewall pfSense en una computadora dedicada (C\$ 15,000).
- Wazuh para monitoreo y EDR en servidores y estaciones de trabajo (gratuito, costo de implementación C\$ 10,000).
- Capacitación de 2 días para 10 empleados (C\$ 8,000).
- Backups automáticos a un NAS local y a la nube (C\$ 17,000 en hardware + servicios).

En 18 meses, el sistema detectó y bloqueó 3 intentos de ransomware, evitando pérdidas estimadas en C\$ 200,000.

### Sistemas heredados (legacy): desafíos y estrategias

Muchas organizaciones aún operan con sistemas heredados que no pueden actualizarse fácilmente. Estos sistemas presentan desafíos únicos de seguridad:

#### Desafíos comunes:

- El fabricante ya no proporciona parches de seguridad.
- No se pueden instalar herramientas de seguridad modernas (ej. antivirus, EDR).
- Están escritos en lenguajes o frameworks obsoletos.
- No soportan autenticación moderna (ej. MFA, SSO).

#### Estrategias de mitigación:

1. **Segmentación de red:** Aislar el sistema legacy en una VLAN separada con políticas de firewall restrictivas.
2. **Proxy inverso / WAF:** Colocar un WAF delante del sistema para filtrar tráfico malicioso.
3. **Monitoreo intensivo:** Aumentar el logging y monitoreo del sistema para detectar actividad anómala.
4. **Compensación con controles de aplicación:** Si el sistema no soporta MFA, usar un proxy de autenticación que sí la soporte.
5. **Plan de migración:** Definir un plan a mediano plazo para migrar a un sistema moderno.

**Ejemplo:** Un hospital nicaragüense tiene un sistema de gestión de pacientes desarrollado en Visual Basic 6 (lanzado en 1998) que aún utiliza una base de datos Access. La estrategia de mitigación fue:

- Aislar el servidor en una VLAN separada sin acceso a Internet.
- Configurar un firewall que solo permite conexiones desde la red interna del hospital.
- Implementar un proxy de autenticación que añade MFA antes de redirigir al sistema legacy.
- Definir un plan de migración a un sistema moderno en 2 años.

---

#### Seguridad en sistemas de información y cumplimiento normativo

Los sistemas de información deben diseñarse para cumplir con las regulaciones aplicables. En Nicaragua, la **Ley 787 de Protección de Datos Personales** establece requisitos específicos:

| Requisito de la Ley 787         | Implicación para el sistema de información                                                                               |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Consentimiento explícito</b> | El sistema debe solicitar consentimiento antes de recolectar datos personales, y debe permitir revocarlo.                |
| <b>Derecho de acceso</b>        | El sistema debe permitir a los usuarios acceder a sus datos personales en cualquier momento.                             |
| <b>Derecho de rectificación</b> | El sistema debe permitir a los usuarios corregir datos incorrectos.                                                      |
| <b>Derecho de cancelación</b>   | El sistema debe permitir a los usuarios solicitar la eliminación de sus datos.                                           |
| <b>Notificación de brechas</b>  | El sistema debe tener mecanismos para detectar brechas y notificar a los afectados y a la autoridad en un plazo máximo.  |
| <b>Seguridad de los datos</b>   | El sistema debe implementar medidas técnicas y organizativas para proteger los datos (cifrado, control de acceso, logs). |

#### Ejemplo de implementación en un sistema universitario:

La Universidad Nacional "Héroes de San José de las Mulas" actualizó su sistema de gestión académica para cumplir con la Ley 787:

- Se agregó una pantalla de consentimiento antes de registrar nuevos estudiantes.
- Se implementó un módulo de autoportal donde los estudiantes pueden ver, corregir y solicitar la eliminación de sus datos.
- Se cifraron las cédulas y direcciones de los estudiantes en la base de datos.
- Se implementaron logs de acceso a datos sensibles con alertas en tiempo real.

---

#### Sistemas de información y continuidad de negocio

La seguridad de los sistemas de información no se limita a prevenir ataques; también incluye asegurar que los sistemas puedan recuperarse rápidamente en caso de incidente. La **continuidad del negocio** y la **recuperación ante desastres** son componentes esenciales.

#### Conceptos clave:

| Concepto                              | Definición                                                                               | Ejemplo                                                                                      |
|---------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>RPO (Recovery Point Objective)</b> | Cantidad máxima de datos que la organización está dispuesta a perder (medido en tiempo). | RPO = 1 hora → se pierde a lo sumo 1 hora de datos.                                          |
| <b>RTO (Recovery Time Objective)</b>  | Tiempo máximo para recuperar el servicio después de un desastre.                         | RTO = 4 horas → el sistema debe estar operativo en $\leq 4$ horas.                           |
| <b>BCP (Business Continuity Plan)</b> | Plan que define cómo la organización mantendrá sus operaciones durante una interrupción. | Procedimientos para operar manualmente mientras el sistema se recupera.                      |
| <b>DRP (Disaster Recovery Plan)</b>   | Plan específico para recuperar los sistemas de información después de un desastre.       | Pasos para restaurar backups, reconstruir servidores y verificar la integridad de los datos. |

#### Ejemplo de DRP para un sistema de facturación:

| Paso | Acción                                                               | Responsable               | Tiempo estimado |
|------|----------------------------------------------------------------------|---------------------------|-----------------|
| 1    | Activar el plan de contingencia y notificar al equipo.               | Gerente de TI             | 15 minutos      |
| 2    | Restaurar la base de datos desde el último backup válido.            | DBA                       | 1 hora          |
| 3    | Restaurar la aplicación web desde el repositorio de código.          | Desarrollador             | 30 minutos      |
| 4    | Configurar el servidor alternativo con la aplicación restaurada.     | Administrador de sistemas | 1 hora          |
| 5    | Verificar la integridad de los datos y la funcionalidad del sistema. | Tester                    | 1 hora          |
| 6    | Reanudar operaciones normales y documentar el incidente.             | Gerente de TI             | 30 minutos      |

**Tiempo total estimado (RTO):** 4 horas y 15 minutos.

#### Caso integrador: Diseño de un sistema de información seguro

##### Escenario:

"Clínica Salud Integral, S.A." es una clínica privada en Managua que atiende a 500 pacientes al mes. Actualmente:

- Maneja historias clínicas en papel.
- Los datos de pacientes (nombres, direcciones, teléfonos, diagnósticos) están en una hoja de cálculo Excel compartida en una carpeta de red.
- No hay control de acceso; todos los empleados (10) pueden ver y modificar cualquier dato.
- No hay backups.
- La clínica planea migrar a un sistema de gestión de pacientes digital.

##### Requisitos del nuevo sistema:

1. Almacenar historias clínicas electrónicas de forma segura.
2. Solo médicos autorizados pueden ver las historias clínicas de sus pacientes.
3. Cumplir con la Ley 787 de Protección de Datos Personales.
4. RPO de 30 minutos, RTO de 2 horas.
5. Presupuesto máximo: C\$ 150,000.

##### Diseño de la solución:

| Componente | Solución propuesta | Justificación |
|------------|--------------------|---------------|
|------------|--------------------|---------------|

| Componente                  | Solución propuesta                                                                                                          | Justificación                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>Autenticación</b>        | MFA con Google Authenticator para todos los usuarios.                                                                       | Protege contra robo de credenciales.             |
| <b>Control de acceso</b>    | RBAC con roles: Médico (solo pacientes asignados), Administrativo (solo datos de contacto), Auditor (solo lectura de logs). | Principio de privilegio mínimo.                  |
| <b>Cifrado en reposo</b>    | TDE en PostgreSQL; columnas de diagnóstico y nombre completo cifradas con pgcrypto.                                         | Protege los datos incluso si el disco es robado. |
| <b>Cifrado en tránsito</b>  | TLS 1.3 para todas las comunicaciones entre cliente y servidor.                                                             | Protege los datos durante la transmisión.        |
| <b>Auditoría</b>            | Wazuh para monitoreo centralizado; logs de todos los accesos a historias clínicas.                                          | Detecta accesos no autorizados.                  |
| <b>Backups</b>              | Backup incremental cada 30 minutos (RPO = 30 min) + completo diario + completo semanal a la nube.                           | Cumple RPO y RTO.                                |
| <b>Plan de contingencia</b> | Procedimiento documentado para operar manualmente (papel) durante 4 horas si el sistema falla.                              | Cumple RTO de 2 horas.                           |
| <b>Capacitación</b>         | Programa de concienciación de seguridad para todo el personal.                                                              | Reduce el riesgo de errores humanos.             |

#### Presupuesto estimado:

| Concepto                                                              | Costo (C\$)     |
|-----------------------------------------------------------------------|-----------------|
| Servidor en la nube (VPS) por 12 meses                                | 24,000          |
| Licencias de software (PostgreSQL, pgcrypto, Wazuh)                   | 0 (open source) |
| Desarrollo personalizado (migración de datos, desarrollo del sistema) | 60,000          |
| Capacitación al personal (10 personas × 16 horas × C\$ 300/h)         | 48,000          |
| Horas de consultoría (40 horas × C\$ 500/h)                           | 20,000          |
| <b>Subtotal</b>                                                       | <b>152,000</b>  |
| Reserva para imprevistos (10%)                                        | 15,200          |
| <b>Total</b>                                                          | <b>167,200</b>  |

*Nota: El presupuesto excede ligeramente el límite de C\$ 150,000. Se recomienda reducir el alcance inicial (ej. implementar MFA en una segunda fase) o buscar financiamiento adicional.*

#### Lecciones aprendidas del caso:

1. La migración de datos (de Excel a base de datos) requiere una limpieza y validación significativa.
2. La capacitación del personal es tan importante como la tecnología; un sistema seguro con usuarios no capacitados sigue siendo inseguro.
3. El cumplimiento de la Ley 787 no es opcional; los costos de no cumplir (multas, pérdida de reputación) superan ampliamente los costos de implementación.

#### Ruta de aprendizaje sugerida

Para aprovechar al máximo esta sección, sigue esta ruta de aprendizaje:

1. **Paso 1 (Fundamentos):** Lee las definiciones clave, los componentes del sistema de información y la tabla de controles de acceso.
2. **Paso 2 (Profundización):** Estudia la sección de Seguridad por Diseño y el modelado de amenazas con STRIDE.

3. **Paso 3 (Aplicación):** Revisa la tabla de OWASP Top 10 con los ejemplos de código. Practica identificando vulnerabilidades en código que tú mismo escribas o en ejemplos de internet.
  4. **Paso 4 (Gestión):** Lee las secciones de auditoría, DevSecOps, métricas y continuidad de negocio. Relaciona estos conceptos con la gestión de proyectos.
  5. **Paso 5 (Práctica):** Resuelve el caso integrador y los ejercicios de comprobación de aprendizaje. Si es posible, forma un equipo y simula una auditoría de un sistema real (con permiso del propietario).
- 

### Comprobación de aprendizaje

**Ejercicio 3.2.1:** Para cada situación, indique qué modelo de control de acceso (RBAC, ABAC, DAC) sería más adecuado:

- a) Una empresa con 500 empleados en 10 departamentos necesita gestionar acceso a carpetas compartidas. Los roles están bien definidos. → \_\_\_\_\_
- b) Un hospital necesita restringir el acceso a historias clínicas según el rol del médico, la relación con el paciente y el horario. → \_\_\_\_\_
- c) Un desarrollador quiere compartir un archivo temporal con un colega sin pasar por el departamento de TI. → \_\_\_\_\_

*Respuestas esperadas:* a) RBAC; b) ABAC (por los atributos contextuales); c) DAC (para uso temporal y no crítico).

**Ejercicio 3.2.2:** Identifique al menos tres riesgos OWASP Top 10 en el siguiente escenario:

Una aplicación móvil de delivery permite a los usuarios registrarse con cualquier contraseña (incluso "1234"). Los pedidos se realizan mediante peticiones HTTP sin TLS. La aplicación muestra el precio total en la URL (`/pedido?total=150&id=45`). Un usuario avanzado notó que si cambia el valor de `total`, el sistema acepta el nuevo monto.

*Respuesta esperada:* A02 (Cryptographic Failures – sin TLS), A07 (Identification and Authentication Failures – contraseñas débiles), A01 (Broken Access Control – el usuario puede modificar el total en la URL sin autorización del servidor).

### Ejercicio 3.2.3 (Caso práctico):

Una universidad almacena datos personales de 30,000 estudiantes (nombres, cédulas, calificaciones, datos de salud). Recientemente, un estudiante descubrió que modificando el parámetro `id_estudiante` en la URL de consulta de calificaciones puede ver las notas de cualquier compañero. La universidad no tiene políticas de seguridad definidas, no hay cifrado en la base de datos y los servidores tienen parches de hace 2 años.

- a) Identifique al menos 3 riesgos OWASP Top 10 presentes en el escenario.
- b) Proponga un plan de remediación priorizado (corto, mediano y largo plazo).
- c) ¿Cómo justificaría la inversión en seguridad ante el rectorado, usando el lenguaje de negocio?

*Respuesta esperada (abierta, pero debe incluir):*

- *Riesgos:* A01 (Broken Access Control), A02 (Cryptographic Failures), A06 (Vulnerable and Outdated Components), A07 (Authentication Failures), A09 (Logging Failures).
- *Plan:* Corto plazo (48h): deshabilitar el endpoint vulnerable, cambiar contraseñas; Mediano plazo (2 semanas): implementar validación de autorización, migrar a consultas parametrizadas; Largo plazo (3 meses): cifrado de BD, MFA, implementar SIEM.
- *Justificación:* Costo de multa potencial (C\$ 5,000,000), daño reputacional, pérdida de confianza de estudiantes y padres.

### Ejercicio 3.2.4 (Reflexión):

¿Por qué cree que muchas organizaciones descuidan la seguridad de los sistemas de información a pesar de conocer los riesgos? Mencione al menos tres razones y proponga cómo abordarlas.

*Respuesta esperada (abierta, pero debe incluir):*

- *Razones:* 1) Costo percibido como alto y sin retorno inmediato; 2) Falta de conciencia de los directivos; 3) Complejidad técnica percibida.
  - *Abordajes:* 1) Presentar el ROI de seguridad (costo de la brecha vs. costo de los controles); 2) Educar a los directivos en lenguaje de negocio; 3) Implementar controles básicos escalonados, no todo a la vez.
-

## Bases de Datos

Las bases de datos almacenan el activo más valioso de una organización: sus datos. La seguridad de bases de datos abarca la protección de los datos contra accesos no autorizados, modificaciones indebidas, pérdidas y exposiciones accidentales o maliciosas. Un error en la configuración de una base de datos puede exponer millones de registros; una brecha en su seguridad puede destruir la reputación de una empresa en cuestión de horas.

Esta sección se conecta directamente con las unidades anteriores:

- En la **Unidad I** aprendimos a evaluar la **viabilidad técnica y económica** de un proyecto. La elección de una base de datos segura, los costos de cifrado y la estrategia de backups son factores determinantes para esa viabilidad.
- En la **Unidad II** planificamos e ingeniamos el proceso. La definición del modelo de datos, la estrategia de cifrado y el plan de recuperación deben ser parte de la ingeniería del proceso.
- En la **Unidad III**, la seguridad de las bases de datos es el corazón de la protección de los datos. Aquí aprendemos a asegurar el almacén de información más crítico de la organización.

**Pregunta guía antes de comenzar:** Piensa en la base de datos de tu universidad (la que almacena tus calificaciones, tu cédula, tu historial académico). ¿Cómo crees que está protegida? ¿Qué controles crees que debería tener? ¿Qué pasaría si esos controles fallaran?

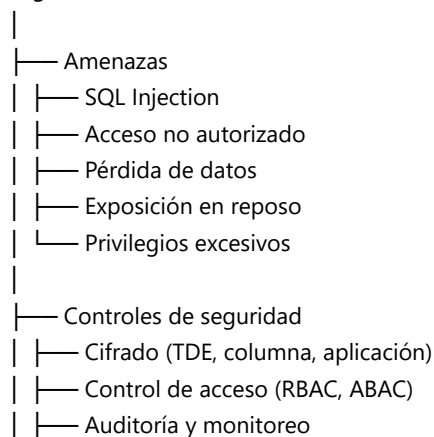
### Recuadro de definiciones clave

| Término                                  | Definición simple                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>SQL Injection</b>                     | Técnica de ataque que inserta código SQL malicioso en una consulta para manipular la base de datos.       |
| <b>TDE (Transparent Data Encryption)</b> | Cifrado automático de los datos en el disco, sin cambios en las aplicaciones.                             |
| <b>Cifrado a nivel de columna</b>        | Cifrado de campos específicos (ej. números de tarjeta) usando funciones de la base de datos.              |
| <b>Cifrado a nivel de aplicación</b>     | La aplicación cifra los datos antes de enviarlos a la base de datos; la BD nunca ve datos en texto plano. |
| <b>RPO (Recovery Point Objective)</b>    | Cantidad máxima de datos que la organización está dispuesta a perder (medido en tiempo).                  |
| <b>RTO (Recovery Time Objective)</b>     | Tiempo máximo permitido para recuperar el servicio después de un desastre.                                |

### Mapa conceptual de la sección

???text

Seguridad en Bases de Datos



- | — Modelado de amenazas (STRIDE)
- |
- | — Recuperación
- | — Backups (completo, incremental, diferencial)
- | — RPO y RTO
- | — Planes de contingencia y recuperación
- |
- | — Nube y NoSQL
- | — Modelo de responsabilidad compartida
- | — DBaaS
- | — Seguridad en NoSQL
- |
- | — Cumplimiento normativo
- | — Ley 787 (Nicaragua)
- | — PCI-DSS, GDPR
- |
- | — Caso integrador
- |
- | — Comprobación de aprendizaje
- ???

## Principales amenazas a la seguridad de bases de datos

| Amenaza                              | Descripción                                                                 | Impacto potencial                                               | Frecuencia relativa              |
|--------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------|
| <b>SQL Injection</b>                 | Insertión de código SQL malicioso a través de entradas de usuario           | Exposición, modificación o eliminación de toda la base de datos | Muy alta                         |
| <b>Acceso no autorizado</b>          | Usuarios o aplicaciones acceden a datos sin los permisos adecuados          | Fuga de información sensible                                    | Alta                             |
| <b>Pérdida de datos</b>              | Eliminación accidental o maliciosa, desastres naturales, fallos de hardware | Pérdida permanente de información crítica                       | Media                            |
| <b>Exposición de datos en reposo</b> | Archivos de base de datos robados o discos extraídos sin cifrar             | Exposición masiva de datos                                      | Baja (pero impacto catastrófico) |
| <b>Privilegios excesivos</b>         | Usuarios o aplicaciones tienen más permisos de los necesarios               | Ataque interno, movimiento lateral                              | Alta                             |
| <b>Denegación de servicio (DoS)</b>  | Saturar la base de datos con peticiones para que no esté disponible         | Indisponibilidad del servicio                                   | Media                            |

## Análisis de riesgos en bases de datos

Para priorizar las inversiones en seguridad, es útil aplicar los criterios de riesgo de la Unidad I (Probabilidad × Impacto). Por ejemplo:

| Amenaza                      | Probabilidad (1-5) | Impacto (1-5) | Nivel de Riesgo (P×I) | Acción recomendada                          |
|------------------------------|--------------------|---------------|-----------------------|---------------------------------------------|
| SQL Injection                | 5                  | 5             | 25 (Crítico)          | Mitigar con consultas parametrizadas y WAF. |
| Pérdida de datos sin backups | 3                  | 5             | 15 (Alto)             | Mitigar con backups automáticos y offsite.  |

| Amenaza                                        | Probabilidad<br>(1-5) | Impacto<br>(1-5) | Nivel de Riesgo<br>(P×I) | Acción recomendada                |
|------------------------------------------------|-----------------------|------------------|--------------------------|-----------------------------------|
| Exposición de datos en reposo<br>(sin cifrado) | 2                     | 5                | 10 (Medio)               | Mitigar con TDE (costo moderado). |

### SQL Injection: funcionamiento y mitigación

La **inyección SQL** es la amenaza más conocida y peligrosa para las bases de datos. Ocurre cuando una aplicación construye consultas SQL concatenando datos proporcionados por el usuario sin validarlos ni escaparlos. En el contexto nicaragüense, donde muchas PYMES y entidades gubernamentales utilizan sistemas desarrollados internamente sin controles de seguridad adecuados, la SQL Injection sigue siendo una de las principales causas de brechas de datos.

#### Diagrama de flujo de un ataque SQL Injection:

```

sequenceDiagram
    actor Atacante
    participant App as Aplicación Web
    participant DB as Base de Datos
  
```

```

Atacante->>App: Enviar formulario de login<br/>con username: admin' --
App->>App: Construir consulta SQL<br/>por concatenación<br/>"SELECT * FROM users WHERE user='" +
input + "'"
Note over App: Código VULNERABLE<br/>Sin prepared statements
App->>DB: Ejecutar consulta modificada:<br/>SELECT * FROM users<br/>WHERE user='admin' -- '
Note over DB: El -- comenta el resto<br/>de la consulta (password)
DB-->>App: Devolver datos del usuario admin<br/>SIN verificar contraseña
App-->>Atacante: Sesión iniciada como admin<br/>Acceso completo al sistema

Note over Atacante,DB: El atacante ahora puede:<br/>- Robar datos de todos los usuarios<br/>-
Modificar registros<br/>- Eliminar tablas (DROP)<br/>- Ejecutar comandos del sistema (si hay
privilegios)
  
```

???

**Explicación del ataque:** El atacante ingresa `admin' --` como nombre de usuario. La consulta original es `SELECT * FROM usuarios WHERE username = 'admin' --' AND password = 'cualquiera'`. Los caracteres `--` comentan el resto de la consulta, por lo que la validación de contraseña se omite. El atacante inicia sesión como `admin` sin conocer la contraseña.

#### Mitigación: consultas parametrizadas (prepared statements)

La forma correcta de construir la consulta es separar la estructura SQL de los datos:

```

???php
$stmt = $pdo->prepare("SELECT * FROM usuarios WHERE username = :username AND password = :password");
$stmt->execute(['username' => $_POST['username'], 'password' => $_POST['password']]);
???

???python
cur.execute("SELECT * FROM usuarios WHERE username = %s AND password = %s", (username, password))
???

???java
PreparedStatement stmt = conn.prepareStatement("SELECT * FROM usuarios WHERE username = ? AND password = ?");
stmt.setString(1, username);
  
```

```
stmt.setString(2, password);  
???
```

**Verificación de mitigación:** Para confirmar que una aplicación no es vulnerable a SQL Injection, se debe realizar una **prueba de penetración** automatizada (con herramientas como SQLMap o Burp Suite Scanner) y manual (probando entradas como `'`, `"`, `OR 1=1, '`; `DROP TABLE usuarios`; `--`).

### Modelado de amenazas para bases de datos (STRIDE)

Aplicar el marco STRIDE a una base de datos permite identificar proactivamente las amenazas antes de que ocurran. A continuación se muestra un ejemplo para un sistema bancario:

| Componente de la BD    | Amenaza STRIDE                                                             | Ejemplo                                                                                             | Control recomendado                                                                      |
|------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| API de consultas       | S (Spoofing) - Un atacante se hace pasar por una aplicación legítima.      | Un atacante usa credenciales robadas de una aplicación para consultar la BD.                        | TLS mutuo y autenticación basada en certificados; JWT con corta expiración.              |
| Tabla de transacciones | T (Tampering) - Un atacante modifica registros de transacciones.           | Un empleado con acceso de administrador modifica el monto de una transferencia.                     | Firmas digitales en cada transacción; auditoría de cambios en la BD.                     |
| Logs de acceso         | R (Repudiation) - Un usuario niega haber accedido a datos sensibles.       | Un DBA afirma no haber consultado datos de clientes, pero los logs lo confirman.                    | Logs inmutables y firma digital de cada acceso.                                          |
| Datos de clientes      | I (Information Disclosure) - Un atacante accede a datos de clientes.       | Un atacante explota una vulnerabilidad de SQL Injection para extraer datos de la BD.                | Cifrado de datos sensibles (TDE + cifrado a nivel de columna); consultas parametrizadas. |
| Motor de BD            | D (Denial of Service) - Un ataque satura la BD, impidiendo consultas.      | Un atacante envía consultas complejas que agotan los recursos de la BD.                             | Rate limiting; optimización de consultas; escalado automático en la nube.                |
| Gestión de usuarios    | E (Elevation of Privilege) - Un usuario obtiene permisos de administrador. | Un atacante explota una vulnerabilidad en el sistema de autenticación para obtener permisos de DBA. | Principio de mínimo privilegio; separación de roles; MFA para cuentas de administrador.  |

### Encriptación de datos en bases de datos

El cifrado protege los datos incluso si el archivo de base de datos o el medio de almacenamiento es robado. Existen tres niveles principales de cifrado:

| Nivel de cifrado                              | Descripción                                                                                                  | Ventajas                                                      | Desventajas                                                                                          | Costo de implementación | Cuándo usarlo                                      |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-------------------------|----------------------------------------------------|
| En reposo (Transparent Data Encryption – TDE) | El motor de base de datos cifra automáticamente los datos al escribirlos en disco y los descifra al leerlos. | Transparente para las aplicaciones; sin cambios en el código. | No protege contra accesos no autorizados a través de consultas SQL (solo protege el archivo físico). | Bajo                    | Siempre que se almacenen datos sensibles en disco. |

| Nivel de cifrado             | Descripción                                                                                                  | Ventajas                                                                             | Desventajas                                                                                               | Costo de implementación | Cuándo usarlo                                                                               |
|------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------|
| <b>A nivel de columna</b>    | Se cifran columnas específicas (ej. números de tarjeta, cédula) usando funciones de cifrado del motor de BD. | Protege datos específicos incluso si el atacante tiene acceso a la BD.               | Requiere cambios en el esquema y en las consultas; puede afectar el rendimiento.                          | Medio                   | Datos altamente sensibles que deben protegerse incluso de administradores de BD.            |
| <b>A nivel de aplicación</b> | La aplicación cifra los datos antes de enviarlos a la BD y los descifra después de recuperarlos.             | La BD nunca ve los datos en texto plano; el administrador de BD no puede accederlos. | Mayor complejidad; las búsquedas sobre datos cifrados son limitadas (no se puede buscar por texto plano). | Alto                    | Datos que requieren el máximo nivel de protección (ej. claves privadas, datos biométricos). |

#### Ejemplo de cifrado a nivel de columna en PostgreSQL usando pgcrypto:

```

---sql
-- Habilitar la extensión
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Insertar con cifrado
INSERT INTO clientes (nombre, cedula_cifrada)
VALUES ('Juan Pérez', pgp_sym_encrypt('123-456789-0123A', 'clave_secreta'));

-- Consultar con descifrado
SELECT nombre, pgp_sym_decrypt(cedula_cifrada, 'clave_secreta') AS cedula
FROM clientes;
---
```

#### Políticas de respaldo y recuperación (backup y restore)

Una base de datos sin respaldo es una pérdida de datos asegurada. Las políticas de respaldo deben definirse según el **RPO (Recovery Point Objective)** y el **RTO (Recovery Time Objective)**.

| Métrica    | Definición                                                                               | Ejemplo                                                            | Implicación                                                                           |
|------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>RPO</b> | Cantidad máxima de datos que la organización está dispuesta a perder (medido en tiempo). | RPO = 1 hora → se pierde a lo sumo 1 hora de datos.                | La frecuencia de backup debe ser $\leq 1$ hora.                                       |
| <b>RTO</b> | Tiempo máximo para recuperar el servicio después de un desastre.                         | RTO = 4 horas → el sistema debe estar operativo en $\leq 4$ horas. | El proceso de restauración debe estar probado y documentado para cumplir con 4 horas. |

#### Estrategias de backup:

| Tipo de backup  | Descripción                                             | Velocidad de backup | Velocidad de restore | Espacio requerido |
|-----------------|---------------------------------------------------------|---------------------|----------------------|-------------------|
| <b>Completo</b> | Copia toda la base de datos. Base para los otros tipos. | Lenta               | Rápida               | Alto              |

| Tipo de backup     | Descripción                                                                     | Velocidad de backup | Velocidad de restore                                                      | Espacio requerido |
|--------------------|---------------------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------|-------------------|
| <b>Incremental</b> | Copia solo los datos cambiados desde el último backup (completo o incremental). | Rápida              | Lenta (requiere restaurar el completo + todos los incrementales en orden) | Bajo              |
| <b>Diferencial</b> | Copia los datos cambiados desde el último backup completo.                      | Media               | Media (requiere restaurar el completo + el último diferencial)            | Medio             |

#### Ejemplo de política de respaldos (para una BD de producción crítica):

| Frecuencia        | Tipo        | Retención | Ubicación                             |
|-------------------|-------------|-----------|---------------------------------------|
| Cada 6 horas      | Diferencial | 7 días    | Almacenamiento local (SSD rápido)     |
| Diario (12:00 AM) | Completo    | 30 días   | Almacenamiento en red (NAS)           |
| Semanal (domingo) | Completo    | 12 meses  | Almacenamiento externo (nube/offsite) |

**Regla de oro:** Los respaldos deben probarse periódicamente (al menos una vez al mes) restaurándolos en un entorno de pruebas. Un respaldo que no se prueba no es un respaldo.

#### Auditoría y monitoreo de bases de datos

La auditoría de bases de datos evalúa si los controles de seguridad son adecuados y si se cumplen las políticas y regulaciones. El monitoreo continuo permite detectar actividad anómala en tiempo real.

#### Componentes clave de la auditoría de bases de datos:

| Componente                    | Descripción                                                                                    | Herramientas                                                            | Frecuencia            |
|-------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------|
| <b>Logs de acceso</b>         | Registro de quién accede a la BD, desde dónde y qué consultas realiza.                         | PostgreSQL pgAudit, MySQL Enterprise Audit, AWS RDS Enhanced Monitoring | Continua (automática) |
| <b>Revisión de permisos</b>   | Verificar que los usuarios y aplicaciones tengan solo los permisos mínimos necesarios.         | Scripts personalizados, herramientas de auditoría (ej. DbProtect)       | Trimestral            |
| <b>Detección de anomalías</b> | Identificar patrones de acceso sospechosos (ej. consultas a las 3:00 AM desde una IP externa). | SIEM (Wazuh, ELK Stack), herramientas de monitoreo de BD                | Continua (automática) |
| <b>Cumplimiento normativo</b> | Verificar que la BD cumple con los requisitos de la Ley 787, PCI-DSS, etc.                     | Informes de auditoría, checklist de cumplimiento                        | Anual                 |

#### Ejemplo de checklist de auditoría de base de datos:

| Ítem de control                                                           | Cumple (Sí/No) | Evidencia                                                             | Riesgo si no cumple                                |
|---------------------------------------------------------------------------|----------------|-----------------------------------------------------------------------|----------------------------------------------------|
| ¿La base de datos tiene TDE habilitado?                                   |                | Configuración de TDE                                                  | Exposición de datos si el disco es robado.         |
| ¿Se auditan todos los accesos a datos sensibles (ej. números de tarjeta)? |                | Logs de auditoría de los últimos 30 días                              | Falta de evidencia forense en caso de brecha.      |
| ¿Las cuentas de administrador tienen MFA habilitado?                      |                | Captura de pantalla de configuración MFA                              | Toma de control de la BD por robo de credenciales. |
| ¿Se realizan backups automáticos y se prueban mensualmente?               |                | Logs de backup de los últimos 30 días; acta de prueba de restauración | Pérdida de datos en caso de incidente.             |

| Ítem de control                                                    | Cumple (Sí/No) | Evidencia                         | Riesgo si no cumple                  |
|--------------------------------------------------------------------|----------------|-----------------------------------|--------------------------------------|
| ¿Los usuarios tienen solo los permisos necesarios para su función? |                | Revisión de permisos (trimestral) | Ataque interno o movimiento lateral. |
| ¿Las consultas SQL son parametrizadas (sin concatenación)?         |                | Revisión de código (SAST)         | Vulnerabilidad a SQL Injection.      |

#### Métricas de monitoreo para bases de datos:

| KPI                                                 | Definición                                                 | Valor objetivo    |
|-----------------------------------------------------|------------------------------------------------------------|-------------------|
| <b>Tiempo medio de detección de acceso anómalo</b>  | Tiempo promedio entre un acceso sospechoso y su detección. | < 15 minutos      |
| <b>Cobertura de cifrado</b>                         | Porcentaje de datos sensibles cifrados en reposo.          | 100%              |
| <b>Tasa de éxito de backups</b>                     | Porcentaje de backups completados exitosamente.            | > 99%             |
| <b>Tiempo de restauración de backup</b>             | Tiempo promedio para restaurar la BD desde un backup.      | < RTO establecido |
| <b>Número de vulnerabilidades críticas en la BD</b> | Vulnerabilidades CVSS $\geq$ 9.0 identificadas en la BD.   | 0                 |

#### Seguridad en bases de datos en la nube

Con la migración masiva a la nube, es crucial entender cómo se aplican los conceptos de seguridad en este entorno. El modelo de responsabilidad compartida es fundamental:

| Nivel de servicio                        | Responsabilidad del proveedor                             | Responsabilidad del cliente                                                              |
|------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>IaaS</b> (BD en VM)                   | Virtualización, almacenamiento físico, red                | SO, motor de BD, parches, cifrado, acceso, backups                                       |
| <b>PaaS</b> (BD como servicio - DBaaS)   | Motor de BD, parches, disponibilidad, backups automáticos | Configuración de acceso, cifrado (opcional), gestión de usuarios, cumplimiento normativo |
| <b>SaaS</b> (BD integrada en aplicación) | Toda la pila (BD, aplicación, infraestructura)            | Configuración de usuarios, políticas de acceso, integración con identidad                |

#### Ejemplo: Seguridad en AWS RDS (Base de Datos como Servicio)

| Responsabilidad del proveedor (AWS)                       | Responsabilidad del cliente                                |
|-----------------------------------------------------------|------------------------------------------------------------|
| Parcheo del motor de BD (ej. MySQL, PostgreSQL)           | Configuración de grupos de seguridad (firewall)            |
| Replicación automática y backups (retención configurable) | Habilitar cifrado en reposo (TDE) y en tránsito (TLS)      |
| Disponibilidad (multi-AZ, failover automático)            | Gestión de usuarios y permisos (RBAC/ABAC)                 |
| Cumplimiento de certificaciones (ISO 27001, SOC 2)        | Cumplimiento de la Ley 787 (configuración de logs, acceso) |

**Práctica recomendada:** Siempre cifrar los datos en reposo en la nube, incluso si el proveedor ofrece cifrado por defecto. El cifrado proporciona una capa adicional de protección en caso de que un atacante obtenga acceso a los discos subyacentes.

#### Seguridad en bases de datos NoSQL

Las bases de datos NoSQL (MongoDB, Cassandra, Redis, Elasticsearch) presentan desafíos de seguridad diferentes a los de las bases de datos relacionales.

| Desafío de seguridad                        | Ejemplo                                                                                                                                  | Mitigación                                                                                             |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Inyección NoSQL</b>                      | Un atacante envía <code>{ \$ne: null }</code> a un campo de login, que siempre devuelve verdadero, permitiendo el acceso sin contraseña. | Validar y sanitizar todas las entradas; usar consultas parametrizadas (si el driver lo soporta).       |
| <b>Falta de transacciones ACID</b>          | En MongoDB (versiones anteriores), una operación podía completarse parcialmente, dejando datos inconsistentes.                           | Usar bases de datos que soporten transacciones (MongoDB 4.0+, Cassandra con Lightweight Transactions). |
| <b>Acceso por defecto sin autenticación</b> | MongoDB y Redis, por defecto, no requieren autenticación, lo que ha llevado a millones de bases de datos expuestas en Internet.          | Habilitar autenticación y crear usuarios con permisos mínimos; no exponer la BD a Internet.            |
| <b>Escalado horizontal</b>                  | En clusters grandes, la gestión de permisos y auditoría es más compleja.                                                                 | Usar herramientas de gestión de identidad (ej. LDAP, Active Directory) integradas con la BD.           |

**Ejemplo de caso real:** En 2019, un investigador de seguridad encontró más de 24,000 bases de datos MongoDB expuestas en Internet sin autenticación, conteniendo datos personales de millones de usuarios. La lección: **nunca** exponer una base de datos a Internet sin autenticación y firewalls.

#### Práctica recomendada para NoSQL:

1. Habilitar autenticación desde el primer momento.
2. Usar firewalls para restringir el acceso solo a las aplicaciones autorizadas.
3. Cifrar los datos en reposo y en tránsito.
4. Auditar todos los accesos y acciones administrativas.

#### Bases de datos y cumplimiento normativo

Las bases de datos deben diseñarse para cumplir con las regulaciones aplicables. En Nicaragua, la **Ley 787 de Protección de Datos Personales** establece requisitos específicos:

| Requisito de la Ley 787         | Implicación para la base de datos                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Consentimiento explícito</b> | La BD debe registrar el consentimiento del usuario para almacenar sus datos.                                     |
| <b>Derecho de acceso</b>        | La BD debe permitir consultar los datos de un usuario específico de manera rápida y completa.                    |
| <b>Derecho de rectificación</b> | La BD debe permitir actualizar los datos de un usuario.                                                          |
| <b>Derecho de cancelación</b>   | La BD debe permitir eliminar los datos de un usuario (o anonimizarlos).                                          |
| <b>Seguridad de los datos</b>   | La BD debe implementar cifrado, control de acceso, logs de acceso y backups.                                     |
| <b>Notificación de brechas</b>  | La BD debe tener mecanismos para detectar accesos no autorizados y notificar a los afectados en un plazo máximo. |

#### Ejemplo de cumplimiento en una BD universitaria:

La Universidad Nacional "Héroes de San José de las Mulass" implementó las siguientes medidas para cumplir con la Ley 787:

- Se cifraron las cédulas, direcciones y datos de salud de los estudiantes usando TDE + cifrado a nivel de columna.
- Se implementaron logs de acceso a datos sensibles con alertas en tiempo real (Wazuh).
- Se creó un módulo de autoportal donde los estudiantes pueden ver, corregir y solicitar la eliminación de sus datos.

- Se estableció un RPO de 1 hora y un RTO de 4 horas para la BD, con backups automáticos a la nube.

## Caso integrador: Diseño de seguridad para una base de datos de una clínica médica

### Escenario:

"Clínica Salud Integral, S.A." es una clínica privada en Managua que atiende a 500 pacientes al mes. Actualmente:

- Los datos de pacientes (nombres, direcciones, teléfonos, diagnósticos) están en una hoja de cálculo Excel compartida en una carpeta de red.
- No hay control de acceso; todos los empleados (10) pueden ver y modificar cualquier dato.
- No hay backups.
- La clínica planea migrar a una base de datos para gestionar historias clínicas electrónicas.

### Requisitos de la nueva base de datos:

1. Almacenar historias clínicas electrónicas de forma segura.
2. Solo médicos autorizados pueden ver las historias clínicas de sus pacientes.
3. Cumplir con la Ley 787 de Protección de Datos Personales.
4. RPO de 30 minutos, RTO de 2 horas.
5. Presupuesto máximo para la BD: C\$ 80,000.

### Diseño de la solución:

| Componente                           | Solución propuesta                                                                                                          | Justificación                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Motor de BD</b>                   | PostgreSQL 16 (open source)                                                                                                 | Soporte para cifrado (pgcrypto), transacciones ACID, y buena comunidad. |
| <b>Cifrado en reposo</b>             | TDE en PostgreSQL (cifrado de tablaspace)                                                                                   | Protege los datos en el disco.                                          |
| <b>Cifrado de columnas sensibles</b> | pgcrypto para cifrar diagnóstico y nombre completo                                                                          | Incluso el administrador de BD no puede leer los datos sin la clave.    |
| <b>Control de acceso</b>             | RBAC con roles: Médico (solo pacientes asignados), Administrativo (solo datos de contacto), Auditor (solo lectura de logs). | Principio de privilegio mínimo.                                         |
| <b>Autenticación</b>                 | Integración con LDAP + MFA para acceder a la aplicación.                                                                    | Protege contra robo de credenciales.                                    |
| <b>Auditoría</b>                     | Wazuh para monitoreo centralizado; logs de todos los accesos a historias clínicas.                                          | Detecta accesos no autorizados.                                         |
| <b>Backups</b>                       | Backup diferencial cada 30 minutos (RPO = 30 min) + completo diario + completo semanal a la nube.                           | Cumple RPO y RTO.                                                       |
| <b>Plan de contingencia</b>          | Procedimiento documentado para operar manualmente (papel) durante 4 horas si el sistema falla.                              | Cumple RTO de 2 horas.                                                  |

### Presupuesto estimado para la BD:

| Concepto                               | Costo (C\$) |
|----------------------------------------|-------------|
| Servidor en la nube (VPS) por 12 meses | 24,000      |
| PostgreSQL (open source)               | 0           |
| pgcrypto (extensión de PostgreSQL)     | 0           |

| Concepto                                                     | Costo (C\$)   |
|--------------------------------------------------------------|---------------|
| Wazuh (monitoreo)                                            | 0             |
| Backups automáticos a la nube (servicio)                     | 6,000/año     |
| Horas de consultoría (40 horas × C\$ 500/h)                  | 20,000        |
| Capacitación al personal (10 personas × 8 horas × C\$ 300/h) | 24,000        |
| <b>Subtotal</b>                                              | <b>74,000</b> |
| Reserva para imprevistos (10%)                               | 7,400         |
| <b>Total</b>                                                 | <b>81,400</b> |

*Nota: El presupuesto excede ligeramente el límite de C\$ 80,000. Se recomienda reducir el alcance inicial (ej. implementar MFA en una segunda fase) o buscar financiamiento adicional.*

#### Lecciones aprendidas del caso:

1. La migración de datos (de Excel a base de datos) requiere una limpieza y validación significativa.
2. La capacitación del personal es tan importante como la tecnología; una base de datos segura con usuarios no capacitados sigue siendo insegura.
3. El cumplimiento de la Ley 787 no es opcional; los costos de no cumplir (multas, pérdida de reputación) superan ampliamente los costos de implementación.
4. El uso de herramientas open source (PostgreSQL, pgcrypto, Wazuh) permite implementar controles robustos con un presupuesto limitado.

#### Comprobación de aprendizaje

**Ejercicio 3.3.1:** Clasifique los siguientes datos en orden del nivel de cifrado requerido (1 = solo TDE, 2 = columna, 3 = aplicación):

- a) Nombres de empleados en una intranet corporativa. → \_\_\_\_\_
- b) Números de tarjeta de crédito en una pasarela de pagos. → \_\_\_\_\_
- c) Claves privadas RSA utilizadas para firmar documentos digitales. → \_\_\_\_\_

*Respuestas esperadas:* a) 1 (bajo riesgo); b) 2 o 3 (según el estándar PCI DSS, se requiere cifrado fuerte); c) 3 (máxima protección).

**Ejercicio 3.3.2:** Una empresa tiene un backup completo diario que tarda 6 horas en restaurarse. ¿Qué RTO puede ofrecer como máximo? ¿Es aceptable si el negocio requiere RTO < 2 horas?

*Respuesta:* Como mínimo, el RTO es 6 horas (el tiempo de restauración). No es aceptable porque  $6 > 2$ . Se debe optimizar: usar backup diferencial + FULL más rápido (ej. SSD), o implementar alta disponibilidad (replicación síncrona) para que el failover sea inmediato.

**Ejercicio 3.3.3 (Aplicación de STRIDE):** Para una base de datos de una plataforma de comercio electrónico, identifique una amenaza para cada categoría STRIDE y proponga un control arquitectónico.

| Categoría        | Amenaza                                                               | Control                                                |
|------------------|-----------------------------------------------------------------------|--------------------------------------------------------|
| Suplantación (S) | Un atacante roba las credenciales de un usuario para acceder a la BD. | MFA y autenticación basada en certificados.            |
| Manipulación (T) | Un empleado modifica el precio de un producto en la BD.               | Auditoría de cambios y firmas digitales.               |
| Repudio (R)      | Un usuario niega haber realizado una compra.                          | Logs inmutables con firma digital de cada transacción. |
| Revelación (I)   | Un atacante accede a los números de tarjeta de los clientes.          | Cifrado de datos sensibles (columnas de tarjetas).     |

| Categoría      | Amenaza                                                | Control                                               |
|----------------|--------------------------------------------------------|-------------------------------------------------------|
| Denegación (D) | Un ataque DDoS satura la BD, impidiendo consultas.     | Rate limiting y WAF.                                  |
| Elevación (E)  | Un usuario estándar obtiene permisos de administrador. | Principio de mínimo privilegio y separación de roles. |

### Ejercicio 3.3.4 (Caso práctico):

Una PYME de logística en Nicaragua almacena los datos de sus 5,000 clientes (nombres, direcciones, teléfonos, historial de compras) en una base de datos MySQL que no tiene cifrado, no tiene backups y está expuesta a Internet en el puerto 3306 (por defecto). Recientemente, un atacante explotó una vulnerabilidad de SQL Injection en el sistema de gestión de clientes y descargó toda la base de datos.

- Identifique al menos 5 fallos de seguridad en la configuración de la base de datos.
- Proponga un plan de remediación con plazos (corto, mediano y largo plazo).
- ¿Cómo justificaría la inversión en seguridad ante el gerente general?

*Respuesta esperada (abierta, pero debe incluir):*

- Fallos: 1) Sin cifrado en reposo (TDE); 2) Sin backups; 3) Puerto 3306 expuesto a Internet; 4) Vulnerabilidad a SQL Injection; 5) Sin auditoría de accesos.
- Plan: Corto plazo (48h): cerrar el puerto 3306 desde Internet, aplicar parches, cambiar contraseñas; Mediano plazo (2 semanas): implementar TDE, backups automáticos, consultas parametrizadas; Largo plazo (3 meses): migrar a la nube con DBaaS, implementar SIEM, capacitación.
- Justificación: Costo de la multa potencial (Ley 787), pérdida de clientes por daño reputacional, costo de recuperación de datos perdidos.

### Ejercicio 3.3.5 (Reflexión):

¿Por qué cree que muchas organizaciones descuidan la seguridad de las bases de datos a pesar de ser el activo más valioso? Mencione al menos tres razones y proponga cómo abordarlas.

*Respuesta esperada (abierta, pero debe incluir):*

- Razones: 1) Costo percibido como alto (cifrado, backups); 2) Falta de conciencia sobre los riesgos; 3) Complejidad técnica (no saben cómo implementar cifrado o backups).
- Abordajes: 1) Presentar el costo de una brecha vs. el costo de los controles; 2) Educar a los directivos con casos reales (ej. el de la PYME); 3) Implementar soluciones open source y escalonadas (empezar con TDE, luego cifrado de columnas).

## Sistemas Operativos

Los sistemas operativos (SO) son la plataforma base sobre la que se ejecutan las aplicaciones y servicios. Un sistema operativo comprometido pone en riesgo todos los datos y aplicaciones que aloja. La seguridad de sistemas operativos abarca la configuración segura (hardening), la gestión de parches, el monitoreo y la respuesta a incidentes.

Esta sección se conecta directamente con las unidades anteriores:

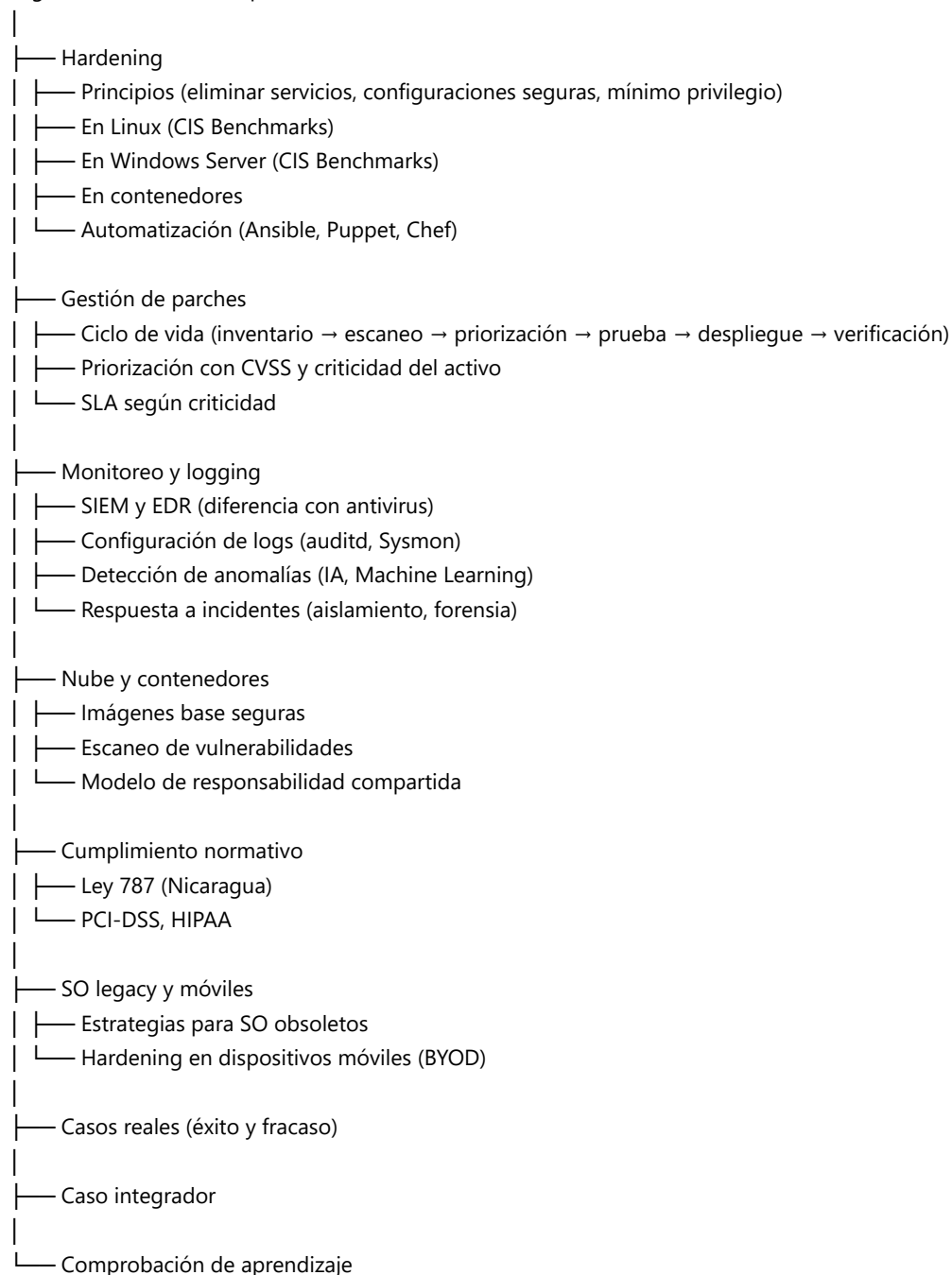
- En la **Unidad I** aprendimos a evaluar la **viabilidad técnica** de un proyecto. La elección del sistema operativo y su configuración de seguridad son determinantes para esa viabilidad.
- En la **Unidad II** planificamos e ingeniamos el proceso. Las decisiones de hardening, parches y monitoreo deben definirse durante la ingeniería del proceso.
- En la **Unidad III**, la seguridad de los sistemas operativos es la base sobre la que se construye la seguridad de toda la infraestructura. Un SO inseguro es como una puerta abierta en una fortaleza.

**Pregunta guía antes de comenzar:** Piensa en el sistema operativo de tu computadora personal. ¿Sabes si tiene todas las actualizaciones de seguridad? ¿Tienes un firewall activo? ¿Crees que está configurado de forma segura? ¿Qué pasaría si un atacante lo comprometiera?

## Mapa conceptual de la sección

???text

### Seguridad en Sistemas Operativos



???

### Recuadro de definiciones clave

| Término                                           | Definición simple                                                                                                                                     |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hardening</b>                                  | Proceso de asegurar un sistema operativo reduciendo su superficie de ataque (eliminar servicios innecesarios, aplicar configuraciones seguras, etc.). |
| <b>CIS Benchmarks</b>                             | Guías de configuración segura publicadas por el Center for Internet Security, el estándar de facto para hardening de sistemas.                        |
| <b>CVSS (Common Vulnerability Scoring System)</b> | Sistema estandarizado de puntuación de vulnerabilidades (0-10), donde 10 es el riesgo más crítico.                                                    |

| Término                                      | Definición simple                                                                                       |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>SIEM</b>                                  | Sistema que centraliza y correlaciona eventos de seguridad de múltiples fuentes para detectar amenazas. |
| <b>EDR (Endpoint Detection and Response)</b> | Solución que monitorea endpoints en busca de comportamientos anómalos y permite responder a incidentes. |
| <b>Parche de seguridad</b>                   | Actualización de software diseñada para corregir una vulnerabilidad de seguridad identificada.          |

### Hardening de sistemas operativos

El **hardening** es el proceso de asegurar un sistema operativo reduciendo su superficie de ataque. Podemos pensarlo como **poner cerraduras adicionales y quitar ventanas innecesarias en una casa**: se eliminan servicios innecesarios, se aplican configuraciones seguras, se gestionan parches y se implementan controles de acceso estrictos.

#### Principios generales de hardening:

| Principio                               | Descripción                                                                           | Ejemplo concreto                                                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Eliminar servicios innecesarios</b>  | Todo servicio que no sea estrictamente necesario debe desinstalarse o deshabilitarse. | Deshabilitar Telnet, FTP, servicios de impresión si no se usan.                               |
| <b>Configuración segura por defecto</b> | Los parámetros predeterminados inseguros deben modificarse.                           | Cambiar contraseñas por defecto; deshabilitar login como root vía SSH.                        |
| <b>Principio de privilegio mínimo</b>   | Los usuarios y procesos deben ejecutarse con los mínimos privilegios.                 | Las aplicaciones web no deben ejecutarse como root/admin; usar cuentas de servicio dedicadas. |
| <b>Parches y actualizaciones</b>        | Mantener el SO actualizado con los últimos parches de seguridad.                      | Habilitar actualizaciones automáticas de seguridad; probar parches críticos en 48 horas.      |
| <b>Monitoreo y logging</b>              | Registrar eventos relevantes y revisarlos periódicamente.                             | Configurar auditd en Linux o Advanced Audit Policy en Windows.                                |
| <b>Segmentación y aislamiento</b>       | Separar servicios y aplicaciones en contenedores, máquinas virtuales o zonas de red.  | Ejecutar cada microservicio en su propio contenedor con recursos limitados.                   |

### Hardening en Linux (basado en CIS Benchmarks)

El **Center for Internet Security (CIS)** es una organización sin fines de lucro fundada en 2000 que desarrolla y mantiene los **CIS Benchmarks**, guías de configuración segura que se han convertido en el estándar de facto en la industria. Estos benchmarks son utilizados por gobiernos, empresas y organizaciones de todo el mundo (incluyendo agencias gubernamentales de EE.UU.) como referencia para configuraciones seguras de sistemas operativos, aplicaciones y dispositivos de red.

A continuación, los controles más importantes para Linux (Ubuntu 24.04 LTS, la versión más reciente con soporte a largo plazo):

| Control CIS  | Descripción                                                                                      | Comando o acción                                                                                               | Explicación del comando                                                                                                                                                     |
|--------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1.1.1</b> | Configurar particiones separadas para <code>/tmp</code> , <code>/var</code> , <code>/home</code> | Usar LVM o particiones dedicadas; montar <code>/tmp</code> con <code>noexec,nosuid,nodev</code>                | <code>noexec</code> evita la ejecución de programas desde <code>/tmp</code> , <code>nosuid</code> bloquea bits SUID, <code>nodev</code> impide la creación de dispositivos. |
| <b>1.5.1</b> | Configurar el bootloader con contraseña                                                          | <code>grub-mkpasswd-pbkdf2</code> y agregar <code>password_pbkdf2</code> en <code>/etc/grub.d/40_custom</code> | Protege el arranque del sistema para que solo personas autorizadas puedan modificar los parámetros de arranque.                                                             |

| Control CIS | Descripción                                                                     | Comando o acción                                                                | Explicación del comando                                                                                                      |
|-------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 3.2.1       | Deshabilitar IP forwarding (a menos que sea un router)                          | <code>sysctl -w net.ipv4.ip_forward=0</code> y en <code>/etc/sysctl.conf</code> | Evita que el servidor actúe como router, lo que podría usarse para ataques de red.                                           |
| 4.1.1       | Habilitar auditd para registrar eventos                                         | <code>apt install auditd;</code><br><code>auditctl -e 1</code>                  | <code>auditd</code> registra todos los eventos de seguridad del sistema para auditoría forense.                              |
| 5.1.1       | Restringir el uso de cron a usuarios autorizados                                | Crear <code>/etc/cron.allow</code> con solo los usuarios que necesitan cron     | Evita que usuarios no autorizados programen tareas automáticas.                                                              |
| 5.2.1       | Configurar SSH con clave pública y deshabilitar login por contraseña            | <code>PasswordAuthentication no</code> en <code>/etc/ssh/sshd_config</code>     | Obliga a usar autenticación por clave pública, mucho más segura que contraseñas.                                             |
| 5.4.1       | Establecer política de contraseñas: expiración cada 90 días, longitud $\geq 14$ | Editar <code>/etc/login.defs</code> y <code>/etc/pam.d/common-password</code>   | Configura la política de contraseñas para cumplir con estándares de seguridad.                                               |
| 6.1.1       | Verificar que los archivos del sistema tengan permisos correctos                | <code>apt install debsums;</code><br><code>debsums -c</code>                    | <code>debsums</code> verifica la integridad de los archivos del sistema comparando con las sumas de verificación originales. |

**Nota importante:** Estos son solo algunos de los cientos de controles que CIS recomienda. Para un sistema en producción, se recomienda implementar al menos el **nivel 1** (perfil básico) y, si el riesgo lo justifica, el **nivel 2** (perfil más restrictivo).

#### Hardening en Windows Server (basado en CIS Benchmarks)

| Control CIS | Descripción                                                          | Acción en Windows Server                                                                                    | Explicación                                                                                                            |
|-------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1.1.1       | Mantener el sistema actualizado                                      | Configurar Windows Update para recibir parches de seguridad automáticamente (WSUS o Windows Update directo) | Garantiza que los parches críticos se apliquen sin demora.                                                             |
| 2.1.1       | Deshabilitar cuentas locales innecesarias                            | Deshabilitar Invitado; renombrar Administrador; crear cuentas de servicio con privilegios mínimos           | Reduce la superficie de ataque al eliminar cuentas que podrían ser explotadas.                                         |
| 2.2.1       | Configurar política de contraseñas                                   | Longitud mínima $\geq 14$ ; expiración cada 60 días; historial de 24 contraseñas                            | Cumple con los estándares de seguridad (ej. NIST, CIS).                                                                |
| 2.3.1       | Configurar bloqueo de cuenta tras 5 intentos fallidos                | Account lockout threshold = 5; lockout duration = 30 minutos                                                | Previene ataques de fuerza bruta al bloquear la cuenta después de varios intentos fallidos.                            |
| 5.1.1       | Deshabilitar servicios innecesarios                                  | Deshabilitar Print Spooler si no se usa; deshabilitar SMBv1                                                 | SMBv1 es un protocolo antiguo y vulnerable; Print Spooler ha sido explotado en múltiples ataques (ej. PrintNightmare). |
| 9.1.1       | Habilitar Windows Defender y mantenerlo actualizado                  | Configurar Microsoft Defender Antivirus con protección en tiempo real y envío de muestras                   | Windows Defender es un antivirus integrado y efectivo que debe mantenerse activo.                                      |
| 10.1.1      | Configurar el firewall de Windows con reglas de entrada restrictivas | Bloquear todo el tráfico entrante excepto puertos necesarios (3389 solo con VPN; 443; 80)                   | Reduce drásticamente la superficie de ataque al cerrar puertos innecesarios.                                           |

| Control CIS | Descripción                 | Acción en Windows Server                                                                  | Explicación                                         |
|-------------|-----------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------|
| 12.1.1      | Configurar logging avanzado | Habilitar auditoría de inicios de sesión, cambios de cuentas, accesos a objetos sensibles | Proporciona evidencia forense en caso de incidente. |

## Hardening en contenedores

Los contenedores (Docker, Kubernetes) son cada vez más comunes en entornos de producción. Su hardening requiere consideraciones específicas:

| Área                        | Medida de hardening                                                                                       | Verificación                                                                        | Prioridad |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------|
| Imagen base                 | Usar imágenes base oficiales y actualizadas (ej. Alpine, Ubuntu LTS) con vulnerabilidades resueltas.      | <code>docker scan</code> o <code>trivy image</code> para escanear vulnerabilidades. | Alta      |
| Privilegios                 | No ejecutar contenedores como root; usar <code>runAsNonRoot: true</code> en Kubernetes.                   | <code>docker run --user 1000:1000</code>                                            | Crítica   |
| Sistema de archivos         | Montar el sistema de archivos como <code>readonly</code> cuando sea posible.                              | <code>docker run --read-only</code>                                                 | Alta      |
| Capabilities                | Eliminar capacidades innecesarias de Linux (ej. <code>--cap-drop=ALL --cap-add=NET_BIND_SERVICE</code> ). | <code>docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE</code>                   | Media     |
| Escaneo de vulnerabilidades | Escanear imágenes antes del despliegue (CI/CD) y periódicamente.                                          | Trivy, Snyk, Gype en el pipeline de CI/CD.                                          | Alta      |
| Políticas de red            | Usar NetworkPolicy en Kubernetes para restringir comunicaciones entre pods.                               | <code>deny-all</code> por defecto, solo permitir tráfico explícito.                 | Alta      |

## Ejemplo de Dockerfile seguro:

```

FROM alpine:3.18
RUN addgroup -g 1000 -S appuser && adduser -u 1000 -S appuser -G appuser
USER appuser
COPY --chown=appuser:appuser app /app
WORKDIR /app
EXPOSE 8080
CMD ["node", "server.js"]

```

## Automatización del hardening con Ansible, Puppet y Chef

En entornos con decenas o cientos de servidores, aplicar hardening manualmente es inviable. Herramientas como **Ansible**, **Puppet** y **Chef** permiten automatizar la configuración segura a escala. Además, herramientas como **CIS-CAT** permiten validar automáticamente el cumplimiento de los CIS Benchmarks.

## Ejemplo de playbook de Ansible para hardening básico en Linux:

```

---

```

- name: Hardening básico para servidores Ubuntu
- hosts: all
- become: yes
- tasks:

- name: Deshabilitar servicios innecesarios  
systemd:  
name: "{{ item }}"  
state: stopped  
enabled: no  
loop:
  - telnet
  - ftp
  - cups
- name: Configurar SSH (deshabilitar root login y contraseñas)  
lineinfile:  
path: /etc/ssh/sshd\_config  
regexp: "{{ item.regexp }}"  
line: "{{ item.line }}"  
state: present  
loop:
  - { regexp: '^PermitRootLogin', line: 'PermitRootLogin no' }
  - { regexp: '^PasswordAuthentication', line: 'PasswordAuthentication no' }
- name: Configurar UFW (firewall) - permitir solo puertos necesarios  
ufw:  
rule: allow  
port: "{{ item }}"  
proto: tcp  
loop:
  - 22 # SSH (puerto cambiado)
  - 80 # HTTP
  - 443 # HTTPS
- name: Habilitar y activar UFW  
ufw:  
state: enabled
- name: Instalar y configurar auditd  
apt:  
name: auditd  
state: present  
notify: restart auditd
- name: Configurar reglas de auditd  
copy:  
dest: /etc/audit/rules.d/audit.rules  
content: |  
-w /etc/passwd -p wa -k passwd\_changes  
-w /etc/shadow -p wa -k shadow\_changes  
-w /etc/ssh/sshd\_config -p wa -k ssh\_config  
???

#### Ventajas de la automatización:

1. **Consistencia:** Todos los servidores tienen la misma configuración segura.
2. **Escalabilidad:** Se puede aplicar a cientos de servidores en minutos.
3. **Auditoría:** Los cambios quedan registrados en el control de versiones (Git).
4. **Repetibilidad:** Se puede reaplicar fácilmente si un servidor se desvía de la configuración base.

## Ejemplo de validación con CIS-CAT:

???bash

## Descargar el evaluador de CIS

wget https://tools.cisecurity.org/cis-cat/cis-cat-full.zip

## Ejecutar la evaluación

java -jar CIS-CAT.jar -t -p CIS\_Ubuntu\_Linux\_24.04\_LTS\_Level\_1.xml -r results.html

???

### Gestión de parches y vulnerabilidades

La gestión de parches es un proceso continuo que incluye:

#### Diagrama de flujo del ciclo de vida de la gestión de parches:

???mermaid

graph TD

A[Inventario de sistemas] --> B[Escaneo de vulnerabilidades]

B --> C[Priorización según CVSS y criticidad]

C --> D[Prueba en entorno de staging]

D --> E{¿Prueba exitosa?}

E -->|No| F[Revisar y ajustar]

F --> D

E -->|Sí| G[Despliegue en producción]

G --> H[Verificación de parches aplicados]

H --> I[Documentación y reporte]

I --> A

???

| Paso                               | Descripción                                                                                 | Herramientas                             | Frecuencia                                                       |
|------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------|
| <b>Inventario</b>                  | Identificar todos los sistemas, SO, aplicaciones y versiones.                               | GLPI, OCS Inventory, Lansweeper          | Continuo (diario)                                                |
| <b>Escaneo de vulnerabilidades</b> | Detectar vulnerabilidades conocidas (CVE) en los sistemas.                                  | Nessus, OpenVAS, Qualys                  | Semanal o mensual                                                |
| <b>Evaluación de riesgo</b>        | Priorizar parches según CVSS (Common Vulnerability Scoring System) y criticidad del activo. | Tablero con CVSS + criticidad del activo | Cada escaneo                                                     |
| <b>Prueba</b>                      | Aplicar parches en un entorno de pruebas antes de producción.                               | Entorno de staging idéntico a producción | Antes de cada parche crítico                                     |
| <b>Despliegue</b>                  | Aplicar parches en producción siguiendo una ventana de mantenimiento.                       | WSUS, SCCM, Ansible, Puppet              | Según criticidad: crítico ≤ 48h, alto ≤ 2 semanas, medio ≤ 1 mes |
| <b>Verificación</b>                | Confirmar que el parche se aplicó correctamente y que el sistema sigue funcionando.         | Escaneo posterior + pruebas funcionales  | Inmediatamente después del despliegue                            |

### Priorización de parches con matriz de criticidad:

La priorización no debe basarse solo en el CVSS; debe considerar también la criticidad del activo. Una vulnerabilidad CVSS 7.0 en un servidor web público (crítico) es más urgente que una CVSS 9.0 en un servidor de pruebas (bajo).

| Criticidad del activo                           | CVSS 9.0-10.0<br>(Crítico) | CVSS 7.0-8.9<br>(Alto) | CVSS 4.0-6.9<br>(Medio) | CVSS 0.1-3.9<br>(Bajo) |
|-------------------------------------------------|----------------------------|------------------------|-------------------------|------------------------|
| <b>Crítico</b> (servidores de producción, BD)   | 24h                        | 48h                    | 7 días                  | 30 días                |
| <b>Alto</b> (aplicaciones internas importantes) | 48h                        | 7 días                 | 14 días                 | 60 días                |
| <b>Medio</b> (sistemas de soporte)              | 7 días                     | 14 días                | 30 días                 | 90 días                |
| <b>Bajo</b> (entornos de prueba)                | 14 días                    | 30 días                | 60 días                 | 120 días               |

### Ejemplo de vulnerabilidad crítica reciente (CVE-2021-44228 – Log4Shell):

- **CVSS:** 10.0 (crítico)
- **Descripción:** Vulnerabilidad en Apache Log4j que permite ejecución remota de código sin autenticación.
- **Mitigación inmediata:** Actualizar a Log4j 2.17.0 o eliminar la clase JndiLookup. Mientras tanto, bloquear a nivel de WAF las peticiones que contengan `${jndi:}`.
- **Lección:** Mantener un inventario actualizado de dependencias (especialmente librerías de logging) es fundamental; muchas organizaciones tardaron semanas en saber si usaban Log4j.

### Monitoreo de seguridad y logging

Un sistema operativo sin monitoreo es ciego ante los ataques. Los principales componentes de monitoreo son:

| Componente                                              | Herramientas (Open Source)                                         | Lo que detecta                                                                                         |
|---------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>SIEM (Security Information and Event Management)</b> | Wazuh, ELK Stack (Elasticsearch + Logstash + Kibana), Grafana Loki | Correlación de eventos, alertas en tiempo real, dashboards de seguridad                                |
| <b>IDS/IPS (Host-based)</b>                             | OSSEC, Wazuh, AIDE (archivos críticos)                             | Modificaciones no autorizadas de archivos, conexiones sospechosas, intentos de escalada de privilegios |
| <b>EDR (Endpoint Detection and Response)</b>            | Wazuh (módulo EDR), osquery                                        | Detección de comportamiento anómalo en endpoints, procesos maliciosos, persistencia                    |
| <b>Gestión de logs centralizada</b>                     | rsyslog + Logstash, Fluentd, Winlogbeat                            | Recolección unificada de logs de todos los sistemas                                                    |

### Comparativa de herramientas de monitoreo:

| Herramienta      | Ventajas                                                 | Desventajas                                  | Costo                  | Ideal para...                                   |
|------------------|----------------------------------------------------------|----------------------------------------------|------------------------|-------------------------------------------------|
| <b>Wazuh</b>     | Open source, integra SIEM y EDR, comunidad activa        | Curva de aprendizaje media                   | Gratuito (open source) | PYMES y organizaciones con presupuesto limitado |
| <b>OSSEC</b>     | Ligero, maduro, estable                                  | Sin interfaz web nativa, difícil de escalar  | Gratuito (open source) | Entornos pequeños y medianos                    |
| <b>ELK Stack</b> | Potente, altamente personalizable, integración con Wazuh | Complejo de configurar, requiere experiencia | Gratuito (open source) | Organizaciones con equipo de TI dedicado        |

| Herramienta   | Ventajas                                            | Desventajas                                          | Costo            | Ideal para...                           |
|---------------|-----------------------------------------------------|------------------------------------------------------|------------------|-----------------------------------------|
| <b>Splunk</b> | Interfaz amigable, potentes capacidades de búsqueda | Muy costoso, requiere licencias por volumen de datos | Alto (comercial) | Grandes empresas con presupuesto amplio |

#### Ejemplo de configuración de logging en Linux (auditd):

```
???bash
```

## Instalar auditd

```
apt install auditd
```

## Regla: monitorear cambios en /etc/passwd y /etc/shadow

```
auditctl -w /etc/passwd -p wa -k passwd_changes
auditctl -w /etc/shadow -p wa -k shadow_changes
```

## Regla: monitorear intentos de acceso fallidos (todos los ejecutables de login)

```
auditctl -w /var/log/auth.log -p r -k auth_log
```

## Ver logs

```
ausearch -k passwd_changes
???
```

#### Ejemplo de detección de ataque con Wazuh:

| Evento                                   | Regla Wazuh                                       | Alerta generada                                                | Acción                                                                   |
|------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------|
| 10 intentos fallidos de SSH en 5 minutos | Regla 5710 (Multiple SSH authentication failures) | "Posible ataque de fuerza bruta SSH desde IP 192.168.1.100"    | Bloquear IP en firewall automáticamente (Fail2ban o integración con API) |
| Modificación de /etc/shadow              | Regla 5501 (File integrity monitoring)            | "El archivo /etc/shadow ha sido modificado por usuario nobody" | Investigar inmediatamente; podría ser escalada de privilegios            |
| Nuevo servicio instalado                 | Regla 5302 (New service added)                    | "Se ha añadido el servicio sshd_trojan en el sistema"          | Verificar firma del paquete; si no es legítimo, aislar el sistema        |

#### Configuración de regla de detección de fuerza bruta en Wazuh:

```
???xml
```

```
<if_group>sshd</if_group>
Failed password|Invalid user
10
300
Posible ataque de fuerza bruta SSH: más de 10 intentos fallidos en 5 minutos
<same_source_ip>yes</same_source_ip>
```

```
???
```

---

## EDR vs. Antivirus tradicional: diferencias clave

Es fundamental entender la diferencia entre un antivirus tradicional y una solución EDR (Endpoint Detection and Response), ya que representan enfoques distintos de seguridad.

| Aspecto           | Antivirus tradicional                                      | EDR                                                                                   |
|-------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Enfoque           | Basado en firmas (detección de malware conocido)           | Basado en comportamiento (detección de actividad anómala)                             |
| Detección         | Detecta malware conocido mediante bases de datos de firmas | Detecta comportamientos sospechosos (ej. ejecución de scripts, movimientos laterales) |
| Respuesta         | Solo alerta o elimina el archivo malicioso                 | Permite investigar, contener y remediar el incidente (ej. aislar el endpoint)         |
| Visibilidad       | Limitada al archivo en sí                                  | Ofrece visibilidad de procesos, conexiones de red, cambios en el sistema              |
| Capacidad forense | Baja (solo indica que se detectó un archivo)               | Alta (permite rastrear el origen, el alcance y el impacto del ataque)                 |
| Ejemplo           | Windows Defender, ClamAV                                   | Wazuh, CrowdStrike, SentinelOne                                                       |

**Conclusión:** El antivirus tradicional es una capa básica de defensa, pero el EDR es esencial para detectar ataques avanzados (APT, ransomware, amenazas sin firmas conocidas). Las organizaciones deben implementar ambos (o una solución que los integre) para una defensa en profundidad efectiva.

---

## Zero Trust en sistemas operativos

El modelo de **Zero Trust** se aplica también a los sistemas operativos, asumiendo que ningún usuario, dispositivo o proceso es confiable por defecto. A nivel de SO, implica:

| Principio Zero Trust    | Implementación en el SO                                                                                                                    |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Verificación continua   | Autenticación multifactor en cada sesión; re-autenticación periódica; verificación de integridad del dispositivo antes de conceder acceso. |
| Mínimo privilegio       | Usuarios y procesos con permisos mínimos; cuentas de servicio con permisos específicos; eliminación de cuentas locales innecesarias.       |
| Microsegmentación       | Políticas de firewall que restringen la comunicación entre procesos y servicios; aislamiento de contenedores y máquinas virtuales.         |
| Monitoreo de integridad | Verificación de integridad de archivos críticos (ej. /etc/passwd, /etc/shadow) y del kernel; detección de cambios no autorizados.          |
| Respuesta automatizada  | Aislamiento automático de endpoints comprometidos; bloqueo de ejecución de procesos sospechosos.                                           |

## Ejemplo de verificación de integridad en Linux con AIDE (Advanced Intrusion Detection Environment):

```
???bash
```

## Instalar AIDE

---

```
apt install aide
```

## Inicializar la base de datos de integridad

---

## Verificar la integridad (comparar con la base de datos)

```
aide --check
???
```

### Hardening en la nube

Con la migración masiva a la nube, el hardening de sistemas operativos debe adaptarse al modelo de responsabilidad compartida. En la nube, el proveedor es responsable de la seguridad de la infraestructura subyacente (hipervisor, red física), pero el cliente es responsable de la seguridad del sistema operativo, las aplicaciones y los datos.

| Nivel de servicio                      | Responsabilidad del proveedor              | Responsabilidad del cliente                               |
|----------------------------------------|--------------------------------------------|-----------------------------------------------------------|
| <b>IaaS</b> (VM en la nube)            | Virtualización, almacenamiento físico, red | SO, aplicaciones, datos, parches del SO, firewall interno |
| <b>PaaS</b> (Plataforma como Servicio) | SO, middleware, runtime                    | Aplicaciones, datos, control de acceso a la plataforma    |
| <b>SaaS</b> (Software como Servicio)   | Toda la pila (SO, aplicación, datos)       | Configuración de usuarios, políticas de acceso            |

### Prácticas recomendadas en la nube:

1. **Usar imágenes base seguras:** Las nubes (AWS, Azure, GCP) ofrecen imágenes base con hardening parcial (ej. Amazon Linux 2 con CIS hardening). Usarlas reduce el esfuerzo inicial.
2. **Aplicar hardening desde el inicio:** No esperar a tener el servidor en producción para aplicar hardening; hacerlo en el pipeline de CI/CD (Infrastructure as Code).
3. **Automatizar parches:** Usar servicios como AWS Systems Manager Patch Manager, Azure Update Management o GCP OS Patch Management para aplicar parches de forma automática.
4. **Monitoreo continuo:** Usar servicios nativos como AWS GuardDuty, Azure Defender o GCP Security Command Center para detectar anomalías en los SO.

### Hardening con presupuesto limitado (para PYMES)

Las pequeñas y medianas empresas a menudo tienen recursos limitados, pero eso no significa que deban descuidar el hardening. Aquí hay un conjunto de controles básicos pero efectivos:

| Control                   | Herramientas open source / bajo costo                 | Impacto                                             |
|---------------------------|-------------------------------------------------------|-----------------------------------------------------|
| <b>Firewall de red</b>    | pfSense, OPNsense                                     | Bloquea accesos no autorizados a la red.            |
| <b>Gestión de parches</b> | WSUS (para Windows), unattended-upgrades (para Linux) | Mantiene los sistemas actualizados.                 |
| <b>Hardening guiado</b>   | Lynis (Linux)                                         | Escanea y recomienda configuraciones seguras.       |
| <b>Monitoreo</b>          | Wazuh, ELK Stack                                      | Detecta actividad anómala en tiempo real.           |
| <b>Antivirus / EDR</b>    | ClamAV, Wazuh (con módulo EDR)                        | Detecta y bloquea malware.                          |
| <b>Cifrado de discos</b>  | BitLocker (Windows), LUKS (Linux)                     | Protege los datos en caso de robo físico del disco. |

### Ejemplo de implementación en una PYME nicaragüense:

Una distribuidora de alimentos en Managua implementó el siguiente conjunto de controles con un presupuesto de C\$ 30,000:

- Lynis para escaneo de hardening en 3 servidores (gratuito, costo de implementación C\$ 5,000).
- Wazuh para monitoreo centralizado en los 3 servidores y 10 estaciones de trabajo (gratuito, costo de implementación C\$ 10,000).
- unattended-upgrades en los servidores Linux para parches automáticos (gratuito).
- Capacitación de 1 día para 2 administradores (C\$ 6,000).
- Backups automáticos a un NAS local (C\$ 9,000 en hardware).

En 12 meses, el sistema detectó y bloqueó 2 intentos de ransomware, evitando pérdidas estimadas en C\$ 150,000.

#### Comparativa de costos (open source vs. comercial):

| Herramienta                 | Open source        | Comercial                     | Diferencia de costo       |
|-----------------------------|--------------------|-------------------------------|---------------------------|
| SIEM                        | Wazuh (gratuito)   | Splunk (C\$ 200,000/año)      | Ahorro de C\$ 200,000/año |
| EDR                         | Wazuh (gratuito)   | CrowdStrike (C\$ 100,000/año) | Ahorro de C\$ 100,000/año |
| Gestión de parches          | Ansible (gratuito) | SCCM (C\$ 50,000/año)         | Ahorro de C\$ 50,000/año  |
| Escaneo de vulnerabilidades | OpenVAS (gratuito) | Nessus (C\$ 30,000/año)       | Ahorro de C\$ 30,000/año  |
| Total                       | C\$ 0              | C\$ 380,000/año               | Ahorro de C\$ 380,000/año |

#### Sistemas operativos legacy: desafíos y estrategias

Muchas organizaciones aún operan con sistemas operativos obsoletos (ej. Windows 7, Windows Server 2012, CentOS 6) que ya no reciben parches de seguridad. Estos sistemas presentan desafíos únicos de seguridad:

##### Desafíos comunes:

- El fabricante ya no proporciona parches de seguridad.
- No se pueden instalar herramientas de seguridad modernas (ej. EDR, antivirus actualizado).
- No soportan autenticación moderna (ej. MFA, SSO).
- Pueden tener vulnerabilidades conocidas que nunca serán parcheadas.

##### Estrategias de mitigación:

1. **Segmentación de red:** Aislar el SO legacy en una VLAN separada con políticas de firewall restrictivas.
2. **Proxy inverso / WAF:** Colocar un WAF delante de las aplicaciones que se ejecutan en el SO legacy.
3. **Monitoreo intensivo:** Aumentar el logging y monitoreo del sistema para detectar actividad anómala.
4. **Compensación con controles de aplicación:** Si el SO no soporta MFA, usar un proxy de autenticación que sí la soporte.
5. **Plan de migración:** Definir un plan a mediano plazo para migrar a un SO moderno (ej. migrar de Windows Server 2012 a Ubuntu 24.04 LTS).

##### Ejemplo de mitigación con segmentación de red:

```
graph LR
  A[SO Legacy] --> B[VLAN Aislada]
  B --> C[Firewall con reglas restrictivas]
  C --> D[Red Interna]
  B --> E[Proxy de autenticación con MFA]
  E --> D
```

#### Hardening en dispositivos móviles (BYOD)

Con la creciente adopción de políticas **BYOD (Bring Your Own Device)**, los sistemas operativos móviles (Android, iOS) también requieren hardening. Aunque el control es limitado, se pueden implementar políticas de seguridad:

| Control                    | Descripción                                                                                      | Implementación                                   |
|----------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>Bloqueo de pantalla</b> | Exigir PIN, patrón o biometría para desbloquear el dispositivo.                                  | Política de MDM (Mobile Device Management)       |
| <b>Cifrado</b>             | Habilitar el cifrado del dispositivo (por defecto en iOS y Android modernos).                    | Configuración del dispositivo                    |
| <b>Actualizaciones</b>     | Exigir que los dispositivos tengan las últimas actualizaciones de seguridad.                     | Política de MDM                                  |
| <b>Aplicaciones</b>        | Restringir la instalación de aplicaciones solo desde tiendas oficiales (Google Play, App Store). | Política de MDM                                  |
| <b>VPN</b>                 | Exigir el uso de VPN para acceder a recursos corporativos.                                       | Política de MDM                                  |
| <b>Borrado remoto</b>      | Permitir el borrado remoto del dispositivo en caso de pérdida o robo.                            | MDM (ej. Microsoft Intune, VMware Workspace ONE) |

### Sistemas operativos y cumplimiento normativo

Los sistemas operativos deben configurarse para cumplir con las regulaciones aplicables. En Nicaragua, la **Ley 787 de Protección de Datos Personales** establece requisitos específicos que el hardening de SO ayuda a cumplir:

| Requisito de la Ley 787             | Implicación para el sistema operativo                                                              | Cómo el hardening ayuda a cumplirlo                                          |
|-------------------------------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Seguridad de los datos</b>       | El SO debe implementar cifrado de discos (BitLocker, LUKS) para proteger los datos en reposo.      | Habilitar cifrado de discos y políticas de acceso.                           |
| <b>Control de acceso</b>            | El SO debe implementar control de acceso basado en roles y mínimo privilegio.                      | Configurar RBAC, cuentas de servicio con privilegios mínimos.                |
| <b>Logs de acceso</b>               | El SO debe registrar todos los accesos a datos sensibles y mantener logs durante al menos 6 meses. | Habilitar auditd, configurar rotación de logs y almacenamiento centralizado. |
| <b>Actualizaciones de seguridad</b> | El SO debe mantenerse actualizado con parches de seguridad.                                        | Configurar unattended-upgrades, WSUS, automatizar parches.                   |
| <b>Políticas de contraseñas</b>     | El SO debe configurarse con políticas de contraseñas robustas (longitud, complejidad, expiración). | Configurar /etc/login.defs, políticas de contraseñas en Windows.             |

### Ejemplo de cumplimiento en una PYME nicaragüense:

"Distribuidora del Norte, S.A." implementó las siguientes medidas en sus servidores para cumplir con la Ley 787:

- Cifrado de discos con LUKS en todos los servidores.
- Políticas de contraseñas configuradas en `/etc/login.defs` (longitud  $\geq 12$ , expiración cada 60 días).
- auditd configurado para registrar todos los accesos a archivos de clientes.
- Parches automáticos con unattended-upgrades.
- Wazuh para monitoreo centralizado y alertas en tiempo real.

### Casos reales documentados

#### Caso 1: Hardening evitó un ransomware (Nicaragua, 2023 – adaptado de un incidente en una PYME de Managua)

Una pequeña empresa de contabilidad tenía un servidor Windows Server 2012 sin parches desde 2020. Un día, un empleado abrió un archivo adjunto de correo que instaló ransomware. El malware intentó cifrar los archivos compartidos, pero como el servidor tenía el firewall de Windows configurado para bloquear todo el tráfico entrante excepto los puertos esenciales, y los

usuarios no tenían permisos de escritura en las carpetas compartidas (solo el administrador), el ransomware solo pudo cifrar los archivos locales del empleado. La empresa perdió solo un día de trabajo de un empleado, en lugar de todos sus datos financieros. **Lección:** el hardening y el principio de privilegio mínimo redujeron drásticamente el impacto.

**Caso 2: Falta de parches provocó una brecha (Costa Rica, 2022 – adaptado)**

Un hospital público en Costa Rica tenía un servidor web con una vulnerabilidad crítica en Apache (CVE-2021-41773) sin parchear. Un atacante explotó la vulnerabilidad para acceder al sistema y luego escaló privilegios hasta obtener acceso a la base de datos de pacientes. La investigación reveló que el equipo de TI sabía de la vulnerabilidad desde hacía 3 meses, pero no la parchearon porque "no tenían tiempo para probarla en el entorno de pruebas". **Lección:** la gestión de parches no es opcional; los parches críticos deben aplicarse en un plazo máximo de 48 horas.

**Caso 3: Segmentación de red y defensa en profundidad (Nicaragua, 2024 – adaptado de una implementación en un banco local)**

Un banco nicaragüense implementó una arquitectura de tres capas con segmentación VLAN y firewall interno después de una auditoría. Seis meses después, un atacante logró comprometer el servidor web mediante una vulnerabilidad en un plugin de WordPress. Sin embargo, al intentar conectarse a la base de datos desde el servidor web, el firewall interno lo bloqueó porque la IP del servidor web no estaba autorizada para acceder al puerto 3306 de la BD. El atacante solo pudo modificar archivos estáticos del sitio web (imágenes y HTML), sin acceder a datos financieros. **Lección:** la segmentación de red y la defensa en profundidad contienen el daño incluso cuando una capa es comprometida.

**Caso 4: Ataque de ransomware a una alcaldía nicaragüense (2023 – adaptado)**

Una alcaldía en Nicaragua sufrió un ataque de ransomware que paralizó sus operaciones durante 10 días. La investigación mostró que el servidor (Windows Server 2016) tenía el puerto RDP (3389) abierto a Internet, sin MFA, y con una contraseña débil ("admin123"). El atacante ingresó por fuerza bruta, instaló el ransomware y cifró tanto los archivos del sistema como los backups que estaban en el mismo servidor. La alcaldía perdió más de C\$ 500,000 en ventas no realizadas (pagos de impuestos, licencias) y costos de recuperación. **Lección:** una configuración insegura (RDP expuesto, contraseña débil, sin MFA, sin segmentación) es una invitación al desastre. El hardening básico (cerrar puertos innecesarios, usar contraseñas robustas) habría prevenido el ataque.

**Caso integrador: Hardening de un servidor web para una PYME nicaragüense**

**Escenario:**

"Distribuidora del Norte, S.A." es una PYME en Estelí, Nicaragua, que distribuye productos agrícolas a 200 clientes. Actualmente tiene un solo servidor Windows Server 2019 que funciona como web, aplicación y base de datos. No hay firewall, todos los empleados usan el mismo usuario (admin/1234), y el servidor está conectado directamente a Internet. La empresa planea migrar a una arquitectura de tres capas con un servidor web Ubuntu 24.04 LTS.

**Requisitos de hardening:**

- 1. Servidor web Ubuntu 24.04 LTS con Apache.
- 2. Solo puertos 80 (HTTP) y 443 (HTTPS) accesibles desde Internet.
- 3. SSH solo desde la red interna y con autenticación por clave pública.
- 4. Cumplir con los CIS Benchmarks nivel 1.
- 5. Monitoreo continuo con Wazuh.
- 6. Parches automáticos para seguridad.
- 7. Presupuesto máximo para el hardening: C\$ 20,000 (sin contar el servidor).

**Plan de hardening:**

| Área | Medida de hardening                                                     | Verificación                                           | Prioridad |
|------|-------------------------------------------------------------------------|--------------------------------------------------------|-----------|
| SO   | Deshabilitar root login vía SSH ( <code>PermitRootLogin no</code> )     | <code>grep PermitRootLogin /etc/ssh/sshd_config</code> | Alta      |
| SO   | Cambiar puerto SSH a 2222 (para reducir ruido de ataques automatizados) | Verificar conectividad por puerto 2222                 | Media     |

| Área       | Medida de hardening                                                                                | Verificación                                           | Prioridad |
|------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------|-----------|
| SO         | Instalar solo paquetes esenciales (ubuntu-server-minimal)                                          | <code>dpkg --get-selections</code> y revisar servicios | Alta      |
| SO         | Habilitar firewall UFW: permitir solo 80, 443, 2222                                                | <code>ufw status</code>                                | Alta      |
| SO         | Configurar auditd para monitorear <code>/etc/apache2/</code> , <code>/var/www/</code>              | <code>auditctl -l</code>                               | Alta      |
| SO         | Instalar y configurar Fail2ban para SSH y Apache                                                   | <code>fail2ban-client status</code>                    | Alta      |
| Apache     | Ocultar versión del servidor ( <code>ServerTokens Prod</code> , <code>ServerSignature Off</code> ) | <code>curl -I https://dominio.com</code>               | Media     |
| Apache     | Deshabilitar listado de directorios ( <code>Options -Indexes</code> )                              | Navegar a <code>/directorio-sin-index/</code>          | Alta      |
| Apache     | Limitar tamaño de peticiones ( <code>LimitRequestBody 10485760</code> )                            | Probar con petición > 10 MB                            | Media     |
| Apache     | Usar HTTPS con TLS 1.3 solamente                                                                   | <code>testssl.sh dominio.com:443</code>                | Alta      |
| Aplicación | Ejecutar Apache con usuario www-data (no root)                                                     | <code>ps aux   grep apache</code>                      | Crítica   |
| Monitoreo  | Enviar logs de Apache a SIEM centralizado                                                          | Verificar en Kibana/Grafana                            | Alta      |
| Parches    | Configurar unattended-upgrades para parches de seguridad automáticos                               | <code>dpkg -l unattended-upgrades</code>               | Alta      |

#### Cronograma de implementación (2 semanas):

| Semana | Actividad                                                                                         | Hito                                     |
|--------|---------------------------------------------------------------------------------------------------|------------------------------------------|
| 1      | Instalación de Ubuntu 24.04 LTS y configuración base (particiones, usuario)                       | SO instalado                             |
| 1      | Configuración de firewall (UFW) y SSH (clave pública, cambio de puerto)                           | Acceso seguro configurado                |
| 1      | Instalación y configuración de auditd y Fail2ban                                                  | Monitoreo básico configurado             |
| 2      | Instalación y hardening de Apache (ocultar versión, deshabilitar listado de directorios, TLS 1.3) | Apache hardening completado              |
| 2      | Configuración de unattended-upgrades y Wazuh                                                      | Actualizaciones y monitoreo configurados |
| 2      | Pruebas de validación y documentación                                                             | Sistema en producción                    |

#### Presupuesto estimado:

| Concepto                                                    | Costo (C\$)     |
|-------------------------------------------------------------|-----------------|
| Horas de consultoría (20 horas × C\$ 500/h)                 | 10,000          |
| Capacitación al personal (2 personas × 8 horas × C\$ 300/h) | 4,800           |
| Configuración de Wazuh (servidor de monitoreo)              | 0 (open source) |
| Configuración de Fail2ban y auditd                          | 0 (open source) |
| <b>Subtotal</b>                                             | <b>14,800</b>   |
| Reserva para imprevistos (10%)                              | 1,480           |
| <b>Total</b>                                                | <b>16,280</b>   |

#### Lecciones aprendidas del caso:

1. El hardening no es un evento único; debe mantenerse con el tiempo (parches, revisiones periódicas).

2. La automatización (unattended-upgrades, scripts de auditoría) reduce el esfuerzo de mantenimiento.
  3. El monitoreo continuo (Wazuh) es esencial para detectar desviaciones de la configuración segura.
  4. La inversión en hardening (C\$ 16,280) es significativamente menor que el costo potencial de una brecha (multas, pérdida de datos, recuperación).
- 

## Comprobación de aprendizaje

**Ejercicio 3.4.1:** Verdadero o falso. Justifique brevemente.

- a) El hardening se aplica una sola vez al instalar el sistema operativo.
- b) El principio de privilegio mínimo implica que los procesos se ejecuten con la menor cantidad de permisos posible.
- c) CIS Benchmarks son estándares de hardening desarrollados por Microsoft.
- d) Una vulnerabilidad con CVSS 9.5 debe parchearse en un plazo máximo de 48 horas.
- e) Fail2ban es una herramienta que previene ataques de fuerza bruta bloqueando IPs temporalmente.

*Respuestas esperadas:* a) Falso, el hardening es un proceso continuo que debe mantenerse (parches, auditorías periódicas). b) Verdadero. c) Falso, CIS Benchmarks son del Center for Internet Security, una organización independiente. d) Verdadero, CVSS  $\geq$  9.0 es crítico. e) Verdadero.

**Ejercicio 3.4.2:** Un administrador ejecuta `chmod 777 /etc/shadow` para que una aplicación pueda leer el archivo de contraseñas. ¿Qué principio de seguridad se viola? ¿Qué recomendaría?

*Respuesta esperada:* Se viola el principio de privilegio mínimo y el de configuración segura. `/etc/shadow` debe tener permisos 640 o 600 y solo ser accesible por root. La aplicación debe rediseñarse para no necesitar acceso a `/etc/shadow`; en su lugar, usar PAM o LDAP para autenticación.

### Ejercicio 3.4.3 (Caso práctico):

Una empresa de logística tiene 5 servidores Linux sin parches desde hace 6 meses. Recientemente, un atacante explotó una vulnerabilidad crítica (CVSS 9.8) en el kernel y comprometió todos los servidores. La empresa no tiene un inventario de sus sistemas, no hay registros de quién aplicó los últimos parches, y no hay un plan de respuesta a incidentes.

- a) Identifique al menos 4 fallos en la gestión de parches y seguridad de la empresa.
- b) Proponga un plan de remediación con plazos (corto, mediano y largo plazo).
- c) ¿Cómo justificaría la inversión en un sistema de gestión de parches ante el gerente general?

*Respuesta esperada (abierta, pero debe incluir):*

- Fallos: 1) Sin inventario de sistemas; 2) Sin proceso de gestión de parches; 3) Sin monitoreo de vulnerabilidades; 4) Sin plan de respuesta a incidentes.
- Plan: Corto plazo (48h): parchar todos los servidores, cambiar contraseñas, implementar firewall; Mediano plazo (2 semanas): implementar inventario y escáner de vulnerabilidades (ej. OpenVAS); Largo plazo (3 meses): implementar Wazuh, automatizar parches con Ansible.
- Justificación: Costo de la brecha (pérdida de datos, tiempo de inactividad, multas) vs. costo del sistema de gestión de parches (herramientas open source, horas de consultoría).

### Ejercicio 3.4.4 (Reflexión):

¿Por qué cree que muchas organizaciones descuidan el hardening de sistemas operativos a pesar de ser la base de la seguridad? Mencione al menos tres razones y proponga cómo abordarlas.

*Respuesta esperada (abierta, pero debe incluir):*

- Razones: 1) Complejidad percibida (demasiados controles); 2) Miedo a romper aplicaciones; 3) Falta de recursos (tiempo, personal).
  - Abordajes: 1) Implementar hardening por fases (empezar con controles de alto impacto); 2) Probar en entornos de staging antes de producción; 3) Usar herramientas de automatización (Ansible) y open source (Lynis, Wazuh).
-

# Autoevaluación

Lea cada pregunta, responda mentalmente y luego consulte las respuestas esperadas al final de cada ítem. Las respuestas no se entregan; son para su propio aprendizaje.

---

## 1. Verdadero o falso

- a)** La arquitectura de seguridad SABSA es un framework específico para diseñar arquitecturas de seguridad, basado en Zachman.
  - b)** En el modelo RBAC, los permisos se asignan directamente a cada usuario, sin intermediarios.
  - c)** La inyección SQL se mitiga eficazmente usando consultas parametrizadas (prepared statements).
  - d)** El TDE (Transparent Data Encryption) protege los datos contra accesos no autorizados a través de consultas SQL.
  - e)** El hardening de un sistema operativo se realiza exclusivamente mediante la instalación de parches de seguridad.
  - f)** CIS Benchmarks proporcionan guías de hardening para múltiples sistemas operativos y aplicaciones.
  - g)** En OWASP Top 10, la inyección SQL se clasifica dentro de A03: Injection.
  - h)** El RPO de 1 hora significa que el servicio debe recuperarse en menos de 1 hora tras un desastre.
  - i)** Un backup completo diario con RTO de 8 horas no es adecuado si el negocio requiere RTO de 2 horas.
  - j)** La defensa en profundidad consiste en confiar en un único control de seguridad muy robusto.
- 

## 2. Selección múltiple (una o varias opciones correctas)

- a)** ¿Cuáles de los siguientes son principios de diseño seguro en arquitectura TI?
    - 1. Defensa en profundidad
    - 2. Privilegio máximo
    - 3. Cifrado generalizado
    - 4. Registro y auditoría
  - b)** ¿Qué modelos de control de acceso se basan en roles y atributos respectivamente?
    - 1. DAC y MAC
    - 2. RBAC y ABAC
    - 3. ABAC y DAC
    - 4. RBAC y DAC
  - c)** ¿Cuáles de las siguientes son amenazas comunes a la seguridad de bases de datos?
    - 1. SQL Injection
    - 2. Buffer overflow en la interfaz de usuario
    - 3. Acceso no autorizado
    - 4. Pérdida de datos
  - d)** ¿Qué niveles de cifrado de base de datos protegen los datos incluso del administrador de la BD?
    - 1. TDE
    - 2. Cifrado a nivel de columna
    - 3. Cifrado a nivel de aplicación
    - 4. Cifrado a nivel de red (TLS)
  - e)** ¿Cuáles de los siguientes son componentes del hardening de sistemas operativos?
    - 1. Eliminar servicios innecesarios
    - 2. Aumentar los privilegios de los usuarios
-

3. Configurar políticas de contraseñas
4. Deshabilitar el logging para ahorrar espacio

**f)** ¿Qué herramienta se utiliza para la detección de vulnerabilidades en sistemas?

1. Nessus
2. Wireshark
3. Nmap
4. OpenVAS

**g)** ¿Cuál es el SLA recomendado para parchear una vulnerabilidad con CVSS 9.5?

1. 24-48 horas
2. 1 mes
3. 3 meses
4. No es necesario parchear

**h)** ¿Qué significa RTO (Recovery Time Objective)?

1. Cantidad máxima de datos que se puede perder
2. Tiempo máximo para recuperar el servicio
3. Frecuencia de los backups
4. Porcentaje de tiempo que el sistema debe estar disponible

**i)** ¿Cuál de los siguientes NO es un framework de arquitectura empresarial?

1. TOGAF
2. SABSA
3. OWASP
4. Zachman

**j)** ¿Qué modelo de seguridad asume que ningún usuario, dispositivo o red es confiable por defecto?

1. Defensa en profundidad
2. Zero Trust
3. Seguridad por defecto
4. Privilegio mínimo

---

### 3. Complete la frase

- a)** La \_\_\_\_\_ en profundidad consiste en superponer múltiples capas de seguridad para que si una falla, la siguiente la contenga.
- b)** En el modelo \_\_\_\_\_, los permisos se asignan a roles y los usuarios se asignan a esos roles.
- c)** La amenaza \_\_\_\_\_ consiste en insertar código SQL malicioso a través de entradas de usuario no validadas.
- d)** El \_\_\_\_\_ de datos en reposo protege los archivos de base de datos en caso de robo del medio de almacenamiento.
- e)** El \_\_\_\_\_ es el proceso de asegurar un sistema operativo reduciendo su superficie de ataque.
- f)** \_\_\_\_\_ es un framework de arquitectura de seguridad basado en Zachman y orientado a la gestión de riesgos.
- g)** El \_\_\_\_\_ mide la cantidad máxima de datos que una organización está dispuesta a perder, expresada en tiempo.
- h)** Las guías más utilizadas para el hardening de sistemas operativos son los \_\_\_\_\_ del Center for Internet Security.
- i)** Un sistema de detección de intrusiones a nivel de host se conoce como \_\_\_\_\_ (siglas en inglés).
- j)** En OWASP Top 10, el riesgo A01 corresponde a \_\_\_\_\_ roto.
-

#### 4. Relacionar columnas

Relacione cada concepto de la columna A con su descripción en la columna B.

| Columna A         | Columna B                                                             |
|-------------------|-----------------------------------------------------------------------|
| 1. TOGAF          | A. Framework de gobierno de arquitectura empresarial                  |
| 2. SABSA          | B. Control de acceso basado en atributos del usuario y contexto       |
| 3. OWASP Top 10   | C. Lista de los riesgos de seguridad más críticos en aplicaciones web |
| 4. RBAC           | D. Control de acceso basado en roles                                  |
| 5. ABAC           | E. Framework de arquitectura de seguridad                             |
| 6. TDE            | F. Cifrado transparente de base de datos a nivel de disco             |
| 7. CIS Benchmarks | G. Guías de hardening para sistemas y aplicaciones                    |
| 8. CVSS           | H. Sistema de puntuación de vulnerabilidades (0-10)                   |
| 9. SIEM           | I. Sistema de gestión de eventos e información de seguridad           |
| 10. EDR           | J. Detección y respuesta en endpoints                                 |

#### 5. Caso práctico integrador

Una empresa de servicios financieros está rediseñando su infraestructura tecnológica. Actualmente tiene:

- Un servidor web Apache en Linux que también funciona como base de datos MySQL (todo en el mismo servidor).
- Las aplicaciones web se comunican por HTTP sin cifrar.
- Las contraseñas de los usuarios se almacenan con MD5.
- No hay segmentación de red; todos los servidores están en la misma VLAN.
- No existe un sistema de monitoreo centralizado; los logs se almacenan localmente y se revisan manualmente cada 3 meses.
- Los desarrolladores tienen acceso root a los servidores de producción.
- No hay política de backups; el último respaldo fue hace 6 meses y no se ha probado.

##### Preguntas:

- Identifique al menos 5 vulnerabilidades de seguridad en la arquitectura actual. Para cada una, indique qué principio de diseño seguro se viola y a qué capa de la arquitectura (Arquitectura TI, Sistemas de Información, Bases de Datos o Sistemas Operativos) pertenece.
- Proponga un plan de remediación priorizado (corto, mediano y largo plazo) para cada vulnerabilidad identificada, incluyendo controles específicos de cada capa.
- ¿Qué recomendaciones de hardening específicas daría para el servidor web Linux (Apache + MySQL)?
- Diseñe una arquitectura de seguridad de tres capas (web, aplicación, datos) con segmentación de red y controles en cada capa. Incluya un diagrama en texto o Mermaid.
- ¿Cómo justificaría la inversión en esta transformación ante el directorio de la empresa, usando lenguaje de negocio (riesgo financiero, reputacional, legal)?

#### 6. Pregunta de desarrollo breve

Explique la diferencia entre **cifrado en reposo (TDE)**, **cifrado a nivel de columna** y **cifrado a nivel de aplicación** en bases de datos. Mencione al menos una ventaja y una desventaja de cada uno, y un escenario donde cada uno sea la opción más adecuada.

## 7. Pregunta de desarrollo breve (STRIDE)

Aplice el marco STRIDE a un sistema de banca en línea. Para cada categoría, identifique una amenaza concreta y proponga un control arquitectónico específico que la mitigue.

| Categoría STRIDE | Amenaza | Control arquitectónico |
|------------------|---------|------------------------|
| Suplantación (S) |         |                        |
| Manipulación (T) |         |                        |
| Repudio (R)      |         |                        |
| Revelación (I)   |         |                        |
| Denegación (D)   |         |                        |
| Elevación (E)    |         |                        |

## 8. Reflexión

Imagine que usted es el responsable de seguridad de una universidad que almacena datos personales de 30,000 estudiantes (nombres, cédulas, calificaciones, datos de salud). Recientemente, un estudiante de ingeniería informática descubrió que modificando el parámetro `id_estudiante` en la URL de consulta de calificaciones puede ver las notas de cualquier compañero. La universidad no tiene políticas de seguridad definidas, no hay cifrado en la base de datos y los servidores tienen parches de hace 2 años.

Responda:

- ¿Qué riesgos del OWASP Top 10 están presentes en el escenario descrito?
- ¿Qué medidas tomaría en las primeras 48 horas para contener la brecha?
- ¿Qué plan de mediano plazo (3 meses) propondría para evitar futuras brechas? Incluya medidas de hardening, control de acceso, cifrado y monitoreo.
- ¿Cómo justificaría la inversión en seguridad ante el rectorado, usando el lenguaje de negocio (riesgo financiero, reputacional, legal)?

## 9. Caso práctico (DevSecOps)

Una empresa de desarrollo de software quiere implementar DevSecOps en su pipeline de CI/CD. Actualmente, el equipo de desarrollo entrega código a producción sin pruebas de seguridad. Los requisitos son:

- Integrar pruebas de seguridad automatizadas en el pipeline.
- Bloquear despliegues si se detectan vulnerabilidades críticas.
- Mantener la velocidad de entrega (al menos 10 despliegues por semana).

- Proponga un pipeline DevSecOps con las herramientas adecuadas para cada fase (SAST, DAST, SCA, etc.).
- ¿Qué métricas utilizaría para medir la efectividad del pipeline?
- ¿Cómo manejaría el trade-off entre velocidad de entrega y seguridad?

## 10. Pregunta de desarrollo (Zero Trust)

Explique cómo se aplica el principio de **Zero Trust** en cada una de las siguientes capas de la arquitectura TI:

- Arquitectura de red.
- Sistemas de información (aplicaciones).

- c) Bases de datos.
  - d) Sistemas operativos.
- 

## Respuestas esperadas

### 1. Verdadero o falso

- a) Verdadero. SABSA es un framework de arquitectura de seguridad basado en Zachman.
  - b) Falso. En RBAC los permisos se asignan a roles, no directamente a usuarios.
  - c) Verdadero. Las consultas parametrizadas separan la estructura SQL de los datos, eliminando la inyección.
  - d) Falso. TDE protege los datos en el disco, pero no contra consultas SQL autorizadas desde la aplicación. Para eso se necesita cifrado a nivel de columna o aplicación.
  - e) Falso. El hardening incluye eliminar servicios, configurar seguridad, gestionar parches, implementar monitoreo, etc., no solo parches.
  - f) Verdadero. CIS Benchmarks cubren SO, bases de datos, navegadores, cloud, etc.
  - g) Verdadero. A03: Injection incluye SQL, NoSQL, LDAP, OS command injection.
  - h) Falso. RPO (Recovery Point Objective) mide la pérdida máxima de datos en tiempo; RTO mide el tiempo de recuperación.
  - i) Verdadero. Si el RTO requerido es 2 horas, un backup que tarda 8 horas en restaurarse no es adecuado.
  - j) Falso. La defensa en profundidad consiste en usar múltiples capas de seguridad, no una sola.
- 

### 2. Selección múltiple

- a) 1, 3 y 4. El privilegio máximo no es un principio de seguridad; es lo opuesto al privilegio mínimo.
  - b) 2. RBAC se basa en roles; ABAC se basa en atributos.
  - c) 1, 3 y 4. Buffer overflow en UI no es una amenaza específica de bases de datos.
  - d) 3. El cifrado a nivel de aplicación impide que la BD vea los datos en texto plano; el cifrado a nivel de columna también protege de administradores de BD, pero la BD aún maneja las claves.
  - e) 1 y 3. Aumentar privilegios y deshabilitar logging son contrarios al hardening.
  - f) 1 y 4. Wireshark es un analizador de protocolos; Nmap es un escáner de puertos.
  - g) 1. CVSS  $\geq 9.0$  es crítico; el SLA recomendado es 24-48 horas.
  - h) 2. El RTO (Recovery Time Objective) es el tiempo máximo para recuperar el servicio.
  - i) 3. OWASP es una organización de seguridad de aplicaciones web, no un framework de arquitectura empresarial.
  - j) 2. Zero Trust asume que ningún usuario, dispositivo o red es confiable por defecto.
- 

### 3. Complete la frase

- a) defensa
  - b) RBAC (Role-Based Access Control)
  - c) SQL Injection
  - d) cifrado
-

- e) hardening
- f) SABSA
- g) RPO (Recovery Point Objective)
- h) CIS Benchmarks
- i) HIDS (Host-based Intrusion Detection System)
- j) control de acceso

#### 4. Relacionar columnas

1-A, 2-E, 3-C, 4-D, 5-B, 6-F, 7-G, 8-H, 9-I, 10-J

#### 5. Caso práctico integrador (respuesta esperada)

##### a) Vulnerabilidades identificadas:

| # | Vulnerabilidad                                          | Principio violado                                              | Capa                    |
|---|---------------------------------------------------------|----------------------------------------------------------------|-------------------------|
| 1 | Servidor web y BD en el mismo servidor                  | Defensa en profundidad (falta segmentación); privilegio mínimo | Arquitectura TI         |
| 2 | Comunicaciones HTTP sin cifrar                          | Cifrado generalizado                                           | Sistemas de Información |
| 3 | Contraseñas almacenadas con MD5                         | Cifrado generalizado (hash débil)                              | Bases de Datos          |
| 4 | Sin segmentación de red                                 | Defensa en profundidad                                         | Arquitectura TI         |
| 5 | Sin monitoreo centralizado; logs revisados cada 3 meses | Registro y auditoría                                           | Sistemas Operativos     |
| 6 | Desarrolladores con acceso root a producción            | Privilegio mínimo; separación de funciones                     | Sistemas Operativos     |
| 7 | Sin backups probados                                    | Registro y auditoría (no hay resiliencia)                      | Bases de Datos          |

##### b) Plan de remediación:

| Prioridad                        | Vulnerabilidad                     | Acción                                                                                              | Plazo     |
|----------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|-----------|
| <b>Corto plazo (48h)</b>         | Desarrolladores root en producción | Revocar acceso root; crear cuentas con permisos mínimos; usar sudo con registros                    | Inmediato |
| <b>Corto plazo (48h)</b>         | HTTP sin cifrar                    | Instalar certificados SSL/TLS; redirigir todo el tráfico a HTTPS; deshabilitar puerto 80            | 24h       |
| <b>Corto plazo (48h)</b>         | Contraseñas MD5                    | Forzar reseteo de contraseñas; migrar a bcrypt con factor de trabajo $\geq 12$                      | 48h       |
| <b>Mediano plazo (2 semanas)</b> | Servidor web + BD juntos           | Migrar BD a un servidor separado en una VLAN distinta; abrir solo puerto 3306 desde el servidor web | 2 semanas |
| <b>Mediano plazo (2 semanas)</b> | Sin backups                        | Configurar backup diario completo + incremental cada 6 horas; probar restauración                   | 1 semana  |
| <b>Mediano plazo (1 mes)</b>     | Sin segmentación de red            | Diseñar VLANs: DMZ (web), aplicación, datos, administración; implementar firewall interno           | 1 mes     |

| Prioridad                    | Vulnerabilidad | Acción                                                                                                             | Plazo   |
|------------------------------|----------------|--------------------------------------------------------------------------------------------------------------------|---------|
| <b>Largo plazo (3 meses)</b> | Sin monitoreo  | Implementar SIEM (Wazuh o ELK); configurar alertas para eventos críticos; definir equipo de respuesta a incidentes | 3 meses |

#### c) Hardening específico para servidor web (Apache + MySQL):

1. Ejecutar Apache con usuario www-data (no root).
2. Deshabilitar módulos innecesarios: `a2dismod autoindex status info`.
3. Configurar `ServerTokens Prod` y `ServerSignature Off`.
4. TLS 1.3 con certificado válido; redirección forzada a HTTPS.
5. Deshabilitar listado de directorios (`Options -Indexes`).
6. MySQL: cambiar puerto por defecto; deshabilitar acceso remoto a root; usar contraseña fuerte ( $\geq 20$  caracteres).
7. Instalar y configurar Fail2ban para Apache y MySQL.
8. Configurar UFW: permitir solo puertos 80, 443 y SSH (en puerto no estándar).
9. Implementar ModSecurity (WAF a nivel de Apache) con reglas OWASP CRS.
10. Configurar auditd para monitorear cambios en `/etc/apache2/` y `/var/www/`.

#### d) Arquitectura de seguridad de tres capas:

```

???mermaid
graph TB
    subgraph "Internet"
        A[Usuarios]
    end
    subgraph "Perímetro"
        B[Firewall Perimetral  
Solo 80/443]
    end
    subgraph "DMZ - Capa Web"
        C[Servidor Web Apache  
Hardened + WAF]
        D[Fail2ban]
    end
    subgraph "Red Interna"
        E[Firewall Interno  
Solo 3306 desde IP web]
        subgraph "VLAN Aplicación"
            F[Servidor de Aplicaciones  
Node.js / PHP 8.2]
        end
        subgraph "VLAN Datos"
            G[Servidor MySQL 8.0  
TDE habilitado]
        end
        subgraph "VLAN Administración"
            H[SIEM Wazuh  
Monitoreo centralizado]
        end
    end
    subgraph "Capa de Respaldo"
        I[Backups Offsite  
NAS + Nube]
    end

```

```

A -->|HTTPS| B
B --> C
C --> E
E -->|Solo puerto 3000| F
F -->|Puerto 3306| G
G --> I
C -. ->|Logs| H
F -. ->|Logs| H
G -. ->|Logs| H

```

???

#### e) Justificación de inversión (lenguaje de negocio):

"Señores directores, la arquitectura actual presenta múltiples vulnerabilidades críticas que exponen a la empresa a:

- **Riesgo financiero:** Una brecha de datos podría generar multas de hasta C\$ 5,000,000 por incumplimiento de la Ley 787, además de pérdidas por interrupción del negocio.
- **Riesgo reputacional:** La confianza de nuestros clientes es nuestro activo más valioso. Una filtración de datos financieros aparecería en titulares y nos costaría clientes.
- **Riesgo legal:** Demandas de clientes afectados, investigaciones regulatorias y posibles sanciones penales.

La inversión estimada de C\$ 302,450 (según el caso integrador de Arquitectura TI) representa menos del 10% del costo potencial de una sola brecha. Estamos proponiendo una migración gradual que minimiza el impacto operativo mientras construimos una base tecnológica sólida y segura."

#### 6. Pregunta de desarrollo breve

| Nivel de cifrado             | Ventaja                                                                                    | Desventaja                                                                                                                            | Escenario adecuado                                                           |
|------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>TDE (en reposo)</b>       | Transparente para aplicaciones; sin cambios en código.                                     | No protege contra consultas SQL autorizadas.                                                                                          | Protección contra robo físico de discos; cumplimiento de normativas básicas. |
| <b>A nivel de columna</b>    | Protege datos específicos incluso de administradores de BD (si la clave no está en la BD). | Requiere cambios en esquema; puede afectar rendimiento en búsquedas.                                                                  | Datos sensibles como números de tarjeta, cédulas, historias clínicas.        |
| <b>A nivel de aplicación</b> | La BD nunca ve los datos en texto plano; máximo nivel de protección.                       | Mayor complejidad; no se pueden hacer búsquedas SQL sobre datos cifrados (o se requiere búsqueda determinista, que reduce seguridad). | Datos ultra sensibles: claves privadas, datos biométricos, secretos de API.  |

#### 7. Pregunta de desarrollo breve (STRIDE)

| Categoría STRIDE | Amenaza                                                         | Control arquitectónico                                                            |
|------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Suplantación (S) | Un atacante roba las credenciales de un usuario.                | MFA (autenticación multifactor) obligatoria; detección de comportamiento anómalo. |
| Manipulación (T) | Un atacante modifica el monto de una transferencia en tránsito. | Firmas digitales en todas las transacciones; TLS 1.3.                             |

| Categoría STRIDE | Amenaza                                                                                | Control arquitectónico                                                             |
|------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Repudio (R)      | Un usuario niega haber realizado una transferencia.                                    | Logs de auditoría con firma digital y sello de tiempo; registro inmutable.         |
| Revelación (I)   | Un atacante accede a los números de tarjeta de los clientes.                           | Cifrado de datos sensibles en la BD; tokenización de tarjetas.                     |
| Denegación (D)   | Un ataque DDoS satura la plataforma de pagos.                                          | WAF, rate limiting, CDN, auto-scaling.                                             |
| Elevación (E)    | Un usuario estándar explota una vulnerabilidad para obtener permisos de administrador. | Principio de mínimo privilegio; separación de roles; actualizaciones de seguridad. |

## 8. Reflexión (respuesta esperada)

### a) Riesgos OWASP Top 10 presentes:

- A01: Broken Access Control (el parámetro `id_estudiante` permite acceder a datos de otros estudiantes).
- A02: Cryptographic Failures (no hay cifrado en la BD; posiblemente contraseñas débiles).
- A06: Vulnerable and Outdated Components (parches de hace 2 años → múltiples CVE conocidas).
- A07: Identification and Authentication Failures (si no hay MFA, contraseñas débiles).
- A09: Security Logging and Monitoring Failures (si no hay monitoreo, la brecha podría haber pasado desapercibida más tiempo).

### b) Medidas en las primeras 48 horas:

1. **Contener la brecha:** deshabilitar temporalmente el endpoint vulnerable de consulta de calificaciones hasta que se implemente la validación de autorización por sesión.
2. **Investigación forense inicial:** revisar logs (si existen) para determinar qué estudiantes o personal accedieron a datos no autorizados y desde cuándo.
3. **Notificación legal:** informar al encargado de protección de datos (Ley 787) sobre la brecha, si aplica.
4. **Parche temporal:** implementar validación del lado del servidor que verifique que el `id_estudiante` corresponde al usuario autenticado.
5. **Cambio de contraseñas:** forzar cambio de contraseñas de todos los estudiantes y personal (si se almacenaban con hash débil).

### c) Plan de mediano plazo (3 meses):

1. **Hardening de servidores:** aplicar CIS Benchmarks a todos los servidores (Linux y Windows).
2. **Migrar a consultas parametrizadas:** revisar todo el código para eliminar concatenación de SQL.
3. **Implementar RBAC:** definir roles (Estudiante, Profesor, Administrativo, Administrador) y asignar permisos.
4. **Cifrado de BD:** implementar TDE + cifrado a nivel de columna para datos sensibles (cédula, datos de salud).
5. **Implementar MFA:** para todos los accesos administrativos y estudiantes (al menos una segunda opción como código por correo).
6. **Implementar SIEM:** Wazuh o ELK para monitoreo centralizado con alertas en tiempo real.
7. **Gestión de parches:** establecer proceso de parcheo con SLA según CVSS.
8. **Segmentación de red:** separar servidores web, aplicación y BD en VLANs.
9. **Política de backups:** configuración de backups diarios + restauración probada mensualmente.
10. **Capacitación:** programa de concienciación en seguridad para todos los empleados y estudiantes.

### d) Justificación de inversión ante el rectorado:

"Señor rector, la brecha detectada esta semana demuestra que cualquier estudiante con conocimientos básicos de hacking puede acceder a las calificaciones y datos personales de todos sus compañeros. Los riesgos concretos son:

- **Financiero:** multas de hasta C\$ 5,000,000 por incumplimiento de la Ley 787 de Protección de Datos Personales.
- **Reputacional:** la universidad podría aparecer en titulares nacionales como 'Universidad X expone datos de 30,000 estudiantes'. La confianza de los padres de familia y futuros estudiantes se vería gravemente afectada.

- **Legal:** posibles demandas de estudiantes afectados por exposición de sus datos personales y de salud.
- **Operativo:** un ataque dirigido podría dejar fuera de servicio el sistema de calificaciones durante semanas, afectando a 30,000 estudiantes y 500 profesores.

La inversión estimada en seguridad (hardening, SIEM, consultoría, capacitación) es de aproximadamente C\$ 800,000. En comparación, una sola multa por incumplimiento de la Ley 787 puede ser hasta 6 veces mayor. Además, la Universidad Nacional Héroes de San José de las Mulass, comprometida con la calidad educativa y la innovación, debe predicar con el ejemplo en la protección de los datos de su comunidad. ¿Qué mensaje enviamos a nuestros estudiantes de Ingeniería en Sistemas si no protegemos ni nuestros propios sistemas?"

---

## 9. Caso práctico (DevSecOps)

### a) Pipeline DevSecOps propuesto:

```

???mermaid
graph LR
  A[Commit de código] --> B[SAST  
SonarQube]
  B --> C[SCA  
Snyk / OWASP DC]
  C --> D[Pruebas unitarias]
  D --> E[Construcción]
  E --> F[DAST  
OWASP ZAP]
  F --> G{¿Vulnerabilidades críticas?}
  G -->|Sí| H[Bloquear despliegue]
  G -->|No| I[Despliegue a producción]
  I --> J[Monitoreo  
Wazuh / ELK]
  ???

```

### Herramientas por fase:

- **SAST:** SonarQube (analiza código en busca de vulnerabilidades como SQL Injection, XSS).
- **SCA:** Snyk o OWASP Dependency Check (analiza dependencias vulnerables).
- **DAST:** OWASP ZAP (prueba la aplicación en ejecución).
- **Monitoreo:** Wazuh o ELK Stack (detección de amenazas en producción).

### b) Métricas de efectividad:

- **Tiempo de detección de vulnerabilidades:** tiempo desde el commit hasta la detección de una vulnerabilidad.
- **Tasa de falsos positivos:** porcentaje de alertas que no son vulnerabilidades reales.
- **Cobertura de código analizado:** porcentaje de código cubierto por SAST/DAST.
- **Tiempo de remediación:** tiempo promedio para corregir una vulnerabilidad detectada.
- **Número de despliegues bloqueados:** cantidad de despliegues detenidos por vulnerabilidades críticas.

### c) Manejo del trade-off velocidad vs. seguridad:

1. **Priorizar vulnerabilidades críticas:** solo bloquear despliegues para CVSS  $\geq 9.0$ ; para vulnerabilidades menores, permitir el despliegue con un plan de remediación en el backlog.
  2. **Automatización de pruebas:** ejecutar pruebas de seguridad en paralelo con las pruebas unitarias para no añadir tiempo al pipeline.
  3. **Capacitación:** educar a los desarrolladores en seguridad para reducir la cantidad de vulnerabilidades introducidas.
  4. **Análisis incremental:** solo analizar el código cambiado (diff) en lugar de todo el código base en cada commit.
  5. **Canales de liberación:** usar un pipeline de CI/CD con diferentes niveles de rigor (ej. despliegue a staging con pruebas completas, producción con pruebas más ligeras pero con monitoreo intensivo).
-

## 10. Pregunta de desarrollo (Zero Trust)

### a) Arquitectura de red:

- Microsegmentación: dividir la red en segmentos pequeños (VLANs) con políticas de firewall restrictivas.
- No confiar en la red interna: asumir que la red interna es tan insegura como Internet.
- Verificación continua: cada solicitud de acceso a un recurso de red se autentica y autoriza individualmente.
- Políticas de red "deny-all" por defecto: solo permitir tráfico explícitamente autorizado.

### b) Sistemas de información (aplicaciones):

- Autenticación y autorización en cada solicitud: no confiar en sesiones previamente autenticadas.
- Verificación de integridad del dispositivo: comprobar que el dispositivo del usuario cumple con las políticas de seguridad antes de conceder acceso.
- MFA obligatoria: autenticación multifactor para todos los accesos.
- Acceso basado en atributos (ABAC): decisiones de acceso basadas en atributos del usuario, recurso y contexto.

### c) Bases de datos:

- Mínimo privilegio: cada aplicación o usuario tiene acceso solo a las tablas y columnas necesarias.
- Cifrado de datos: cifrar datos en reposo y en tránsito.
- Auditoría continua: registrar todos los accesos a la base de datos y alertar sobre accesos anómalos.
- Microsegmentación de datos: aislar datos sensibles en bases de datos o esquemas separados.

### d) Sistemas operativos:

- Verificación de integridad: monitorear cambios en archivos críticos (ej. /etc/passwd, /etc/shadow) y alertar sobre modificaciones no autorizadas.
- Mínimo privilegio: procesos y servicios ejecutándose con los menores permisos necesarios.
- Actualizaciones continuas: parches automáticos para mantener el SO actualizado.
- Endpoint Detection and Response (EDR): monitoreo continuo de comportamiento anómalo en el SO.

---

## Sugerencia de revisión

Si obtuvo menos de **12 respuestas correctas** (considerando los ítems de opción múltiple y verdadero/falso como un punto cada uno, el caso práctico como tres puntos y las preguntas de desarrollo como dos puntos), revise nuevamente las secciones de:

- **Arquitectura TI:** principios de diseño seguro, frameworks (TOGAF, SABSA), STRIDE, Zero Trust, trade-offs, métricas.
- **Sistemas de Información:** componentes, modelos de control de acceso (RBAC, ABAC, DAC), OWASP Top 10, auditoría, DevSecOps, casos de éxito/fracaso.
- **Bases de Datos:** amenazas (SQL Injection), cifrado (TDE, columna, aplicación), backups, RPO/RTO, auditoría, NoSQL.
- **Sistemas Operativos:** hardening, CIS Benchmarks, gestión de parches, CVSS, monitoreo (SIEM, EDR), Zero Trust en SO, SO legacy.

Recuerde que la autoevaluación no tiene calificación, pero le permite identificar sus fortalezas y áreas de mejora antes de las evaluaciones sumativas.

---

## Bibliografía y Webgrafía

### Libros y textos académicos (selección relevante)

Gómez Vieites, Á. (2020). *Enciclopedia de la seguridad informática* (2ª ed.). RA-MA Editorial.

Mata García, A. E. (2024). *Ciberseguridad* (IFCT133PO). Editorial FC Editorial.

Piattini Velthuis, M. G., & Ruiz González, F. (2022). *Gobierno de las tecnologías y sistemas de información* (2ª ed.). RA-MA Editorial.

Postigo Palacios, A. (2020). *Seguridad informática* (Edición 2020). Editorial Paraninfo.

## Estándares y marcos de referencia (documentos oficiales)

ISO/IEC 27001:2022. (2022). *Tecnología de la información — Técnicas de seguridad — Sistemas de gestión de la seguridad de la información — Requisitos*. International Organization for Standardization.

ISO/IEC 27005:2022. (2022). *Tecnología de la información — Técnicas de seguridad — Gestión de riesgos de seguridad de la información*. International Organization for Standardization.

NIST Special Publication 800-207. (2020). *Zero Trust Architecture*. National Institute of Standards and Technology.

OWASP Foundation. (2021). *OWASP Top 10 – 2021: The ten most critical web application security risks*. OWASP. Recuperado de <https://owasp.org/Top10/>

## Guías de hardening (recursos electrónicos)

Center for Internet Security. (2024). *CIS Benchmarks: Ubuntu Linux 24.04 LTS*. CIS.

Center for Internet Security. (2024). *CIS Benchmarks: Microsoft Windows Server 2022*. CIS.

## Legislación nacional (Nicaragua)

República de Nicaragua. (2012). *Ley N.º 787: Ley de Protección de Datos Personales*. La Gaceta, Diario Oficial.

## Glosario

**ABAC (Attribute-Based Access Control):** Modelo de control de acceso que utiliza atributos del usuario, del recurso y del contexto (ej. hora, ubicación) para tomar decisiones de autorización.

**Arquitectura de seguridad:** Diseño de alto nivel que define los controles, políticas y mecanismos de seguridad en una organización, alineado con la arquitectura TI y los objetivos de negocio.

**Arquitectura Empresarial (EA):** Marco que alinea la TI con los objetivos estratégicos de la organización, definiendo procesos, datos, aplicaciones e infraestructura.

**Arquitectura TI:** Estructura conceptual que define la organización, el comportamiento y la evolución de los sistemas de información de una empresa.

**Auditoría de sistemas:** Proceso de evaluación de los controles, políticas y procedimientos de seguridad de un sistema de información para verificar su cumplimiento y eficacia.

**Backup (copia de seguridad):** Copia de los datos de un sistema que se utiliza para restaurar la información en caso de pérdida o desastre.

**BCP (Business Continuity Plan):** Plan que define cómo la organización mantendrá sus operaciones durante una interrupción.

**Cifrado a nivel de aplicación:** Técnica de cifrado donde la aplicación cifra los datos antes de enviarlos a la base de datos, asegurando que la BD nunca vea los datos en texto plano.

**Cifrado a nivel de columna:** Cifrado de campos específicos (ej. números de tarjeta, cédula) usando funciones del motor de base de datos.

**Cifrado en reposo (TDE):** Cifrado automático de los datos al escribirlos en disco, transparente para las aplicaciones. Protege contra robos físicos del medio de almacenamiento.

**CIS Benchmarks:** Guías de configuración segura (hardening) publicadas por el Center for Internet Security para sistemas operativos, aplicaciones y dispositivos de red.

**Control de acceso:** Mecanismo que regula quién puede acceder a qué recursos y bajo qué condiciones.

**CVSS (Common Vulnerability Scoring System):** Sistema estandarizado de puntuación de vulnerabilidades, con valores de 0 a 10, donde 10 representa el riesgo más crítico.

**DAC (Discretionary Access Control):** Modelo de control de acceso donde el propietario del recurso decide quién puede acceder.

**DAST (Dynamic Application Security Testing):** Prueba de seguridad que analiza la aplicación en ejecución simulando ataques.

**Defensa en profundidad (Defense in Depth):** Principio de seguridad que consiste en superponer múltiples capas de controles para que si una capa falla, la siguiente la contenga.

**DevSecOps:** Integración de la seguridad en el ciclo de vida del desarrollo (Development, Security, Operations).

**DMZ (Zona Desmilitarizada):** Subred de red que actúa como zona de amortiguamiento entre la red interna (confiable) y la red externa (no confiable, como Internet).

**DRP (Disaster Recovery Plan):** Plan específico para recuperar los sistemas de información después de un desastre.

**EDR (Endpoint Detection and Response):** Solución de seguridad que monitorea los endpoints en busca de comportamientos anómalos y permite responder a incidentes.

**Hardening:** Proceso de asegurar un sistema operativo reduciendo su superficie de ataque (eliminación de servicios innecesarios, configuración segura, gestión de parches, etc.).

**HIDS (Host-based Intrusion Detection System):** Sistema de detección de intrusiones que monitorea la actividad en un host específico (archivos, procesos, conexiones).

**Impacto:** Consecuencia negativa (financiera, reputacional, legal) de la materialización de una amenaza.

**Microservicio:** Arquitectura donde una aplicación se divide en servicios pequeños, independientes y desplegables por separado.

**MTBF (Mean Time Between Failures):** Tiempo promedio entre fallos de un sistema.

**MTTD (Mean Time to Detect):** Tiempo promedio para detectar un incidente de seguridad.

**MTTR (Mean Time to Recover/Respond):** Tiempo promedio para recuperar un servicio después de un fallo, o para contener y resolver un incidente.

**NIDS (Network-based Intrusion Detection System):** Sistema que monitorea el tráfico de red en busca de patrones de ataque conocidos o comportamiento anómalo.

**NoSQL:** Tipo de base de datos no relacional (ej. MongoDB, Cassandra) que presenta desafíos de seguridad específicos (inyección NoSQL, autenticación por defecto, etc.).

**OWASP Top 10:** Lista actualizada periódicamente por la Open Web Application Security Project que identifica los diez riesgos de seguridad más críticos en aplicaciones web.

**Parche de seguridad:** Actualización de software diseñada para corregir una vulnerabilidad de seguridad identificada.

**Privilegio mínimo (Least Privilege):** Principio de seguridad que establece que cada usuario, proceso o sistema debe tener solo los permisos mínimos necesarios para realizar su función.

**RBAC (Role-Based Access Control):** Modelo de control de acceso donde los permisos se asignan a roles, y los usuarios se asignan a esos roles.

**Registro y auditoría (Logging and Auditing):** Principio que establece que toda acción relevante en el sistema debe quedar registrada con suficiente detalle para permitir auditorías forenses.

**Riesgo:** Combinación de la probabilidad de que ocurra una amenaza y el impacto de la misma. Se calcula como:  $\text{Riesgo} = \text{Amenaza} \times \text{Vulnerabilidad} \times \text{Impacto}$ .

**RPO (Recovery Point Objective):** Cantidad máxima de datos que una organización está dispuesta a perder, medida en tiempo (ej. 1 hora de datos).

**RTO (Recovery Time Objective):** Tiempo máximo permitido para recuperar un servicio después de un desastre o interrupción.

**SABSA (Sherwood Applied Business Security Architecture):** Framework de arquitectura de seguridad que extiende Zachman con un enfoque en gestión de riesgos y seguridad de negocio.

**SAST (Static Application Security Testing):** Prueba de seguridad que analiza el código fuente en busca de vulnerabilidades (ej. inyección SQL, XSS).

**SBOM (Software Bill of Materials):** Inventario de todos los componentes, librerías y dependencias utilizados en un proyecto de software. Esencial para gestionar vulnerabilidades en la cadena de suministro.

**SCA (Software Composition Analysis):** Análisis de dependencias (librerías, frameworks) en busca de vulnerabilidades conocidas (CVE).

**Segmentación de red:** División de una red en subredes más pequeñas (VLANs) para aislar tráfico, limitar el movimiento lateral de atacantes y aplicar controles de seguridad específicos por zona.

**Seguridad por defecto (Secure by Default):** Principio que establece que las configuraciones predeterminadas deben ser las más seguras posibles.

**Seguridad por diseño (Security by Design):** Enfoque que integra la seguridad en cada etapa del ciclo de vida de un sistema o proceso desde su concepción.

**Separación de funciones (Segregation of Duties):** Principio que establece que las funciones críticas deben dividirse entre diferentes personas o sistemas para evitar conflictos de interés y fraudes.

**SIEM (Security Information and Event Management):** Sistema que recolecta, correlaciona y analiza eventos de seguridad de múltiples fuentes para detectar amenazas en tiempo real.

**SQL Injection:** Técnica de ataque que consiste en insertar código SQL malicioso a través de entradas de usuario no validadas, permitiendo al atacante leer, modificar o eliminar datos de la base de datos.

**STRIDE:** Marco de Microsoft para clasificar amenazas en seis categorías: Suplantación (S), Manipulación (T), Repudio (R), Revelación (I), Denegación (D) y Elevación (E).

**TDE (Transparent Data Encryption):** Tecnología de cifrado que cifra automáticamente los datos al escribirlos en disco y los descifra al leerlos, sin cambios en las aplicaciones.

**TOGAF (The Open Group Architecture Framework):** Framework de gobierno de arquitectura empresarial que proporciona un método (ADM) para el desarrollo y gestión de arquitecturas empresariales.

**Vulnerabilidad:** Debilidad o fallo en un activo que puede ser aprovechado por una amenaza.

**WAF (Web Application Firewall):** Solución de seguridad que filtra el tráfico HTTP/HTTPS hacia y desde una aplicación web, protegiendo contra ataques como SQL Injection, XSS y CSRF.

**Zachman Framework:** Taxonomía que proporciona una estructura para considerar la arquitectura empresarial desde múltiples perspectivas (datos, función, red, personas, tiempo, motivación).

**Zero Trust:** Modelo de seguridad que asume que ningún usuario, dispositivo o red es confiable por defecto, incluso si están dentro del perímetro de la organización. Requiere verificación continua de identidad y autorización para cada acceso.